LAPORAN TUGAS BESAR I

IF3170 INTELIGENSI ARTIFISIAL

Optimasi Pengisian Paket pada Truk Peti Kemas Menggunakan Algoritma Local Search

Disusun oleh:

Kelompok 22 - LKCJS

Aziza Dharma Putri 13524017

Suryani Mulia Utami 13524042

Alya Nur Rahmah 13524081

LABORATORIUM AI

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026/2027

DAFTAR ISI

BAB 1 DESKRIPSI PERSOALAN..... 1
1.1. Latar Belakang..... 1
1.2. Tujuan..... 2
1.3. Rumusan Permasalahan..... 2
1.4. State..... 4
1.5. Objective Function dan Constraint..... 4
1.6. Neighbor/Move..... 6
BAB 2 DASAR TEORI..... 8
2.1. Local Search..... 8
2.2. Hill Climbing..... 8
2.3. Simulated Annealing..... 11
2.4. Genetic Algorithm..... 12
BAB 3 DESAIN DAN IMPLEMENTASI..... 15
3.1. Struktur Program..... 15
3.2. Implementasi Hill-Climbing..... 16
3.3. Implementasi Simulated Annealing..... 26
3.4. Implementasi Genetic Algorithm..... 34
BAB 4 EKSPERIMEN..... 40
4. Metodologi Eksperimen..... 40
4.1.1. Lingkungan eksperimen..... 40
4.1.2. Data uji..... 40
4.1.3. Parameter algoritma..... 41
4.1.4. Jumlah Pecobaan..... 42
4.1.5. Data yang Dicatat..... 42
4.1.6. Menjaga perbandingan tetap adil..... 43
4.2. Hasil Hill Climbing..... 43
4.2.1. Steepest-Ascent Hill Climbing..... 44
4.2.2. Hill Climbing with Sideways Move..... 48
4.2.3. Stochastic Hill Climbing..... 52
4.2.4. Random-Restart Hill Climbing..... 57
4.2.5. Ringkasan Hasil Hill Climbing..... 61
4.3. Hasil Simulated Annealing..... 62
4.4. Hasil Genetic Algorithm..... 68
4.5. Hasil Multi-Truck..... 88
4.6. Analisis dan Perbandingan..... 90

BAB 5 KESIMPULAN DAN SARAN..... 92
5.1. Kesimpulan..... 92
5.2. Saran..... 92
DAFTAR PUSTAKA..... 93
LAMPIRAN..... 94

BAB 1
DESKRIPSI PERSOALAN

1.1. Latar Belakang

Pengiriman barang merupakan salah satu aspek penting dalam kegiatan logistik yang membutuhkan perencanaan agar proses distribusi dapat berjalan secara efektif. Salah satu permasalahan yang dihadapi dalam pengiriman barang menggunakan truk peti kemas adalah menentukan konfigurasi penempatan paket yang optimal. Setiap paket memiliki karakteristik yang berbeda, seperti dimensi, nilai, berat, orientasi, tingkat kerapuhan, dan estimasi waktu kedatangan (Estimated Time of Arrival atau ETA). Perbedaan karakteristik tersebut menyebabkan proses penempatan paket perlu mempertimbangkan berbagai batasan agar kapasitas peti kemas dapat dimanfaatkan secara optimal.

Permasalahan pengisian peti kemas tidak hanya berkaitan dengan banyaknya paket yang dapat dimasukkan, tetapi juga dengan nilai paket yang berhasil diangkut. Konfigurasi penempatan yang dihasilkan harus memenuhi sejumlah batasan, antara lain paket tidak boleh melayang, tidak boleh menembus dinding peti kemas, tidak boleh bertumpang tindih dengan paket lain, dan tidak boleh melebihi kapasitas berat maksimum truk. Selain itu, paket yang mudah pecah tidak diperbolehkan menjadi penopang bagi paket lain. Berbagai batasan tersebut membuat pencarian konfigurasi optimal menjadi permasalahan optimasi dengan ruang solusi yang beragam.

Untuk menyelesaikan permasalahan tersebut, diperlukan pendekatan yang dapat mengeksplorasi berbagai kemungkinan konfigurasi penempatan paket. Salah satu pendekatan yang dapat digunakan adalah local search, yaitu metode pencarian yang mengevaluasi solusi berdasarkan nilai fungsi objektif dan melakukan perubahan terhadap solusi saat ini untuk menemukan konfigurasi yang lebih baik. Dalam penerapannya, konfigurasi penempatan paket dapat diubah melalui operasi pertukaran posisi, pemindahan paket ke koordinat lain, dan perubahan orientasi paket. Setiap konfigurasi yang dihasilkan perlu diperiksa agar tetap memenuhi batasan yang berlaku.

Pada Tugas Besar I IF3170 Inteligensi Artifisial, permasalahan optimasi pengisian paket pada truk peti kemas diselesaikan menggunakan tiga algoritma, yaitu Hill Climbing, Simulated Annealing, dan Genetic Algorithm. Ketiga algoritma tersebut diimplementasikan untuk mencari konfigurasi yang

memaksimalkan nilai paket yang diangkut berdasarkan fungsi objektif yang ditetapkan. Kinerja algoritma kemudian dievaluasi melalui eksperimen dengan memperhatikan nilai fungsi objektif yang diperoleh, durasi pencarian, serta perubahan nilai fungsi objektif selama proses pencarian. Melalui implementasi dan evaluasi tersebut, dapat dianalisis karakteristik serta perbandingan kinerja setiap algoritma dalam menyelesaikan permasalahan optimasi pengisian peti kemas.

1.2. Tujuan

Mengimplementasikan dan mengevaluasi algoritma-algoritma local search untuk menyelesaikan permasalahan optimasi pengisian paket pada truk peti kemas. Tugas ini bertujuan untuk membuat konfigurasi pengisian paket paling optimal untuk sebuah truk peti kemas. Optimal disini berarti memaksimalkan fungsi objektif.

1.3. Rumusan Permasalahan

Berikut adalah masukan dan keluaran dari permasalahan optimasi pengisian paket pada truk peti kemas.

Masukan	`(jumlah_paket)`
	`(id w d h [x y z] value weight isFragile eta)+`
	`(wp dp hp maxCapacity)`
	
	dengan
	• `id` = *identifier* paket
	• `w` = *width* paket
	• `d` = *depth* paket
	• `h` = *height* paket
	• `x y z` = orientasi paket (terdapat 6 kemungkinan lain, misal x z y. Bersifat opsional. Secara default digunakan x y z.
	• `value` = nilai harga sebuah paket (satuan unit)
	• `weight` = berat paket (satuan unit berat)
	• `isFragile` = benda pecah-belah

	• `eta` = *estimated Time of Arrival*, perkiraan waktu kedatangan paket (dalam sisa hari) • `wp` = *width* peti • `dp` = *depth* peti • `hp` = *height* peti • `maxCapacity` = kapasitas angkut maksimum pada suatu peti kemas (satuan unit berat)
Keluaran	• Algoritma yang digunakan • Initial dan final state • Initial, final, dan history dari value • Banyak iterasi/generasi • Jumlah accepted moves (kecuali GA) • Lama waktu (detik) program berjalan • Alasan program berhenti

Berikut adalah representasi dari permasalahan ini:

- Paket hanya berbentuk kubus / balok. Paket dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada truk sehingga suatu paket dapat berada di luar (seluruh bagian paket di luar)
- Koordinat berupa bilangan bulat. Representasinya sebagai berikut: Peti kemas memiliki titik pojok kiri-bawah-depan peti (0,0,0) dan paket memiliki titik pojok kiri-bawah-depan paket (x,y,z).
- Struktur data yang digunakan untuk representasi menggunakan *key-value*
- Inisialisasi (*initial state*) random.
- Langkah neighbor (*move*) yang diperbolehkan tiap iterasi:
    - Menukar dua paket, baik di dalam maupun di luar truk
    - Memindahkan sebuah paket ke suatu koordinat yang berbeda
    - Merotasi paket 90° terhadap sumbu x, y, atau z

- Representasi solusi (*final state*) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada truk. Secara detail yang dapat ditunjukkan oleh state dan visualisasi adalah:
    - Bagaimana keadaan semua paket yang di dalam maupun yang di luar
    - Bagaimana keadaan peti kemas pada truk
    - Value pada state tersebut

1.4. State

State dalam konteks ini adalah posisi dari setiap objek paket yang ada. Diasumsikan peti kemas memiliki titik pojok kiri-bawah-depan peti (0,0,0) dan paket memiliki titik pojok kiri-bawah-depan paket (x,y,z).

Setiap paket juga memiliki dimensi (w,d,h) yang dapat berubah sesuai dengan orientasi paket. Dimensi efektif merupakan ukuran paket pada masing-masing sumbu koordinat (x,y,z) setelah orientasi paket diterapkan. Dimensi efektif ditentukan berdasarkan pemetaan orientasi paket terhadap sumbu koordinat, yaitu:

`getDimOri(p) = (width_x, length_y, height_z)`

Saat menginisiasi state, tiap-tiap paket akan diletakkan secara random (dapat di dalam maupun di luar peti), dengan cara sebagai berikut:

a. Randomisasi nilai (x, y, z) dengan *range*

`([0…wp + w], [0…dp + d], [0…hp + h])`

b. Lakukan pengecekan apakah posisi tersebut merupakan posisi yang valid untuk paket, yaitu memenuhi *constraint* tertentu.
    - Apabila posisi tersebut valid, maka paket akan dimasukkan ke posisi (menyusun *partial state*)
    - Apabila tidak, kembali ke *step* a.

Maka misal terdapat 3 paket (p1, p2, p3) yang akan diletakkan. Representasi state adalah [p1, p2, p3] dimana p1, p2, dan p3 memiliki domain

$$x \in [0, w_p + w], y \in [0, d_p + d], z \in [0, h_p + h]$$

1.5. Objective Function dan Constraint

Terdapat dua fungsi objektif yang digunakan. Keduanya bertujuan untuk memaksimalkan keuntungan.

1. Memaksimalkan value peti kemas

   Rumus yang digunakan untuk fungsi objektif ini adalah

   totalValue += p.value

   untuk setiap paket p yang berada di dalam peti kemas.

2. Memaksimalkan value peti kemas berbobot urgensi peti kemas

   - Nilai peti berbanding lurus dengan value peti
   - Urgency suatu peti berbanding terbalik dengan ETA

     Jika ETA = 0 => urgensi = 1, sehingga digunakan skor urgensi

     1 / (1 + ETA)

   Maka, rumus yang digunakan untuk fungsi objektif ini adalah

   totalValue += p.value / (1 + p.eta)

   untuk setiap paket p yang berada di dalam peti kemas.

Terdapat beberapa *constraint* dalam menempatkan paket sebagai berikut:

1. Paket tidak bisa melayang

   Artinya, paket *p* harus ditopang oleh lantai peti kemas

   z==0

   atau oleh paket lain yang berada tepat di bawahnya.

   isSupport(other, p) == True

2. Paket tidak bisa menembus dinding peti kemas

   Artinya paket p hanya dapat berada di dalam peti (*totally inside*) atau di luar peti (*totally outside*)

   isInsidePeti(p) or isOutsidePeti(p)

3. Paket tidak bisa menembus paket lainnya (*overlapping*)

   Artinya paket p1 dan paket p2 overlap sumbu x, sumbu y, dan sumbu z secara bersamaan.

   Overlap dinyatakan dengan

   `(start1 < end2 and start2 < end1)`

   Maka,

   overlap_sb_x = (x1 < x2+w2 and x2 < x1+w1)

```
overlap_sb_y = (y1 < y2+d2 and y2 < y1+d1)
overlap_sb_z = (z1 < z2+h2 and z2 < z1+h1)
(overlap_sb_x and overlap_sb_y and overlap_sb_z) == false
```

4. Barang pecah belah tidak boleh menjadi support

   Artinya, apabila paket p merupakan barang pecah belah, paket tersebut tidak boleh menopang paket lain.

   ```
   (isSupport(p, other) and p.isFragile) == false
   ```

5. Total berat paket yang diangkut tidak boleh melebihi kapasitas truk

   ```
   totalWeight <= maxCapacity
   ```

   di mana

   ```
   totalWeight += p.weight
   ```

   untuk setiap paket p yang berada di dalam peti kemas.

1.6. Neighbor/Move

Neighbor merupakan state baru yang diperoleh dari state awal melalui perubahan posisi atau orientasi paket. Terdapat tiga operasi yang digunakan untuk menghasilkan neighbor, yaitu sebagai berikut:

1. Menukar posisi dua paket (*swap*)

   Operasi swap dilakukan dengan menukar posisi dua paket yang berbeda, sedangkan posisi paket lainnya tetap.

   Misalkan p1 dan p2 adalah dua paket yang berbeda, dan posisi paket p1 dan p2 semula adalah (x1, y1, z1) dan (x2, y2, z2). Setelah operasi swap, posisi kedua paket menjadi:

   ```
   positions[p1.id] = (x2, y2, z2)
   positions[p2.id] = (x1, y1, z1)
   ```

2. Memindahkan posisi paket (*move*)

   Operasi move dilakukan dengan memindahkan satu paket ke koordinat baru, sedangkan posisi dan orientasi paket lainnya tetap.

   Posisi baru paket ditentukan dalam bentuk koordinat tiga dimensi dengan nilai bilangan bulat.

   ```
   positions[package_id] = (x, y, z)
   ```

3. Memutar orientasi paket (*rotate*)

   Operasi rotate dilakukan dengan mengubah orientasi satu paket sebesar 90 derajat terhadap salah satu sumbu, yaitu sumbu x, y, atau z. Posisi paket tetap, tetapi orientasi dan dimensi efektifnya dapat berubah.

   Perubahan orientasi dilakukan dengan menukar pemetaan sumbu sebagai berikut:

   - Rotasi terhadap sumbu x: sumbu y dan z dipertukarkan.
   - Rotasi terhadap sumbu y: sumbu x dan z dipertukarkan.
   - Rotasi terhadap sumbu z: sumbu x dan y dipertukarkan.

Setelah salah satu operasi dilakukan, state baru akan diperiksa menggunakan fungsi isValid() untuk memastikan bahwa seluruh constraint penempatan paket tetap terpenuhi. Apabila state baru valid, state tersebut dikembalikan sebagai neighbor. Jika tidak valid, neighbor tidak dihasilkan.

Neighbor dapat dibangkitkan melalui dua cara:

- Pembangkitan menyeluruh: menghasilkan seluruh neighbor valid melalui operasi *swap*, *move*, dan *rotate*.
- Pembangkitan acak: memilih operasi dan paket secara acak untuk menghasilkan satu neighbor valid. Proses ini dibatasi hingga maksimal 100 percobaan secara default. Jika tidak ditemukan neighbor valid, fungsi mengembalikan `None`.

BAB 2
DASAR TEORI

2.1. Local Search

Local Search merupakan metode pencarian yang bertujuan menemukan state terbaik berdasarkan nilai fungsi objektif. Berbeda dengan *classical search* yang mengeksplorasi ruang pencarian secara sistematis dan berfokus pada jalur menuju solusi, *local search* hanya menyimpan satu state saat ini (*current state*) dan berpindah ke state lain yang merupakan tetangganya.

Setiap state merepresentasikan konfigurasi lengkap dari suatu permasalahan dan memiliki nilai (*state value*) yang digunakan untuk mengevaluasi kualitasnya. Proses pencarian dilakukan dengan membangkitkan kandidat state melalui fungsi *successor*, kemudian memilih state tetangga (*neighbor*) untuk melanjutkan pencarian. Tujuannya adalah memperoleh state dengan nilai fungsi objektif maksimum/minimum tergantung representasi permasalahan.

Kumpulan state yang dapat dicapai dari state saat ini melalui suatu operasi atau perubahan tertentu disebut *neighborhood*. Dalam konteks optimasi penempatan paket, *neighborhood* dapat dibentuk melalui operasi *swap* untuk menukar posisi dua paket, *move* untuk memindahkan posisi satu paket, dan *rotate* untuk mengubah orientasi satu paket. Setiap kandidat state yang dihasilkan kemudian diperiksa validitasnya berdasarkan *constraint* yang berlaku sebelum dapat digunakan dalam proses pencarian.

Kualitas neighbor dan cara pemilihannya bergantung pada algoritma *local search* yang digunakan. Pemilihan neighbor yang tepat memungkinkan algoritma mengeksplorasi ruang solusi untuk mencari konfigurasi penempatan paket yang memberikan nilai objektif terbaik.

2.2. Hill Climbing

Hill Climbing adalah algoritma *local search* yang paling sederhana. Pencarian dimulai dari sebuah state awal yang dibangkitkan secara acak. Setelah itu, algoritma berulang kali berpindah ke neighbor yang memiliki nilai fungsi objektif lebih tinggi (untuk persoalan maksimisasi). Pencarian berhenti ketika tidak ada lagi neighbor yang lebih baik daripada *state* saat ini, artinya algoritma sudah mencapai sebuah "puncak". Algoritma ini tidak menyimpan

pohon pencarian maupun jalur menuju solusi. Yang disimpan hanya *current state* beserta nilai objektifnya. Russell dan Norvig menggambarkan proses ini sebagai "*like climbing Everest in thick fog with amnesia*": algoritma hanya melihat sekelilingnya dan tidak mengingat langkah yang sudah diambil.

Hill Climbing bersifat *greedy* karena hanya mau berpindah ke state yang lebih baik dan tidak pernah menerima langkah "turun". Sifat ini membuatnya efisien, tetapi hasil akhirnya sangat bergantung pada state awal. Ada tiga kondisi yang membuat algoritma berhenti sebelum mencapai *global maximum*:

- **Local maximum**: puncak yang nilainya lebih tinggi dari semua tetangganya, tetapi lebih rendah dari global maximum.
- **Flat local maximum (plateau)**: daerah datar tempat semua neighbor bernilai sama dan tidak ada jalan naik.
- **Shoulder**: daerah datar yang sebenarnya masih bisa menuju nilai yang lebih tinggi, tetapi tidak bisa dilewati karena algoritma tidak mau berpindah ke neighbor yang bernilai sama.

Untuk mengatasi kelemahan tersebut, terdapat beberapa varian Hill Climbing.

a. Steepest-Ascent Hill Climbing

Pada setiap iterasi, algoritma membangkitkan seluruh *successor* valid dari current state melalui operasi *swap, move*, dan *rotate*. Algoritma lalu memilih successor dengan nilai objektif tertinggi sebagai neighbor. Jika ada lebih dari satu successor dengan nilai tertinggi yang sama, salah satunya dipilih secara acak. Pencarian berhenti ketika nilai neighbor tidak lebih besar dari nilai current state.

```
current ← initial state
loop do
   neighbor ← successor dengan nilai tertinggi dari current
   if neighbor.value ≤ current.value then return current
   current ← neighbor
```

Varian ini mengambil langkah terbaik di setiap iterasi, sehingga biasanya cepat mencapai puncak dalam jumlah iterasi yang sedikit. Namun, biaya satu iterasinya besar karena seluruh successor harus dibangkitkan dan dievaluasi. Pada persoalan pengisian peti kemas, jumlah successor bertambah seiring banyaknya paket, pasangan paket yang bisa ditukar, dan koordinat tujuan yang mungkin.

b. Hill Climbing with Sideways Move

Varian ini mengubah kondisi berhenti pada Steepest-Ascent dari neighbor.value ≤ current.value menjadi neighbor.value < current.value. Dengan begitu, algoritma boleh berpindah ke neighbor yang nilainya sama dengan current state (*sideways move*), sehingga bisa melewati daerah *shoulder*.

Masalahnya, sideways move juga bisa membuat algoritma berputar tanpa akhir di flat local maximum. Karena itu, jumlah sideways move berturut-turut dibatasi dengan parameter maximum sideways move. Penghitungnya di-reset ke nol setiap kali ditemukan neighbor yang lebih baik. Pencarian berhenti jika tidak ada neighbor yang lebih baik atau sama, atau jika batas sideways move berturut-turut sudah tercapai. Sebagai gambaran, pada persoalan 8-queens, batas 100 sideways move meningkatkan tingkat keberhasilan dari 14% menjadi 94%. Konsekuensinya, jumlah langkah bertambah: rata-rata 21 langkah saat berhasil dan 64 langkah saat gagal.

c. Random-Restart Hill Climbing

Varian ini menerapkan prinsip "*if at first you don't succeed, try, try again*". Algoritma menjalankan serangkaian pencarian Hill Climbing (dalam implementasi ini Steepest-Ascent), masing-masing dimulai dari state awal acak yang berbeda. State terbaik dari seluruh pencarian dipilih sebagai hasil akhir.

Banyak restart dibatasi dengan parameter maximum restart. Jika peluang satu kali pencarian mencapai global maximum adalah p, maka:

- Perkiraan banyak restart yang dibutuhkan adalah 1/p.
- Perkiraan jumlah langkah totalnya adalah s + f(1 − p)/p, dengan s adalah rata-rata langkah saat berhasil dan f adalah rata-rata langkah saat gagal.

Restart dari titik acak membuat algoritma tidak terlalu bergantung pada satu state awal. Konsekuensinya, waktu yang dibutuhkan naik kira-kira sebanding dengan banyak restart.

d. Stochastic Hill Climbing

Berbeda dengan Steepest-Ascent yang mengevaluasi seluruh successor, Stochastic Hill Climbing hanya membangkitkan satu

successor acak pada setiap iterasi. Algoritma berpindah ke successor tersebut hanya jika nilainya lebih tinggi daripada current state. Jika nilainya sama atau lebih buruk, successor tersebut dilewati. Pencarian berhenti setelah mencapai batas jumlah iterasi (nmax).

```
current ← initial state
repeat nmax times
   neighbor ← successor acak dari current
   if neighbor.value > current.value then current ← neighbor
return current
```

Biaya satu iterasinya jauh lebih kecil daripada Steepest-Ascent, tetapi jumlah iterasi yang dibutuhkan untuk mencapai puncak lebih banyak, karena tidak setiap iterasi menghasilkan perbaikan. Stochastic Hill Climbing juga menjadi dasar dari Simulated Annealing. Perbedaannya, Simulated Annealing masih mengizinkan perpindahan ke neighbor yang lebih buruk dengan probabilitas tertentu.

2.3. Simulated Annealing

Simulated Annealing (SA) merupakan algoritma local search yang terinspirasi dari proses annealing dalam metalurgi, yaitu proses pemanasan dan pendinginan material untuk memperoleh struktur yang lebih stabil. Dalam optimasi, konsep ini diterapkan dengan mengizinkan perpindahan menuju state yang memiliki nilai objektif lebih buruk dengan probabilitas tertentu. Mekanisme tersebut membantu algoritma menghindari jebakan optimum lokal yang dapat membatasi pencarian pada algoritma Hill Climbing.

Pada setiap iterasi, algoritma menghasilkan neighbor, yaitu state lain yang diperoleh melalui perubahan terhadap state saat ini. Selanjutnya, algoritma mengevaluasi nilai fungsi objektif dari neighbor tersebut dan menentukan apakah perpindahan akan diterima. Untuk masalah optimasi maksimisasi, perubahan nilai objektif dapat dinyatakan sebagai:

$$\Delta E = f(S') - f(S)$$

dengan S sebagai state saat ini, S' sebagai neighbor, dan $f(S)$ sebagai nilai fungsi objektif dari suatu state. Keputusan penerimaan neighbor dilakukan berdasarkan aturan berikut.

1. Jika $\Delta E > 0$, neighbor memiliki nilai objektif yang lebih baik sehingga perpindahan diterima.

2. Jika $\Delta E = 0$, neighbor memiliki nilai objektif yang sama. Keputusan penerimaannya mengikuti kebijakan yang digunakan dalam implementasi.
3. Jika $\Delta E < 0$, neighbor memiliki nilai objektif yang lebih buruk.

Perpindahan tetap dapat diterima dengan probabilitas:

$$P = e^{\Delta E/T}$$

dengan P sebagai probabilitas penerimaan dan T sebagai temperatur saat iterasi berlangsung, dengan $T > 0$.

Sebagai contoh, jika nilai objektif state saat ini adalah 10 dan nilai objektif neighbor adalah 8, maka $\Delta E = -2$. Pada temperatur $T = 10$, probabilitas penerimaannya adalah:

$$P = e^{-2/10} \approx 0.819$$

Artinya, solusi yang lebih buruk tersebut memiliki peluang sekitar 81,9% untuk diterima. Ketika temperatur menurun, peluang penerimaan untuk penurunan nilai objektif yang sama juga semakin kecil.

2.4. Genetic Algorithm

Genetic Algorithm (GA) merupakan metode *local search* yang menggunakan pendekatan populasi untuk mencari solusi. GA memproses beberapa state secara paralel. Setiap state disebut individu.

GA dimulai dengan membentuk populasi awal yang terdiri dari *nPopulation* (sesuai parameter algoritma) individu yang dibangkitkan secara acak. Kualitas setiap individu dinilai menggunakan *fitness function*, yaitu fungsi evaluasi yang memberikan nilai lebih tinggi untuk solusi yang lebih baik. Dalam hal ini, fungsi evaluasi yang digunakan adalah fungsi objektif yang didefinisikan pada bab 1.

Selanjutnya, GA membentuk generasi baru melalui tiga proses utama:

1. Seleksi (*selection*): memilih individu yang akan menjadi induk (*parent*) secara acak dengan probabilitas yang sebanding dengan nilai *fitness*-nya. Individu dengan *fitness* lebih tinggi memiliki peluang lebih besar untuk terpilih.

2. *Crossover*: menggabungkan informasi posisi paket dari dua individu induk untuk menghasilkan individu baru. Mekanismenya hampir sama dengan inisialisasi *state*, tetapi posisi setiap paket diambil dari kedua *parent*, bukan langsung dibangkitkan secara acak.

   Untuk setiap paket, kandidat posisi dibentuk sebagai berikut:

```
candidatePos = [
    parent1.getPosition(package),
    parent2.getPosition(package)
]
self.rng.shuffle(candidatePos)
```

   Selanjutnya, kandidat posisi dicoba berdasarkan urutan yang telah diacak untuk membentuk *partial state*

   - Apabila posisi paket menghasilkan *partial state* yang valid, posisi tersebut akan dipilih dan ditambahkan ke *partial state*.
   - Apabila posisi dari kedua *parent* tidak menghasilkan *partial state* yang valid, posisi *random* akan dicoba hingga ditemukan posisi yang valid, kemudian ditambahkan ke *partial state*.

   Proses ini dilakukan untuk setiap paket hingga seluruh paket memiliki posisi dan terbentuk individu (*state*) baru. Di mana posisi paket dalam individu baru dapat berasal dari *parent* pertama, *parent* kedua, ataupun posisi *random*.

3. Mutasi (*mutation*): mengubah individu untuk menghasilkan variasi solusi. Akan dipilih titik mutasi (*mutation point*), yaitu paket mana secara acak dan diterapkan salah satu operasi neighbor (*move, rotate,* atau *swap*) yang juga dipilih secara acak. Operasi yang dipilih dicoba hingga ditemukan perubahan yang menghasilkan state valid, dengan jumlah percobaan sesuai mekanisme masing-masing operasi. Jika seluruh percobaan gagal menghasilkan state valid, state asli dikembalikan tanpa perubahan.

Hasil dari proses-proses ini digunakan dalam pembentukan populasi generasi berikutnya. Proses tersebut diulang hingga mencapai jumlah generasi (*nGeneration*) yang ditentukan sebagai parameter algoritma.

BAB 3
DESAIN DAN IMPLEMENTASI

3.1. Struktur Program

Struktur direktori program IF3170_G22_LKCJS ditunjukkan pada *tree* berikut.

```
IF3170_G22_LKCJS/
├── docs/
├── input/
├── output/
├── src/
│   ├── algorithm/
│   │   ├── genetic_algorithm/
│   │   │   ├── genetic_algorithm.py
│   │   │   └── result.py
│   │   ├── hill_climbing/
│   │   │   ├── base.py
│   │   │   ├── random_restart.py
│   │   │   ├── result.py
│   │   │   ├── sideways_move.py
│   │   │   ├── steepest_ascent.py
│   │   │   └── stochastic.py
│   │   └── simulated_annealing/
│   │       ├── result.py
│   │       └── simulated_annealing.py
│   ├── core/
│   │   ├── entity.py
│   │   ├── neighbor.py
│   │   ├── fleet.py
│   │   └── state.py
│   ├── util/
│   │   ├── fileHelper.py
│   │   ├── parser.py
```

```
│   │   └── stateHelper.py
│   ├── visualization/
│   │   ├── draw_3d.py
│   │   ├── draw_top_view.py
│   │   ├── figure.py
│   │   ├── geometry.py
│   │   ├── package_table.py
│   │   ├── style.py
│   │   └── summary.py
│   ├── main.py
│   ├── run_genetic_algorithm.py
│   ├── run_hill_climbing.py
│   ├── run_simulated_annealing.py
│   ├── run_visual_scenarios.py
│   └── run_visualization.py
├── tests/
│   ├── test_hill_climbing.py
│   ├── test_multi_truck.py
│   ├── test_neighbor.py
│   ├── test_simulated_annealing.py
│   └── test_visualization.py
└── README.md
```

3.2. Implementasi Hill-Climbing

Seluruh varian Hill-Climbing diimplementasikan di folder src/algorithm/hill_climbing/. Setiap varian adalah turunan dari kelas HillClimbingBase, yang berisi fungsi pembantu bersama: penyalinan state, pemilihan successor terbaik, dan pembangkitan state awal acak. Hasil setiap varian disimpan dalam format yang sama, yaitu HillClimbingResult.

Fungsi/Kelas	Deskripsi

`HillClimbingBase`	Kelas induk semua varian HC. Menyimpan seed dan generator bilangan acak (rng).
`_clone()`	Membuat salinan (*deep copy*) state agar perubahan pada neighbor tidak mengubah state asal.
`_best_neighbor()`	Membangkitkan seluruh successor valid dan mengembalikan successor dengan nilai objektif tertinggi. Jika ada beberapa successor terbaik dengan nilai sama, salah satunya dipilih secara acak.
`_random_initial_state()`	Membangkitkan state awal acak baru untuk Random-Restart HC.
`Neighbor.iter_all()`	Membangkitkan seluruh *successor* valid dari operasi *swap, move*, dan *rotate*. Dipakai oleh Steepest-Ascent, Sideways Move, dan Random-Restart.
`Neighbor.random_neighbor()`	Membangkitkan satu *successor* valid secara acak. Dipakai oleh Stochastic HC.
`SteepestAscentHillClimbing`	Implementasi Steepest-Ascent Hill-Climbing.
`RandomRestartHillClimbing`	Implementasi Random-Restart Hill-Climbing (memakai Steepest-Ascent di setiap restart).
`SidewaysMoveHillClimbing`	Implementasi Hill-Climbing with Sideways Move.
`StochasticHillClimbing`	Implementasi Stochastic Hill-Climbing.
`HillClimbingResult`	Menyimpan state awal dan akhir, jumlah iterasi, jumlah accepted moves, riwayat nilai, durasi, alasan berhenti, parameter, dan metadata.

`run()`	Menjalankan pencarian dan mengembalikan HillClimbingResult.
`max_sideways`	Batas sideways move berturut-turut pada Sideways Move HC.
`max_restarts`	Banyak restart setelah run pertama pada Random-Restart HC, sehingga total run maksimum adalah max_restarts + 1.
`target_value`	Nilai target. Random-Restart berhenti lebih awal jika nilai ini tercapai.
`max_iterations`	Batas iterasi (nmax) pada Stochastic HC.
`max_neighbor_attempts`	Batas percobaan mencari satu neighbor valid secara acak pada Stochastic HC.
`seed`	Mengatur generator bilangan acak agar hasil dapat diulang.

Berikut adalah alur implementasi dari algortima Hill-Climbing:

1. Pemilihan successor terbaik

File : src/algorithm/hill_climbing/base.py

```python
    def _best_neighbor(self, state: State) -> State | None:
        best_value: int | None = None
        best_states: list[State] = []

        for neighbor in Neighbor.iter_all(state):
            value = neighbor.getValue()

            if best_value is None or value > best_value:
                best_value = value
                best_states = [neighbor]
            elif value == best_value:
                best_states.append(neighbor)

        if not best_states:
            return None
```

```
        return self.rng.choice(best_states)
```

Fungsi ini dipakai oleh Steepest-Ascent, Sideways Move, dan Random-Restart. Seluruh *successor* valid dibangkitkan melalui Neighbor.iter_all(), lalu dicatat *successor* dengan nilai objektif tertinggi. Jika ada beberapa *successor* dengan nilai tertinggi yang sama, salah satunya dipilih secara acak, sesuai materi kuliah. Jika tidak ada successor valid sama sekali, fungsi mengembalikan None.

2. Pembangkitan seluruh successor

File : src/core/neighbor.py

```
    @staticmethod
    def iter_all(state: State) -> Iterator[State]:
        yield from Neighbor.iter_swap_neighbors(state)
        yield from Neighbor.iter_move_neighbors(state)
        yield from Neighbor.iter_rotation_neighbors(state)
```

```
    def iter_move_neighbors(state: State) -> Iterator[State]:
        extent_w, extent_d, extent_h = state.extent

        for package in state.packages:
            package_w, package_d, package_h = state.getDimOri(package)
            current_position = state.getPosition(package)

            max_x = extent_w + package_w
            max_y = extent_d + package_d
               max_z = extent_h + package_h

            others = state.otherBoxes(package)
            support_heights = {0}
            for _, _, _, other_z, _, _, other_h in others:
                if other_z + other_h <= max_z:
                    support_heights.add(other_z + other_h)

            for x in range(max_x + 1):
                for y in range(max_y + 1):
                    for z in sorted(support_heights):
                        new_position = (x, y, z)

                        if new_position == current_position:
                                    continue
```

```
                candidate = Neighbor._try_move(
                    state,
                    package,
                    new_position,
                    others
                )

                if candidate is not None:
                    yield candidate
```

Successor dibangkitkan dari tiga operasi:

- Swap: setiap pasangan paket ditukar posisinya.
- Move: setiap paket dicoba dipindahkan ke semua koordinat dalam domain.
- Rotate: setiap paket dicoba dirotasi terhadap sumbu x, y, dan z.

Setiap kandidat diperiksa dengan isValid(), dan hanya kandidat valid yang dihasilkan. Successor dibangkitkan secara bertahap dengan yield (generator), sehingga tidak perlu menyimpan semua successor di memori sekaligus.

Heuristik yang digunakan pada operasi move: koordinat z hanya dicoba pada ketinggian yang mungkin memiliki penopang, yaitu lantai (z = 0) atau tepat di atas permukaan paket lain. Paket pada ketinggian selain itu pasti melayang dan tidak valid. Dengan begitu, himpunan successor yang dihasilkan tetap sama, tetapi jumlah pengecekan berkurang drastis.

3. Steepest-Ascent Hill-Climbing

File : src/algorithm/hill_climbing/steepest_ascent.py

```
        while True:
              iterations += 1
           current_value = current.getValue()
           best = self._best_neighbor(current)

           if best is None:
               stop_reason = "no_valid_neighbor"
               value_history.append(current_value)
               break

           best_value = best.getValue()

           if best_value <= current_value:
```

```
                stop_reason = (
                    "flat_local_optimum"
                    if best_value == current_value
                    else "local_optimum"
                )
                value_history.append(current_value)
                break

            current = best
            accepted_moves += 1
            value_history.append(current.getValue())
```

Pada setiap iterasi, algoritma mengambil successor terbaik dari current state. Jika nilainya lebih tinggi, algoritma berpindah ke successor tersebut. Jika tidak, pencarian berhenti. Ini sesuai dengan kondisi neighbor.VALUE ≤ current.VALUE pada pseudocode materi kuliah.

Alasan berhenti dibedakan menjadi tiga:
- local_optimum: semua neighbor lebih buruk.
- flat_local_optimum: neighbor terbaik bernilai sama dengan current state.
- no_valid_neighbor: tidak ada neighbor valid sama sekali.

4. Hill-Climbing with Sideways Move

File : src/algorithm/hill_climbing/sideways_move.py

```
    def __init__(self, max_sideways: int = 100, seed: int | None = None):
        super().__init__(seed=seed)
        if max_sideways < 0:
            raise ValueError("max_sideways tidak boleh negatif.")
        self.max_sideways = max_sideways
```

```
            if best_value < current_value:
                stop_reason = "local_optimum"
                value_history.append(current_value)
                break

            if best_value == current_value:
                if consecutive_sideways >= self.max_sideways:
                    stop_reason = "max_sideways_reached"
                    value_history.append(current_value)
                    break

                current = best
```

```
            consecutive_sideways += 1
            total_sideways_moves += 1
            accepted_moves += 1
            value_history.append(current.getValue())
            continue

        #best_value > current_value
        current = best
        consecutive_sideways = 0
        accepted_moves += 1
        value_history.append(current.getValue())
```

Alurnya sama dengan Steepest-Ascent, tetapi kondisi berhentinya diubah menjadi best_value < current_value. Jika neighbor terbaik bernilai sama, algoritma tetap berpindah (sideways move) selama jumlah sideways move berturut-turut belum mencapai max_sideways. Penghitung consecutive_sideways di-reset ke 0 setiap kali ditemukan neighbor yang lebih baik.

Jika batas tercapai, pencarian berhenti dengan alasan max_sideways_reached. Jumlah total sideways move dicatat pada metadata sideways_moves.

5. Random-Restart Hill-Climbing

File : src/algorithm/hill_climbing/random_restart.py

```
def _run_steepest(self, state: State) -> HillClimbingResult:
    #Seed berbeda per run tetapi tetap reproducible dari seed utama.
    child_seed = self.rng.randrange(0, 2**32)
    return SteepestAscentHillClimbing(seed=child_seed).run(state)
```

```
      for run_index in range(self.max_restarts + 1):
        run_result = self._run_steepest(current_initial)
        run_results.append(run_result)
        restart_histories.append(run_result.value_history)
        iterations_per_restart.append(run_result.iterations)

        if (
            self.target_value is not None
            and run_result.final_value >= self.target_value
        ):
            stop_reason = "target_value_reached"
            break
```

```
            if run_index == self.max_restarts:
                break

            current_initial = self._random_initial_state(initial_state)
            actual_restarts += 1

        best_index = max(
            range(len(run_results)),
            key=lambda index: run_results[index].final_value,
         )
        best = run_results[best_index]
```

Run pertama dimulai dari state awal yang diberikan. Setelah itu, algoritma membangkitkan state awal acak baru melalui _random_initial_state() dan menjalankan Steepest-Ascent lagi, sampai max_restarts restart tercapai. Pencarian juga bisa berhenti lebih awal jika target_value tercapai.

Setiap run memakai seed turunan dari seed utama, sehingga keseluruhan proses tetap dapat diulang. State akhir yang dikembalikan adalah hasil terbaik dari seluruh run.

Data yang dicatat pada metadata:

- Banyak restart (restarts)
- Banyak iterasi per restart (iterations_per_restart)
- Nilai akhir tiap restart (final_values_per_restart)
- Riwayat nilai tiap restart (restart_histories)

Untuk plot, riwayat nilai semua restart digabung berurutan, sehingga titik restart terlihat sebagai penurunan nilai pada grafik.

6. Stochastic Hill-Climbing

File : src/algorithm/hill_climbing/stochastic.py

```
        for _ in range(self.max_iterations):
            iterations += 1

            candidate = Neighbor.random_neighbor(
                current,
                rng=self.rng,
                max_attempts=self.max_neighbor_attempts,
```

```
            )

            if candidate is None:
                stop_reason = "no_valid_neighbor"
                value_history.append(current.getValue())
                break

            if candidate.getValue() > current.getValue():
                current = candidate
                accepted_moves += 1
            else:
                rejected_moves += 1

            value_history.append(current.getValue())
```

File : src/core/neighbor.py

```
        for _ in range(max_attempts):
            operation = rng.choice(operations)

            if operation == "swap":
                p1, p2 = rng.sample(state.packages, 2)
                candidate = Neighbor.swap(state, p1.id, p2.id)

            elif operation == "rotate":
                package = rng.choice(state.packages)
                axis = rng.choice(("x", "y", "z"))
                candidate = Neighbor.rotate(state, package.id, axis)

            else:
                    package = rng.choice(state.packages)
                package_w, package_d, package_h = state.getDimOri(package)
                new_position = (
                    rng.randint(0, truck_w + package_w),
                    rng.randint(0, truck_d + package_d),
                    rng.randint(0, truck_h + package_h),
                )
                candidate = Neighbor.move(state, package.id, new_position)

            if candidate is not None:
                return candidate

        return None
```

Pada setiap iterasi, Neighbor.random_neighbor() memilih satu operasi dan paket secara acak. Proses ini diulang hingga

menghasilkan satu neighbor valid, dengan batas max_neighbor_attempts percobaan.

Algoritma hanya berpindah jika neighbor tersebut lebih baik (>). Neighbor yang nilainya sama atau lebih buruk dilewati dan dicatat sebagai rejected_moves. Pencarian berhenti setelah max_iterations iterasi (nmax), atau jika neighbor valid tidak ditemukan.

7. Menyimpan hasil

File : src/algorithm/hill_climbing/result.py

```python
@dataclass
class HillClimbingResult:
    #Format hasil yang seragam untuk seluruh varian Hill-Climbing.

    algorithm: str
    initial_state: State
    final_state: State
    iterations: int
    accepted_moves: int
    value_history: list[int]
    duration_seconds: float
    stop_reason: str
    parameters: dict[str, Any] = field(default_factory=dict)
    metadata: dict[str, Any] = field(default_factory=dict)
```

Semua varian mengembalikan HillClimbingResult, yang berisi:

- State awal dan akhir
- Jumlah iterasi dan accepted moves
- Riwayat nilai objektif (untuk plot objective terhadap iterasi)
- Durasi pencarian, diukur dengan perf_counter()
- Alasan berhenti
- Parameter yang dipakai
- Metadata khusus tiap varian

Fungsi to_dict() mengubah hasil ke bentuk dictionary agar bisa disimpan sebagai JSON di folder output/. Kelas ini juga diwarisi oleh SimulatedAnnealingResult.

3.3. Implementasi Simulated Annealing

Berikut kelas, fungsi, dan konstanta yang digunakan dalam implementasi algoritma.

Fungsi/Kelas	Deskripsi
`SimulatedAnnealing`	Kelas utama yang mengelola pencarian, temperatur, penerimaan neighbor, dan kondisi berhenti.
`SimulatedAnnealingResult`	Menyimpan state awal dan akhir, jumlah iterasi, riwayat nilai dan temperatur, probabilitas penerimaan, jumlah langkah stuck, parameter, serta metadata hasil.
`__init__()`	Menginisialisasi temperatur awal, cooling rate, temperatur minimum, batas iterasi, batas percobaan mencari neighbor valid, ambang stuck, dan seed. Nilai parameter divalidasi sebelum pencarian dimulai.
`_clone()`	Membuat salinan state agar perubahan pada neighbor tidak mengubah state asal.
`_acceptance_probability()`	Menghitung probabilitas penerimaan berdasarkan selisih nilai objektif dan temperatur.
`Neighbor.random_neighbor()`	Mencoba menghasilkan satu neighbor valid melalui operasi perpindahan, rotasi, atau pertukaran paket.
`run()`	Menjalankan iterasi SA, memperbarui state dan temperatur, mencatat riwayat serta statistik, lalu mengembalikan hasil.
`calibrate_initial_temperature()`	Fungsi di src/main.py yang memperkirakan temperatur awal dari sampel neighbor jika pengguna tidak memberikan suhu secara manual.

`initial_temperature`	Diestimasi dari sampel neighbor awal. Dapat ditentukan manual.
`cooling_rate`	Faktor pengali temperatur setiap iterasi.
`min_temperature`	Batas bawah temperatur.
`max_iterations`	Batas jumlah iterasi.
`max_neighbor_attempts`	Batas percobaan untuk mendapatkan neighbor valid.
`stuck_threshold`	Ambang untuk mencatat streak tanpa perubahan; bukan kondisi berhenti.
`seed`	Mengatur generator bilangan acak agar hasil dapat diulang.

Berikut adalah alur implementasi dari algortima Simulated Annealing

1. Inisialisasi parameter

File : src/algorithm/simulated_annealing/simulated_annealing.py

```
    def __init__(
        self,
        initial_temperature: float = 100.0,
        cooling_rate: float = 0.99,
        min_temperature: float = 1e-3,
        max_iterations: int = 5000,
          max_neighbor_attempts: int = 100,
        stuck_threshold: int = 20,
        seed: int | None = None,
    ):
        if initial_temperature <= 0:
            raise ValueError("initial_temperature harus lebih dari 0.")
        if not 0 < cooling_rate < 1:
            raise ValueError("cooling_rate harus berada di antara 0 dan 1
(eksklusif).")
        if min_temperature <= 0 or min_temperature >= initial_temperature:
            raise ValueError("min_temperature harus > 0 dan <
initial_temperature.")
        if max_iterations <= 0:
            raise ValueError("max_iterations harus lebih dari 0.")
        if max_neighbor_attempts <= 0:
```

```
            raise ValueError("max_neighbor_attempts harus lebih dari 0.")
        if stuck_threshold <= 0:
            raise ValueError("stuck_threshold harus lebih dari 0.")

        self.initial_temperature = initial_temperature
        self.cooling_rate = cooling_rate
        self.min_temperature = min_temperature
        self.max_iterations = max_iterations
        self.max_neighbor_attempts = max_neighbor_attempts
        self.stuck_threshold = stuck_threshold
        self.seed = seed
        self.rng = random.Random(seed)
```

Konstruktor memeriksa agar suhu awal dan suhu minimum bernilai positif, cooling rate berada di antara 0 dan 1, dan batas iterasi serta jumlah percobaan neighbor lebih dari 0. Jika ada parameter yang tidak valid, program menampilkan ValueError.

2. Kalibrasi temperatur awal

File : src/main.py

```
def calibrate_initial_temperature(
    state,
    target_acceptance=0.9,
    sample_size=100,
    max_neighbor_attempts=100,
    min_temperature=0.001,
    seed=42,
):
    rng = random.Random(seed + 1)
    current_value = state.getValue()
    losses = []

    for _ in range(sample_size):
        candidate = Neighbor.random_neighbor(
            state,
            rng=rng,
            max_attempts=max_neighbor_attempts,
        )
        if candidate is None:
            continue


          loss = current_value - candidate.getValue()
        if loss > 0:
            losses.append(loss)
```

```
    if not losses:
        typical_loss = sum(
            abs(package.value) for package in state.packages
        ) / max(1, len(state.packages))
        temperature = (
            typical_loss / -math.log(target_acceptance)
            if typical_loss
            else 1.0
        )
        return max(temperature, min_temperature * 1.01)

    def mean_acceptance(temperature):
        return sum(
            math.exp(-loss / temperature) for loss in losses
        ) / len(losses)

    low = 0.0
    high = max(max(losses), min_temperature)

    while mean_acceptance(high) < target_acceptance:
        high *= 2

    for _ in range(80):
        middle = (low + high) / 2
        if mean_acceptance(middle) < target_acceptance:
            low = middle
        else:
            high = middle

    return max(high, min_temperature * 1.01)
```

Untuk mencari suhu awal, program mencoba membangkitkan sejumlah neighbor dari state awal. Jika sebuah neighbor memiliki nilai objektif lebih rendah, besar penurunan nilainya dicatat sebagai loss.
Untuk kumpulan loss tersebut, program mencari suhu T yang membuat rata-rata probabilitas penerimaan mendekati target_acceptance. Program memakai pencarian biner untuk memperoleh estimasi suhu. Nilai bawaan target penerimaan adalah 0.9 dengan 100 sampel.

3. Inisialisasi state pencarian

File : src/algorithm/simulated_annealing/simulated_annealing.py

```
        initial = self._clone(initial_state)
```

```
        current = self._clone(initial_state)
        current_value = current.getValue()
        best_value = current_value

        temperature = self.initial_temperature
        iterations = 0
        accepted_moves = 0
        improving_moves = 0
        worse_moves_accepted = 0   # "escape" dari local optima
        rejected_moves = 0
        unchanged_streak = 0
        stuck_count = 0
        stop_reason = "max_iterations_reached"

        value_history = [current_value]
        probability_history: list[float] = []
        temperature_history: list[float] = []
```

State awal disalin menjadi initial dan current. Nilai terbaik yang pernah ditemukan dimulai dari nilai state awal. Temperatur dimulai dari initial_temperature. current menunjukkan state yang sedang dieksplorasi. best_value mencatat nilai objektif tertinggi yang pernah dicapai selama pencarian.

4. Pembangkitan neighbor dan perubahan nilai

File : src/algorithm/simulated_annealing/simulated_annealing.py

```
            candidate = Neighbor.random_neighbor(
                current,
                rng=self.rng,
                max_attempts=self.max_neighbor_attempts,
            )

            if candidate is None:
                stop_reason = "no_valid_neighbor"
                iterations -= 1
                break

            candidate_value = candidate.getValue()
            delta = candidate_value - current_value
            probability = self._acceptance_probability(delta, temperature)
```

Pada setiap iterasi, Neighbor.random_neighbor() mencoba

mendapatkan satu neighbor valid. Jika neighbor valid tidak ditemukan dalam jumlah percobaan yang ditentukan, SA berhenti dengan alasan no_valid_neighbor.
Selisih nilai dihitung sebagai:

$$\Delta E = f(S') - f(S)$$

dengan S sebagai state saat ini, S' sebagai neighbor, dan $f(S)$ sebagai nilai fungsi objektif dari suatu state. Karena program memaksimalkan nilai, $\Delta > 0$ berarti neighbor lebih baik dan $\Delta < 0$ berarti neighbor lebih buruk.

5. Perhitungan probabilitas

File : src/algorithm/simulated_annealing/simulated_annealing.py

```
    def _acceptance_probability(delta: int, temperature: float) -> float:
        """e^(ΔE/T), di-cap ke 1.0 (ΔE > 0 selalu diterima)."""
        if delta >= 0:
            return 1.0
        return math.exp(delta / temperature)
```

Program menerima neighbor jika nilai meningkat atau bilangan acak berada di bawah probabilitas penerimaannya. Jika diterima, state dan nilai saat ini diperbarui. Jika ditolak, state saat ini dipertahankan.

```
                if delta > 0 or self.rng.random() < probability:
                    current = candidate
                    current_value = candidate_value
                    accepted_moves += 1
                    if delta != 0:
                        unchanged_streak = 0
                    else:
                        unchanged_streak += 1
                        if unchanged_streak == self.stuck_threshold:
                            stuck_count += 1
                    if delta > 0:
                        improving_moves += 1
                    elif delta < 0:
                        worse_moves_accepted += 1
                    best_value = max(best_value, current_value)
                else:
                    rejected_moves += 1
                    unchanged_streak += 1
                    if unchanged_streak == self.stuck_threshold:
```

```
                stuck_count += 1
```

Neighbor yang sama baik atau lebih baik memiliki probabilitas 1.0. Neighbor yang lebih buruk menggunakan rumus:

$$P = e^{\Delta E/T}$$

Karena Δ bernilai negatif untuk neighbor yang lebih buruk, probabilitasnya berada di antara 0 dan 1. Temperatur yang lebih tinggi meningkatkan peluang menerima neighbor yang lebih buruk.

6. Kondisi berhenti

File : src/algorithm/simulated_annealing/simulated_annealing.py

```
while iterations < self.max_iterations:
  if temperature <= self.min_temperature:
      stop_reason = "min_temperature_reached"
      break

  iterations += 1

  candidate = Neighbor.random_neighbor(
      current,
      rng=self.rng,
      max_attempts=self.max_neighbor_attempts,
  )

  if candidate is None:
      stop_reason = "no_valid_neighbor"
      iterations -= 1
      break

  candidate_value = candidate.getValue()
  delta = candidate_value - current_value
  probability = self._acceptance_probability(delta, temperature)

     probability_history.append(probability)
  temperature_history.append(temperature)

  if delta > 0 or self.rng.random() < probability:
      current = candidate
      current_value = candidate_value
      accepted_moves += 1
      if delta != 0:
```

```
                    unchanged_streak = 0
                else:
                    unchanged_streak += 1
                    if unchanged_streak == self.stuck_threshold:
                        stuck_count += 1
                if delta > 0:
                    improving_moves += 1
                elif delta < 0:
                    worse_moves_accepted += 1
                best_value = max(best_value, current_value)
            else:
                rejected_moves += 1
                unchanged_streak += 1
                if unchanged_streak == self.stuck_threshold:
                    stuck_count += 1

            value_history.append(current_value)
            temperature *= self.cooling_rate
```

Pencarian berhenti ketika:

- Temperatur mencapai atau turun di bawah min_temperature.
- Iterasi mencapai max_iterations.
- Neighbor valid tidak ditemukan setelah max_neighbor_attempts.

stuck_threshold tidak menghentikan algoritma. Parameter ini hanya digunakan untuk mencatat berapa kali streak tanpa perubahan mencapai ambang batas.

7. Menyimpan hasil

File : src/algorithm/simulated_annealing/result.py

```
class SimulatedAnnealingResult(HillClimbingResult):
    probability_history: list[float] = field(default_factory=list)
    temperature_history: list[float] = field(default_factory=list)
    stuck_count: int = 0

    def to_dict(self, include_states: bool = True) -> dict[str, Any]:
        data = super().to_dict(include_states=include_states)
        data["probability_history"] = self.probability_history
        data["temperature_history"] = self.temperature_history
        data["stuck_count"] = self.stuck_count
        return data
```

SimulatedAnnealingResult mewarisi struktur hasil Hill Climbing dan menambahkan riwayat temperatur, riwayat probabilitas penerimaan, serta jumlah kejadian stuck. Hasil juga memuat best_value_seen dan

final_temperature pada metadata. Perlu dibedakan bahwa final_state adalah state saat algoritma berhenti, sedangkan best_value_seen hanya mencatat nilai tertinggi yang pernah ditemukan.

3.4. Implementasi Genetic Algorithm

Berikut kelas, fungsi, dan konstanta yang digunakan dalam implementasi algoritma.

Fungsi/Kelas	Deskripsi
`GeneticAlgorithm`	Kelas utama yang mengimplementasikan algoritma Genetic Algorithm untuk mencari solusi penyusunan paket.
`GeneticAlgorithmResult`	Kelas hasil yang menyimpan informasi eksekusi algoritma, seperti solusi awal dan akhir, jumlah iterasi, riwayat nilai solusi, durasi, alasan berhenti, parameter, dan metadata.
`MAX_CROSSOVER_LIMIT`	Konstanta yang membatasi jumlah percobaan crossover hingga 1.000 kali untuk satu proses pembentukan individu.
`__init__()`	Menginisialisasi parameter algoritma, yaitu jumlah populasi (*nPopulation*), jumlah generasi (*nGeneration*), dan *seed* untuk bilangan acak.
`_fitness_function()`	Menghitung nilai fitness individu berdasarkan perbandingan nilai solusi terhadap total nilai seluruh populasi.
`_random_roulette()`	Memilih individu sebagai *parent* menggunakan metode roulette wheel selection, dengan peluang pemilihan berdasarkan nilai fitness.
`_mutate()`	Melakukan mutasi terhadap individu dengan memilih satu paket dan satu operasi secara acak, yaitu *move,*

	rotate, atau *swap*. Jika tidak ditemukan perubahan yang valid, individu asli dikembalikan.
`_run_one_generation()`	Menjalankan satu generasi GA yang terdiri dari proses *selection, crossover,* dan *mutation*, kemudian menghasilkan populasi baru.
`run()`	Menjalankan algoritma secara keseluruhan, mulai dari pembentukan populasi awal, iterasi antargenerasi, pencatatan statistik, hingga pemilihan individu dengan nilai solusi terbaik.

Berikut adalah implementasi kode dari proses utama pada GA:

1. Pembentukan populasi awal

File: src/algorithm/genetic_algorithm/genetic_algorithm.py

```
def run(self, initial_state: State):
        # initiate population
        population: list[State] = []
        for _ in range(self.nPopulation):
            state = State(initial_state.packages, initial_state.truck, self.rng)
            population.append(state)
```

2. Selection

File: src/algorithm/genetic_algorithm/genetic_algorithm.py

```
    def _run_one_generation(self, population: list[State]) -> list[State]:
        # fitness function
        sumValue = 0
        for p in population:
            sumValue += p.getValue()

        # random roulette, sekali jalan pick 2
        selectedStates: list[State] = []
        parent_pair_count = (self.nPopulation + 1) // 2 #tangani ganjil
        for _ in range(parent_pair_count):
            selectedState1 = self._random_roulette(population, self.nPopulation,
```

```
sumValue, self.rng)
            selectedState2 = self._random_roulette(population, self.nPopulation,
sumValue, self.rng)

            selectedStates.append(selectedState1)
            selectedStates.append(selectedState2)
```

```
    def _random_roulette(population: list[State], nPopulation: int, sumValue:
int, rng) -> State:
        selectedState: State
        rr = rng.randint(0,100) #random pick

        for _ in range(nPopulation):
            f = GeneticAlgorithm._fitness_function(population[0].getValue(),
sumValue)
            if f >= rr:
                selectedState = population[0]
                return selectedState
            else:
                for i in range (1, nPopulation):
                    f +=
GeneticAlgorithm._fitness_function(population[i].getValue(), sumValue)
                    if f >= rr:
                        selectedState = population[i]
                        return selectedState

        return population[-1]
```

3. Crossover

File: src/algorithm/genetic_algorithm/genetic_algorithm.py

```
    def _run_one_generation(self, population: list[State]) -> list[State]:
        ...selection...
        # gabung state paket antar 2 parent secara package by package
        crossedState: list[State] = []
        self.crossover_limit_reached = False

        for i in range(0, len(selectedStates), 2):
            # selectedStates[i] <- ayah
            # selectedStates[i+1] <- ibu
            parent1: State = selectedStates[i]
            parent2: State = selectedStates[i+1]

            for _ in range(2):  # dari satu pasangan parent menghasilkan 2 child
                positions = {}
```

```
                crossedPackages: list[Package] = []
                crossover_attempts = 0

                # urutkan paket berdasarkan z terkecil dari kedua parent,
meminimalkan floating
                packageOrder = list(parent1.packages)
                self.rng.shuffle(packageOrder)
                packageOrder.sort(
                    key=lambda p: min(
                        parent1.getPosition(p)[2],
                        parent2.getPosition(p)[2]
                    )
                )

                # tempatkan paket per paket
                for package in packageOrder:
                    placed = False

                    # coba posisi dari parent 1 dan parent 2
                    candidatePositions = [parent1.getPosition(package),
parent2.getPosition(package)]
                    self.rng.shuffle(candidatePositions)

                    for candidatePosition in candidatePositions:
                        crossover_attempts += 1
                        if crossover_attempts >= MAX_CROSSOVER_LIMIT:
                            self.crossover_limit_reached = True
                            return population

                        if State.isValidPlacement(
                            package,
                            candidatePosition,
                            positions,
                            parent1.packages,
                            truck,
                            self.rng
                        ):
                            crossedPackages.append(package)
                            placed = True
                            break

                    # Kalau posisi kedua parent gagal, coba posisi random.
                    while not placed and crossover_attempts <
MAX_CROSSOVER_LIMIT:
                        crossover_attempts += 1

                        newPosition = StateHelper.randomize(package.dim,
truck.dimPeti, self.rng)
```

```
                        if State.isValidPlacement(
                            package,
                            newPosition,
                            positions,
                            parent1.packages,
                            truck,
                            self.rng
                        ):
                            crossedPackages.append(package)
                            placed = True
                            self.random_position_used += 1

                    if not placed:
                        self.crossover_limit_reached = True
                        return population

                # Semua paket sudah ditempatkan -> satu child
                crossedState.append(State(crossedPackages, truck, self.rng,
False, positions))
```

4. Mutation

File: src/algorithm/genetic_algorithm/genetic_algorithm.py

```
    def _run_one_generation(self, population: list[State]) -> list[State]:
        ...selection...
        ...crossover...
        mutation_point = self.rng.randint(0, countPackageIndividual-1)
        childStates: list[State] = []
        for st in crossedState:
            childStates.append(self._mutate(st, mutation_point))

        return childStates[:self.nPopulation]
```

```
    def _mutate(self, state: State, mutation_point: int) -> State:
        package = state.packages[mutation_point]
        operations = ["move", "rotate"]
        if len(state.packages) > 1:
            operations.append("swap")

        operation = self.rng.choice(operations) #random mutation value
        if operation == "move":
            for _ in range(100):
                new_position = StateHelper.randomize(state.getDimOri(package),
state.truck.dimPeti, self.rng)
```

```
                candidate = Neighbor.move(state, package.id, new_position)
                if candidate is not None: return candidate
        elif operation == "rotate":
            for axis in self.rng.sample(("x", "y", "z"), 3):
                candidate = Neighbor.rotate(state, package.id, axis)
                if candidate is not None: return candidate
        else:
            partners: list[Package] = []
            for other in state.packages:
                if other.id == package.id: continue
                partners.append(other)

            self.rng.shuffle(partners)
            for partner in partners:
                candidate = Neighbor.swap(state, package.id, partner.id)
                if candidate is not None: return candidate

        return state
```

BAB 4
EKSPERIMEN

4. Metodologi Eksperimen

4.1.1. Lingkungan eksperimen

Eksperimen dijalankan pada laptop dengan prosesor Intel Core i7-13620H (10 core, 16 thread) dan RAM 16 GB, dengan sistem operasi Windows 11 Home 64-bit. Program diimplementasikan dalam Python 3.13.5 dengan Matplotlib 3.10.7 untuk visualisasi. Program berjalan pada satu thread, sehingga durasi pencarian terutama dipengaruhi oleh kecepatan per-core prosesor. Seluruh eksperimen dijalankan melalui src/main.py (Hill Climbing dan Simulated Annealing) dan src/run_genetic_algorithm.py (Genetic Algorithm). Hasil setiap run disimpan dalam format JSON di folder output/<algoritma>/, sedangkan visualisasi state dan plot riwayat nilai disimpan di output/visualization/<algoritma>/.

4.1.2. Data uji

Eksperimen menggunakan file input/input2.txt yang berisi 20 paket dengan dimensi, nilai, berat, status fragile, dan ETA yang beragam. Peti kemas berukuran 6 × 5 × 4 dengan kapasitas maksimum 40 satuan berat. Total berat seluruh paket jauh melebihi kapasitas, sehingga algoritma harus memilih kombinasi paket yang dimuat.

Karena ruang peti kemas relatif longgar dibanding kapasitas beratnya, batasan yang paling menentukan adalah kapasitas berat. Dengan menyelesaikan persoalan knapsack berdasarkan berat, diperoleh nilai maksimum 355, yaitu dari paket P2, P4, P9, P16, dan P18 (total berat 40). Luas alas kelima paket tersebut hanya 21 dari 30 satuan luas lantai, sehingga seluruhnya dapat diletakkan di lantai peti kemas tanpa melanggar constraint. Dengan demikian, nilai 355 adalah global maximum dan digunakan sebagai acuan untuk menilai seberapa dekat setiap algoritma mendekati solusi optimal.

Fungsi objektif yang digunakan pada seluruh eksperimen adalah fungsi objektif utama, yaitu total value paket di dalam peti kemas.

4.1.3. Parameter algoritma

Algoritma	**Parameter**	**Nilai**
Steepest-Ascent HC	-	Tidak ada parameter tambahan;
HC with Sideways Move	max_sideways	100 sideways move berturut-turut
Stochastic HC	max_iterations	1000
	max_neighbor_attempts	100
Random-Restart HC	max_restarts	10 (total 11 kali pencarian Steepest-Ascent)
	target_value	Tidak digunakan (pencarian selalu dijalankan hingga batas restart)
Simulated Annealing	Temperatur awal (T_0)	Dikalibrasi otomatis dari 100 sampel neighbor dengan target rata-rata peluang penerimaan langkah menurun 0,9
	cooling_rate	0,999 (pendinginan geometris, $T \leftarrow 0{,}999 \cdot T$ setiap iterasi)
	min_temperature	0,001
	max_iterations	[5000, sesuaikan jika diubah]
	max_neighbor_attempts	100
	stuck_threshold	20 iterasi berturut-turut tanpa perubahan nilai dihitung sebagai satu kejadian stuck
Genetic Algorithm	Seleksi	Roulette wheel berdasarkan proporsi nilai objektif

	Crossover	Per paket, posisi diambil dari salah satu parent secara acak; posisi acak dipakai jika kedua posisi parent tidak valid
	Mutasi	Satu operasi acak (move, rotate, atau swap) pada satu paket; diterapkan pada setiap individu anak
	MAX_CROSSOVER_LIMIT	1000 percobaan penempatan per individu anak

4.1.4. Jumlah Pecobaan

Hill Climbing (4 varian) dan Simulated Annealing: setiap algoritma dijalankan 3 kali menggunakan seed 1, 2, dan 3. Total 15 run. Genetic Algorithm: dua kelompok eksperimen, masing-masing dengan 3 konfigurasi × 3 run, sehingga total 18 run.

- Jumlah populasi sebagai kontrol (20 individu), dengan variasi jumlah generasi 25, 50, dan 100.
- Jumlah generasi sebagai kontrol (50 generasi), dengan variasi jumlah populasi 10, 20, dan 40.

Seed setiap run GA dibangkitkan dari seed dasar 0, sehingga setiap run memiliki populasi awal acak yang berbeda tetapi tetap dapat diulang.

4.1.5. Data yang Dicatat

Untuk setiap run dicatat:

- State awal dan state akhir
- Nilai objektif awal dan akhir
- Riwayat nilai objektif terhadap iterasi
- Jumlah iterasi
- Durasi pencarian
- Alasan berhenti

Selain itu, setiap algoritma mencatat informasi tambahan:

- Sideways Move: jumlah sideways move.
- Random Restart: jumlah restart, iterasi per restart, dan nilai akhir tiap restart.
- Stochastic HC: jumlah langkah yang ditolak.

- SA: riwayat e^(ΔE/T) dan frekuensi stuck.
- GA: riwayat nilai objektif maksimum dan rata-rata populasi.

4.1.6. Menjaga perbandingan tetap adil

Untuk menjaga perbandingan dalam eksperimen kami tetap adil, beberapa hal ini kami pastikan.

1. State awal yang sama. State awal dibangkitkan dari seed yang sama, sehingga untuk setiap nomor run, keempat varian Hill Climbing dan Simulated Annealing memulai pencarian dari state yang identik, dengan nilai objektif awal 80 (seed 1), 255 (seed 2), dan 320 (seed 3). Perbedaan hasil antaralgoritma dengan demikian berasal dari strategi pencariannya, bukan dari state awal. GA tidak memakai satu state awal, melainkan populasi acak. Untuk GA, nilai awal yang dilaporkan adalah nilai individu terbaik pada populasi awal.
2. Data uji, fungsi objektif, dan constraint yang sama. Seluruh algoritma memakai input, fungsi objektif, dan fungsi validasi (isValid()) yang sama.
3. Operasi neighbor yang sama. Seluruh algoritma memakai operasi swap, move, dan rotate dari kelas Neighbor yang sama. Hill Climbing (kecuali Stochastic) membangkitkan seluruh neighbor, sedangkan Stochastic HC, SA, dan mutasi GA membangkitkan neighbor secara acak.
4. Reproducibility. Setiap run memakai seed tetap, sehingga hasil dapat diulang dengan nilai dan jumlah iterasi yang sama. Durasi dapat sedikit berbeda antar-eksekusi karena bergantung pada kondisi komputer.
5. Pengukuran durasi yang sama. Durasi diukur dengan perf_counter() dan hanya mencakup proses pencarian, tanpa pembacaan input, penyimpanan hasil, maupun pembuatan visualisasi.

4.2. Hasil Hill Climbing

Eksperimen Hill Climbing dilakukan terhadap empat varian: Steepest-Ascent, Sideways Move, Stochastic, dan Random-Restart. Setiap varian dijalankan tiga kali dengan seed 1, 2, dan 3. Karena state awal dibangkitkan dari seed yang sama, keempat varian memulai pencarian dari state awal yang identik pada setiap nomor run:

Run	Seed	Nilai Objektif Awal	Paket di Dalam Truk	Berat
1	1	80	P2	8/40
2	2	255	P1, P4, P9	40/40
3	3	320	P2, P4, P14, P16	37/40

4.2.1. Steepest-Ascent Hill Climbing

Steepest-Ascent berhenti setelah 1–2 iterasi pada ketiga run karena neighbor terbaik bernilai sama dengan current state (flat local maximum).

Run	Nilai Awal	Nilai Akhir	Iterasi	Accepted Moves	Durasi (detik)
1	80	230	2	1	0,41
2	255	255	1	0	0,13
3	320	335	2	1	0,41

State Akhir:

Run	Paket di Dalam Truk	Berat
1	P2, P3	38/40
2	P1, P4, P9	40/40
3	P2, P4, P12, P14	40/40

Pada run 2, state awal sudah memenuhi kapasitas (40/40), sehingga tidak ada neighbor yang lebih baik sejak iterasi pertama.

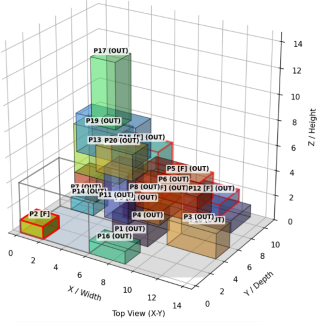
Berikut ini gambaran paket yang termuat :

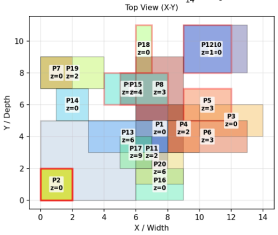
Run	State

1

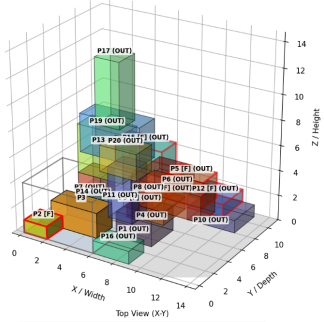
Steepest Ascent: 80 → 230 | Iterations: 2 | Duration: 0.4514s | Stop: flat_local_optimum

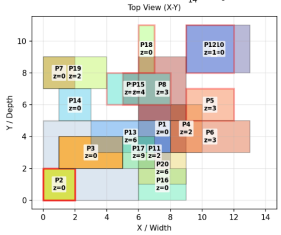
Steepest Ascent — Initial State
Objective Value: 80 | Inside: 1 | Outside: 19 | Weight: 8/40





Steepest Ascent — Final State
Objective Value: 230 | Inside: 2 | Outside: 18 | Weight: 38/40





ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P2	IN	(0, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P1	OUT	(6, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(9, 8, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(9, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 3, 6)	(5, 2, 3)	210	42	No	-
P14	OUT	(1, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(4, 6, 4)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 0, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(5, 2, 9)	(2, 2, 5)	155	32	No	-
P18	OUT	(6, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(0, 7, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 1, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 4, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(8, 3, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(9, 5, 3)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 3, 3)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 5, 3)	(3, 4, 1)	130	20	No	-
P9	OUT	(5, 6, 0)	(1, 2, 1)	45	4	Yes	-

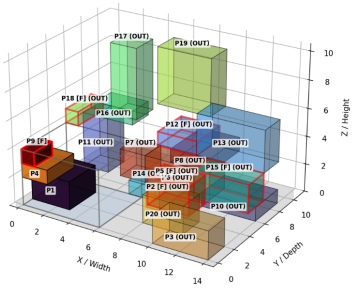
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P2	IN	(0, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P3	IN	(1, 2, 0)	(4, 2, 2)	150	30	No	T1
P1	OUT	(6, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(9, 8, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(9, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 3, 6)	(5, 2, 3)	210	42	No	-
P14	OUT	(1, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(4, 6, 4)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 0, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(5, 2, 9)	(2, 2, 5)	155	32	No	-
P18	OUT	(6, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(0, 7, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 1, 6)	(3, 2, 2)	115	21	No	-
P4	OUT	(8, 3, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(9, 5, 3)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 3, 3)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 5, 3)	(3, 4, 1)	130	20	No	-
P9	OUT	(5, 6, 0)	(1, 2, 1)	45	4	Yes	-

Truck boundary Truck floor Outside floor Inside truck Outside truck (OUT) Fragile [F]

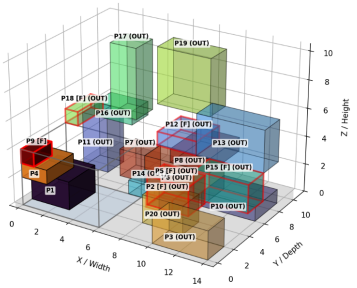
2

Steepest Ascent: 255 → 255 | Iterations: 1 | Duration: 0.1827s | Stop: flat_local_optimum

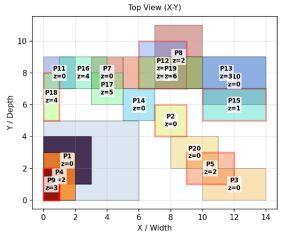
Steepest Ascent — Initial State
Objective Value: 255 | Inside: 3 | Outside: 17 | Weight: 40/40



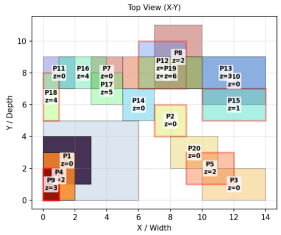
Steepest Ascent — Final State
Objective Value: 255 | Inside: 3 | Outside: 17 | Weight: 40/40



Top View (X-Y)



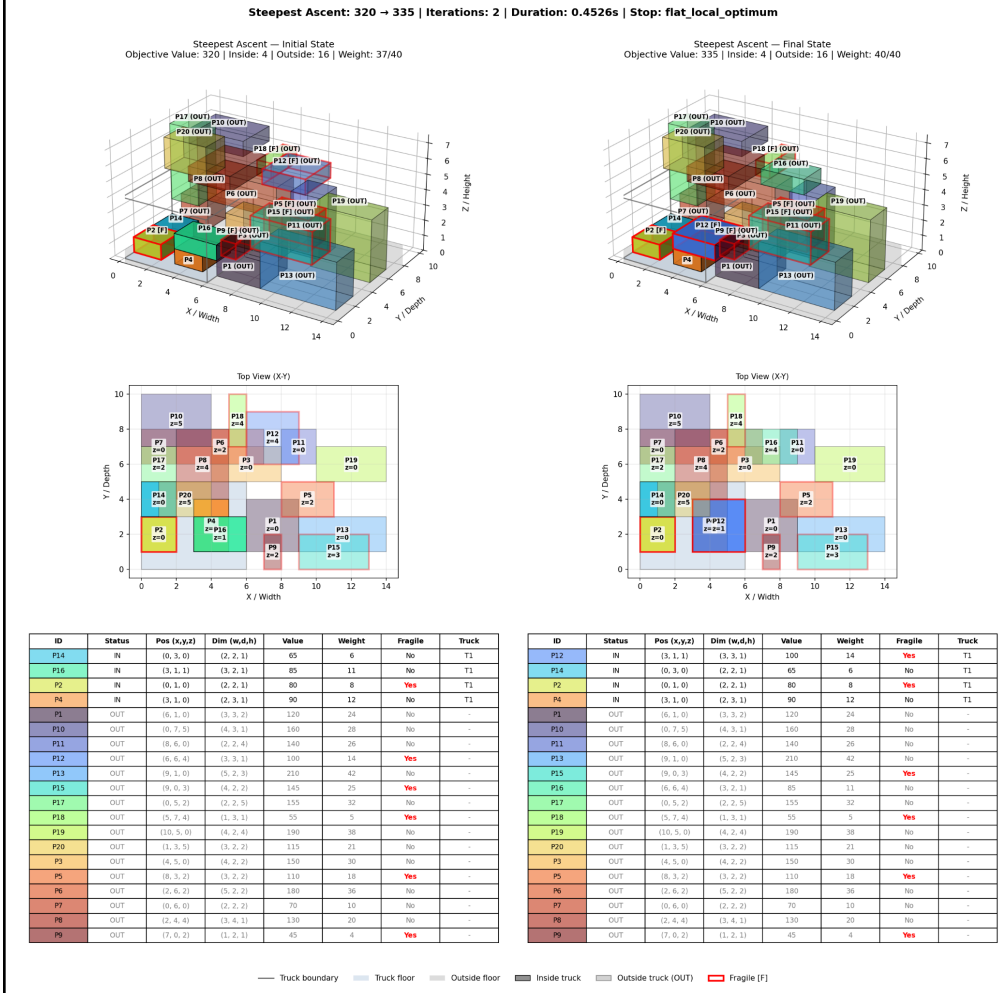
Top View (X-Y)



ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 1, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(0, 0, 3)	(1, 2, 1)	45	4	Yes	T1
P10	OUT	(10, 6, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 7, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 7, 3)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 5, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 7, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(3, 6, 5)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(6, 7, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 4, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(8, 2, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 1, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 7, 2)	(3, 4, 1)	130	20	No	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 1, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(0, 0, 3)	(1, 2, 1)	45	4	Yes	T1
P10	OUT	(10, 6, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 7, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 7, 3)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 5, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 7, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(3, 6, 5)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(6, 7, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 4, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(8, 2, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 1, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 7, 2)	(3, 4, 1)	130	20	No	-

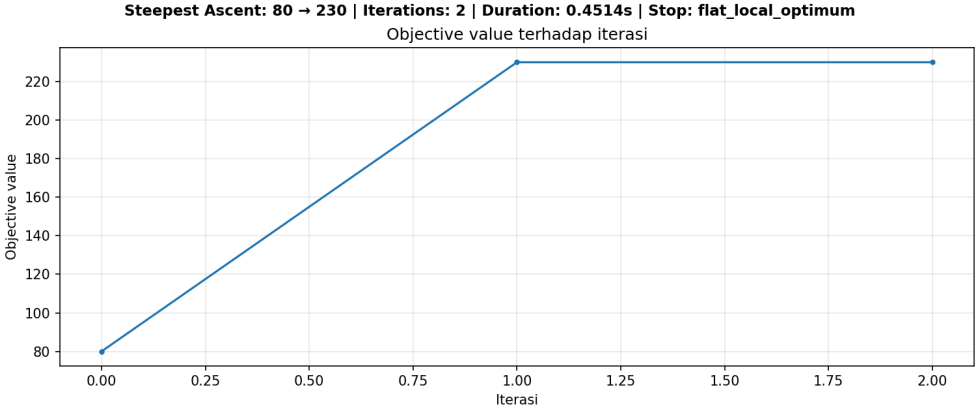
Truck boundary | Truck floor | Outside floor | Inside truck | Outside truck (OUT) | Fragile [F]

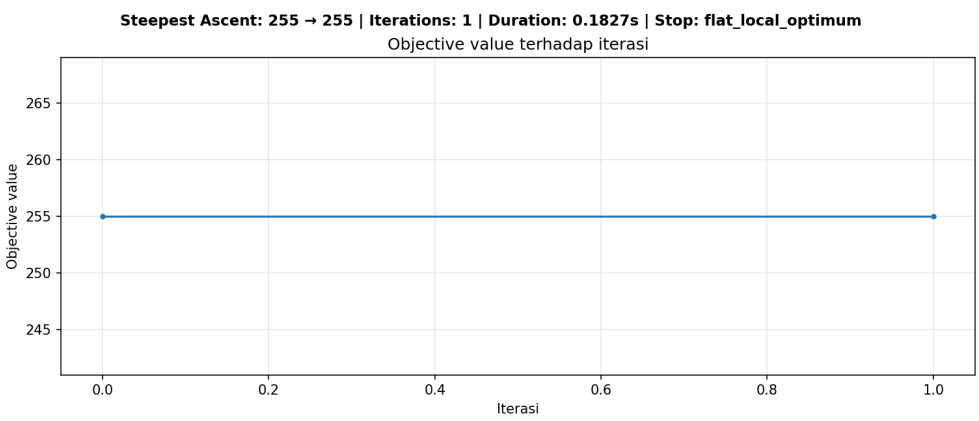
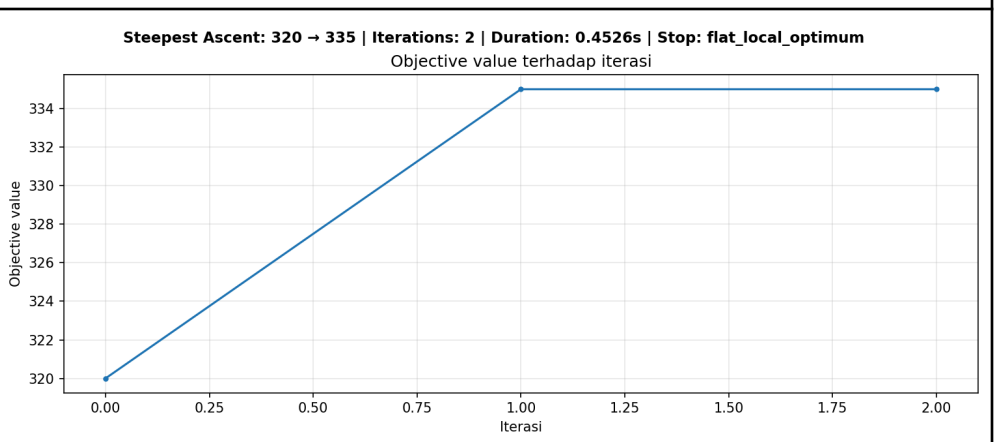
3	

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(3, 1, 1)	(3, 2, 1)	85	11	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 6, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P12	IN	(3, 1, 1)	(3, 3, 1)	100	14	Yes	T1
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 6, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

Berikut ini grafik objective function terhadap iterasi :

Run	Grafik
1	

2	
3	

4.2.2. Hill Climbing with Sideways Move

Sideways Move berhenti pada ketiga run karena batas 100 sideways move berturut-turut tercapai, dengan nilai akhir yang hampir sama dengan Steepest-Ascent. Parameter yang digunakan adalah max_sideways = 100.

Run	Nilai Awal	Nilai Akhir	Iterasi	Sideways Move	Durasi (detik)
1	80	235	105	102	21,83
2	255	255	101	100	17,77
3	320	335	102	100	32,22

State Akhir:

Run	Paket di Dalam Truk	Berat
1	P2, P17	40/40
2	P1, P4, P9	40/40
3	P2, P4, P12, P14	40/40

Pada run 1, setelah naik ke 230, algoritma melakukan dua sideways move lalu menemukan perbaikan kecil ke 235. Penghitung sideways kemudian di-reset, dan algoritma menghabiskan 100 sideways move berturut-turut tanpa perbaikan. Pada run 2 dan 3, seluruh langkah setelah perbaikan pertama adalah sideways move.

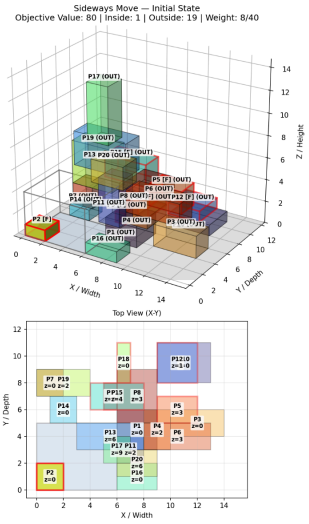
Sideways move membantu menjelajahi plateau, tetapi pada persoalan ini plateau-nya sangat luas sehingga 100 langkah tidak cukup untuk menemukan jalan naik.

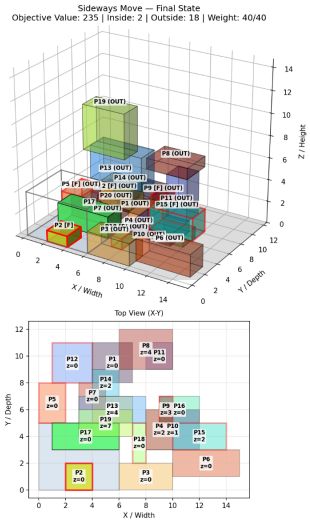
Berikut ini gambaran paket yang termuat :

Run	**State**

1

Sideways Move: 80 → 235 | Iterations: 105 | Duration: 21.8297s | Stop: max_sideways_reached





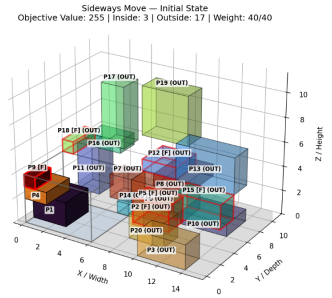
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P2	IN	(0, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P1	OUT	(6, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(9, 8, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(9, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 3, 6)	(5, 2, 3)	210	42	No	-
P14	OUT	(1, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(4, 6, 4)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 0, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(5, 2, 9)	(2, 2, 5)	155	32	No	-
P18	OUT	(6, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(0, 7, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 1, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 4, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(8, 3, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(9, 5, 3)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 3, 3)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 5, 3)	(3, 4, 1)	130	20	No	-
P9	OUT	(5, 6, 0)	(1, 2, 1)	45	4	Yes	-

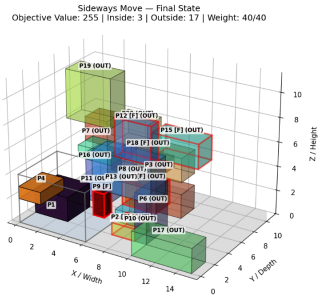
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P17	IN	(1, 3, 0)	(5, 2, 2)	155	32	No	T1
P2	IN	(2, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P1	OUT	(4, 8, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(8, 3, 1)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 9, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(1, 8, 0)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 5, 4)	(5, 2, 3)	210	42	No	-
P14	OUT	(4, 7, 2)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 3, 2)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(9, 5, 0)	(3, 2, 1)	85	11	No	-
P18	OUT	(7, 2, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(3, 4, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(4, 5, 2)	(3, 2, 2)	115	21	No	-
P3	OUT	(6, 0, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(8, 3, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(0, 5, 0)	(2, 3, 2)	110	18	Yes	-
P6	OUT	(10, 1, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 9, 4)	(4, 3, 1)	130	20	No	-
P9	OUT	(9, 5, 3)	(1, 2, 1)	45	4	Yes	-

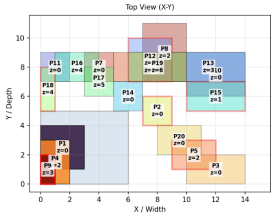
Truck boundary | Truck floor | Outside floor | Inside truck | Outside truck (OUT) | Fragile [F]

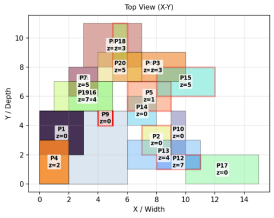
2

Sideways Move: 255 → 255 | Iterations: 101 | Duration: 17.7687s | Stop: max_sideways_reached





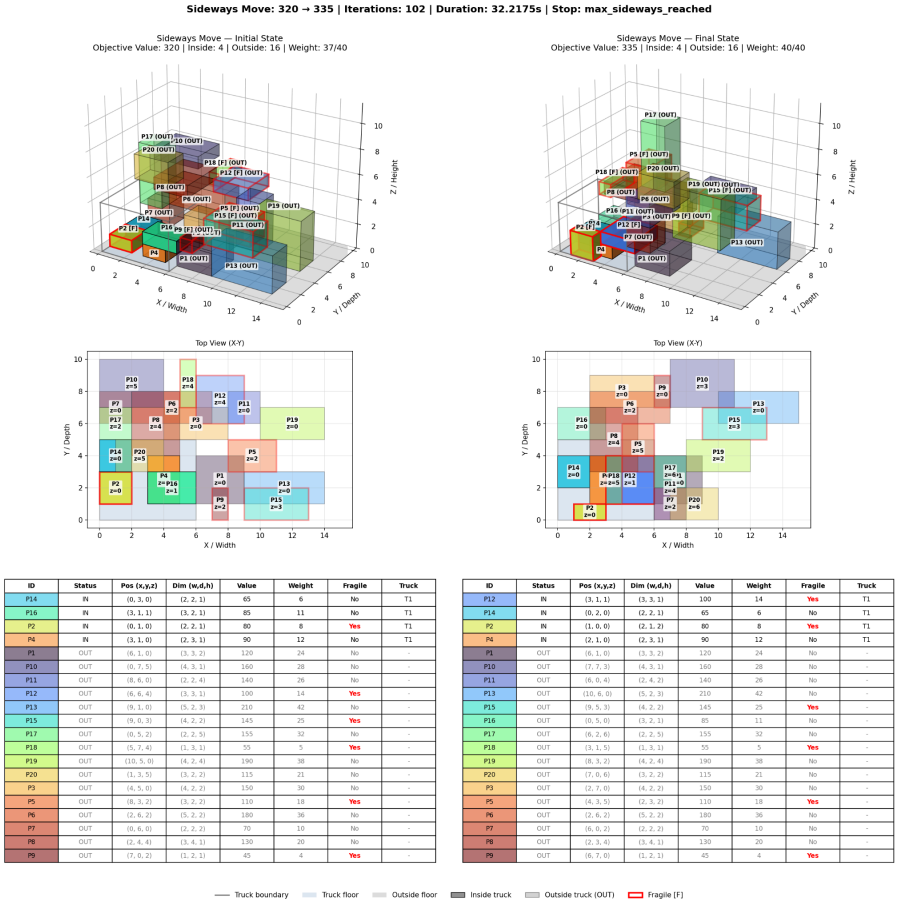




ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 1, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(0, 0, 3)	(1, 2, 1)	45	4	Yes	T1
P10	OUT	(10, 6, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 7, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 7, 3)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 5, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 7, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(3, 6, 5)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(6, 7, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 4, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(8, 2, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 1, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 7, 2)	(3, 4, 1)	130	20	No	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 2, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(4, 4, 0)	(1, 1, 2)	45	4	Yes	T1
P10	OUT	(9, 2, 0)	(1, 3, 4)	160	28	No	-
P11	OUT	(2, 6, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(8, 1, 7)	(3, 1, 3)	100	14	Yes	-
P13	OUT	(6, 1, 4)	(5, 2, 3)	210	42	No	-
P14	OUT	(6, 4, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(8, 6, 5)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(2, 5, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(10, 0, 0)	(5, 2, 2)	155	32	No	-
P18	OUT	(5, 8, 3)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(1, 5, 7)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 2, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(4, 7, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(6, 7, 3)	(4, 2, 2)	150	30	No	-
P5	OUT	(6, 5, 1)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(2, 6, 5)	(2, 2, 2)	70	10	No	-
P8	OUT	(3, 8, 2)	(4, 3, 1)	130	20	No	-

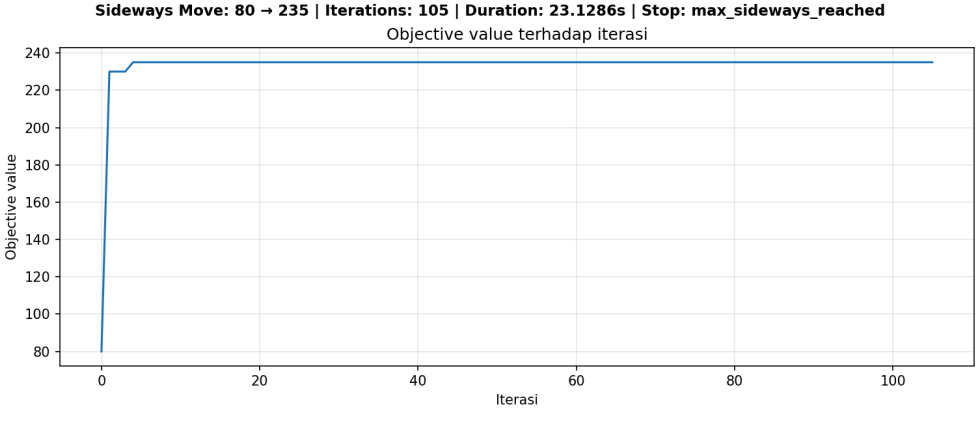
Truck boundary | Truck floor | Outside floor | Inside truck | Outside truck (OUT) | Fragile [F]

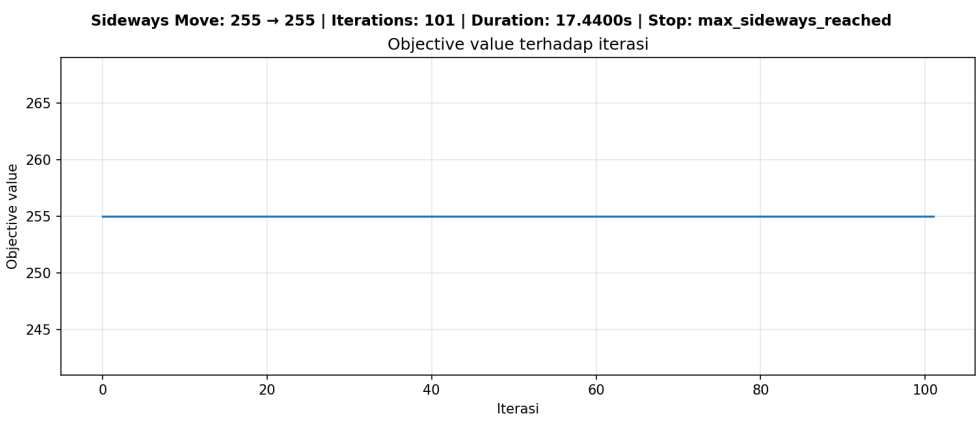
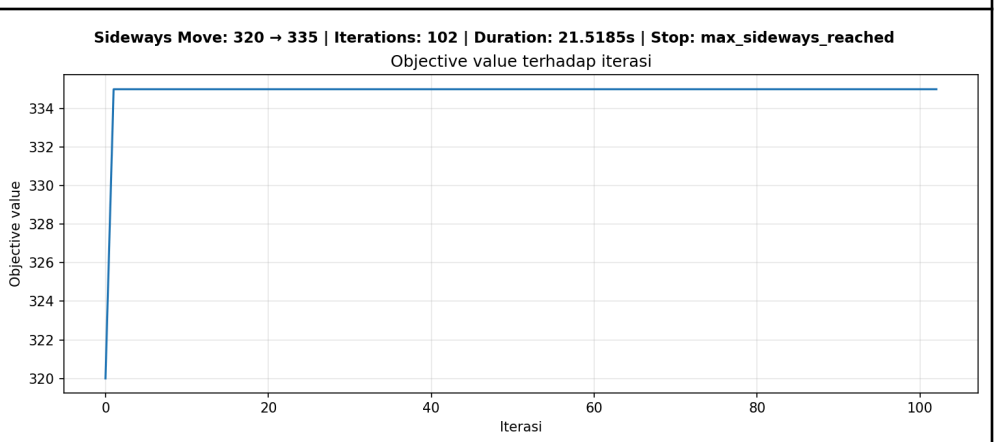
3	

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(3, 1, 1)	(3, 2, 1)	85	11	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 6, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P12	IN	(3, 1, 1)	(3, 3, 1)	100	14	Yes	T1
P14	IN	(0, 2, 0)	(2, 2, 1)	65	6	No	T1
P2	IN	(1, 0, 0)	(2, 1, 2)	80	8	Yes	T1
P4	IN	(2, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(7, 7, 3)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 0, 4)	(2, 4, 2)	140	26	No	-
P13	OUT	(10, 6, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 5, 3)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(0, 5, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(6, 2, 6)	(2, 2, 5)	155	32	No	-
P18	OUT	(3, 1, 5)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(8, 3, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(7, 0, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(2, 7, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(4, 3, 5)	(2, 3, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(6, 0, 2)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 3, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(6, 7, 0)	(1, 2, 1)	45	4	Yes	-

Berikut ini grafik objective function terhadap iterasi :

Run	Grafik
1	

2	
3	

4.2.3. Stochastic Hill Climbing

Stochastic hanya menerima 0–2 dari 1000 neighbor acak, tetapi pada run 1 menghasilkan nilai lebih baik (290) daripada Steepest-Ascent dan Sideways Move. Parameter yang digunakan adalah max_iterations = 1000 dan max_neighbor_attempts = 100.

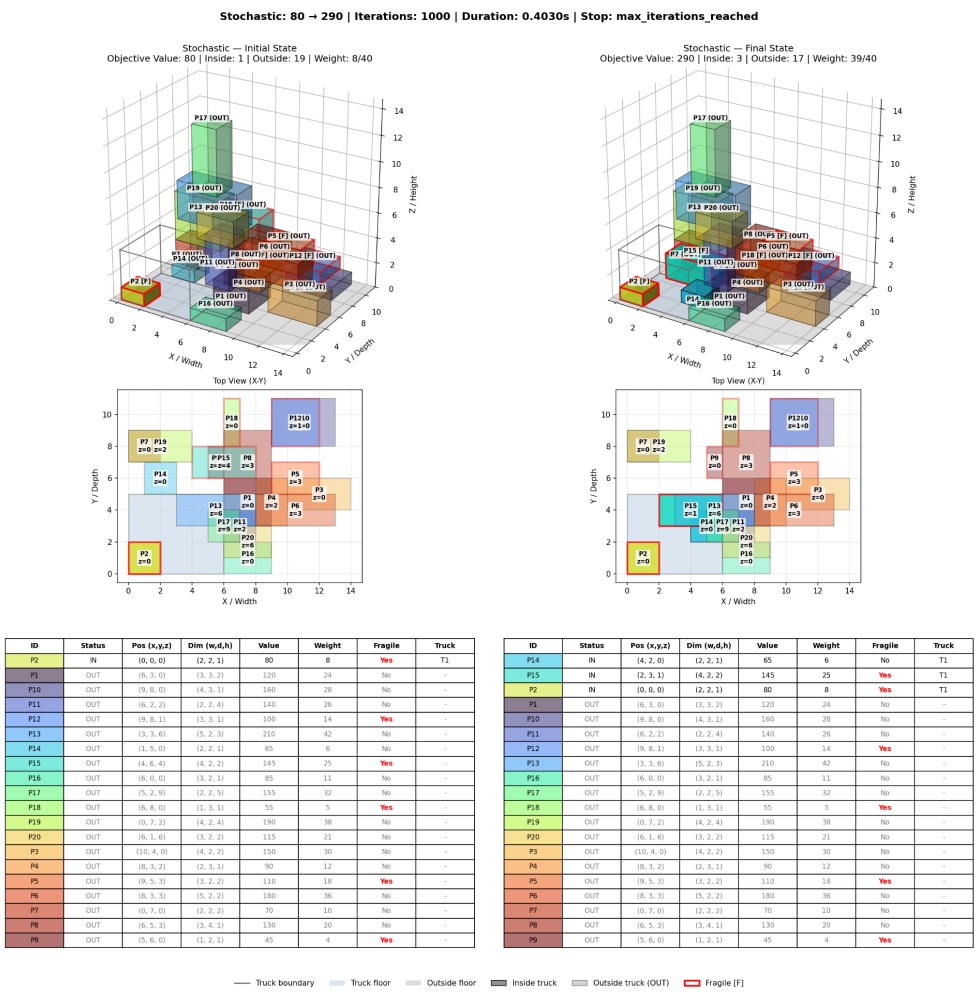
Run	Nilai Awal	Nilai Akhir	Iterasi	Diterima	Ditolak	Durasi (detik)
1	80	290	1000	2	998	0,40
2	255	255	1000	0	1000	0,67
3	320	335	1000	1	999	0,43

State Akhir:

Run	Paket di Dalam Truk	Berat
1	P2, P14, P15	39/40
2	P1, P4, P9	40/40
3	P2, P4, P12, P14	40/40

Hampir semua neighbor acak bernilai sama atau lebih buruk, misalnya memindahkan paket yang sudah di luar truk ke posisi luar lain. Pada run 1, langkah acak membawa algoritma ke jalur pendakian yang berbeda dari Steepest-Ascent, sehingga hasilnya lebih tinggi.

Berikut ini gambaran paket yang termuat :

Run	State
1	

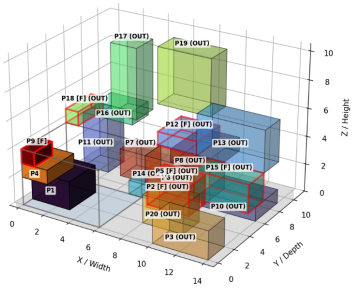
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P2	IN	(0, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P1	OUT	(6, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(9, 8, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(9, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 3, 6)	(5, 2, 3)	210	42	No	-
P14	OUT	(1, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(4, 6, 4)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 0, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(5, 2, 9)	(2, 2, 5)	155	32	No	-
P18	OUT	(6, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(0, 7, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 1, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 4, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(8, 3, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(9, 5, 3)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 3, 3)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 5, 3)	(3, 4, 1)	130	20	No	-
P9	OUT	(5, 6, 0)	(1, 2, 1)	45	4	Yes	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(4, 2, 0)	(2, 2, 1)	65	6	No	T1
P15	IN	(2, 3, 1)	(4, 2, 2)	145	25	Yes	T1
P2	IN	(0, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P1	OUT	(6, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(9, 8, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(9, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 3, 6)	(5, 2, 3)	210	42	No	-
P16	OUT	(6, 0, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(5, 2, 9)	(2, 2, 5)	155	32	No	-
P18	OUT	(6, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(0, 7, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 1, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 4, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(8, 3, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(9, 5, 3)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 3, 3)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 5, 3)	(3, 4, 1)	130	20	No	-
P9	OUT	(5, 6, 0)	(1, 2, 1)	45	4	Yes	-

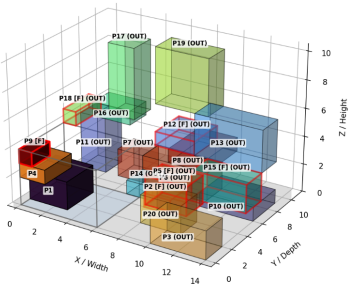
2

Stochastic: 255 → 255 | Iterations: 1000 | Duration: 0.5385s | Stop: max_iterations_reached

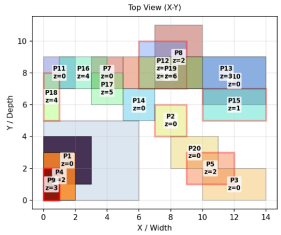
Stochastic — Initial State
Objective Value: 255 | Inside: 3 | Outside: 17 | Weight: 40/40



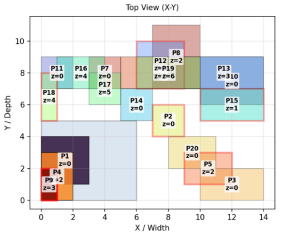
Stochastic — Final State
Objective Value: 255 | Inside: 3 | Outside: 17 | Weight: 40/40



Top View (X-Y)



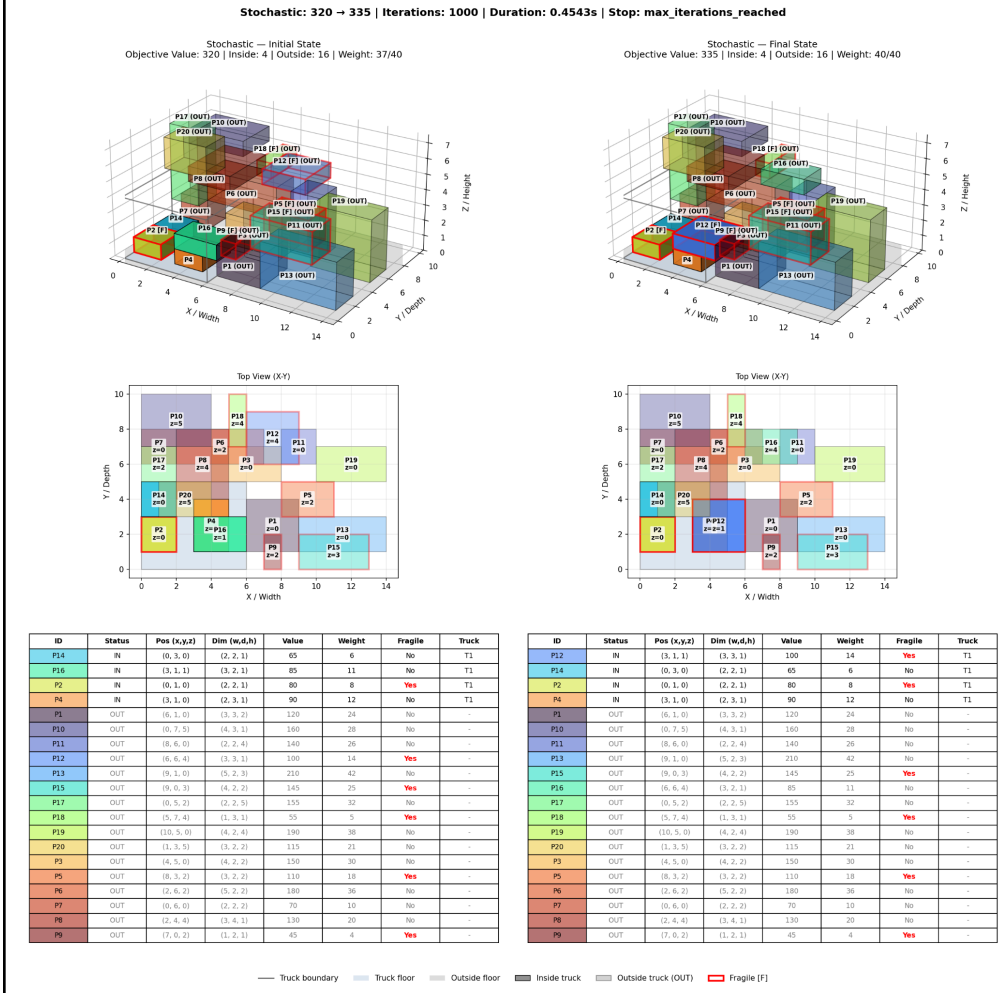
Top View (X-Y)



ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 1, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(0, 0, 3)	(1, 2, 1)	45	4	Yes	T1
P10	OUT	(10, 6, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 7, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 7, 3)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 5, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 7, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(3, 6, 5)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(6, 7, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 4, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(8, 2, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 1, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 7, 2)	(3, 4, 1)	130	20	No	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 1, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(0, 0, 3)	(1, 2, 1)	45	4	Yes	T1
P10	OUT	(10, 6, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 7, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 7, 3)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 5, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 7, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(3, 6, 5)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(6, 7, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 4, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(8, 2, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 1, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 7, 2)	(3, 4, 1)	130	20	No	-

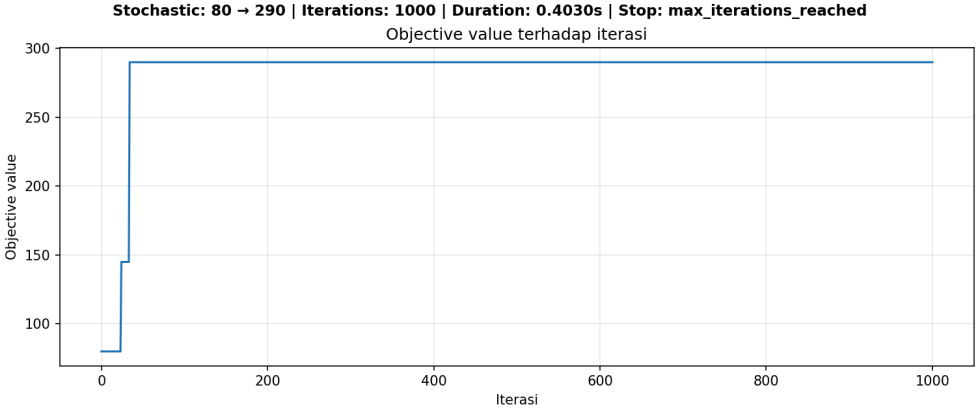
Truck boundary | Truck floor | Outside floor | Inside truck | Outside truck (OUT) | Fragile [F]

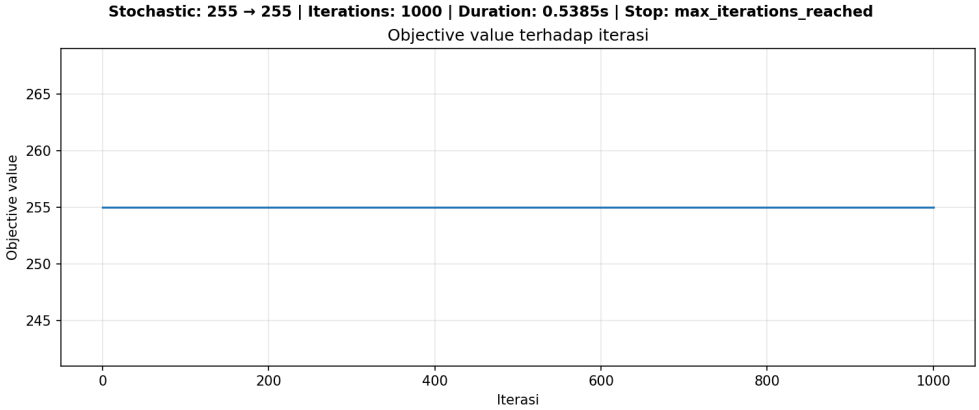
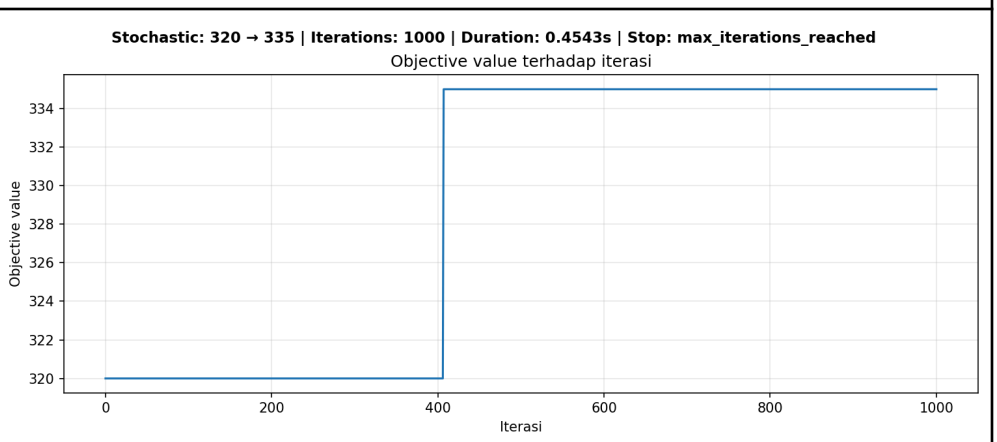
3	

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(3, 1, 1)	(3, 2, 1)	85	11	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 6, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P12	IN	(3, 1, 1)	(3, 3, 1)	100	14	Yes	T1
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 6, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

Berikut ini grafik objective function terhadap iterasi :

Run	Grafik
1	

2	
3	

4.2.4. Random-Restart Hill Climbing

Random-Restart memperoleh hasil terbaik di antara semua varian, dengan nilai 345 pada run 1. Parameter yang digunakan adalah max_restarts = 10, sehingga setiap run terdiri atas 1 pencarian awal dan 10 restart (total 11 pencarian Steepest-Ascent).

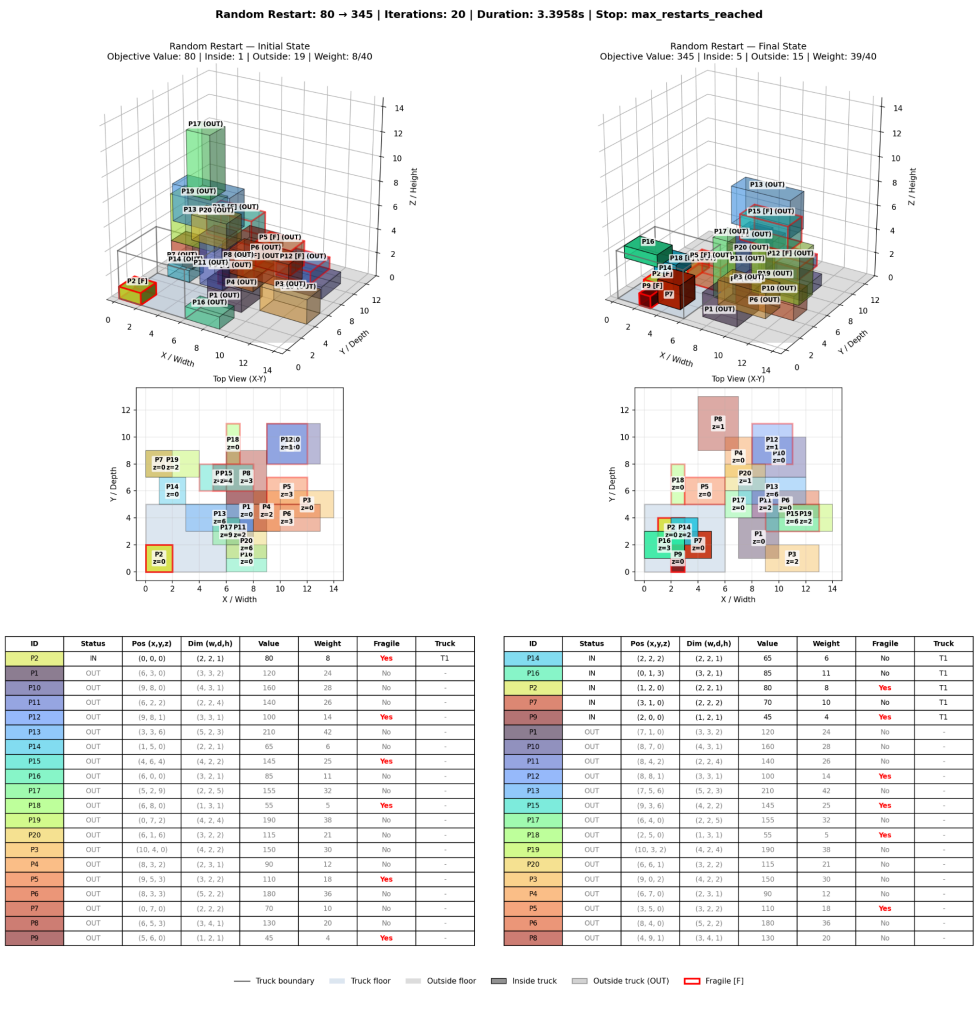
Run	Nilai Awal	Nilai Akhir	Banyak Restart	Total Iterasi	Durasi (detik)	Alasan Berhenti
1	80	345	10	20	3,96	batas restart tercapai
2	255	310	10	22	4,34	batas restart tercapai
3	320	335	10	22	5,80	batas restart tercapai

State Akhir:

Run	Paket di Dalam Truk	Berat
1	P2, P7, P9, P14, P16	39/40
2	P2, P8, P9, P18	37/40
3	P2, P4, P12, P14	40/40

Nilai akhir setiap pencarian bervariasi dari 220 hingga 345, yang menunjukkan banyaknya local maximum pada persoalan ini. Pada run 2, restart memungkinkan algoritma keluar dari state awal yang sudah penuh (255) dan mencapai 310.

Berikut ini gambaran paket yang termuat :

Run	State
1	

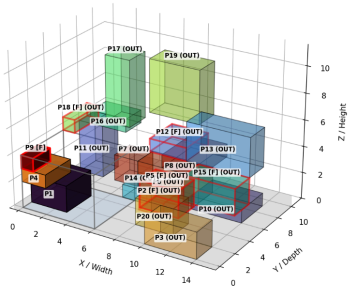
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P2	IN	(0, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P1	OUT	(6, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(9, 8, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(9, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 3, 6)	(5, 2, 3)	210	42	No	-
P14	OUT	(1, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(4, 6, 4)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 0, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(5, 2, 9)	(2, 2, 5)	155	32	No	-
P18	OUT	(6, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(0, 7, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 1, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 4, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(8, 3, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(9, 5, 3)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 3, 3)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 5, 3)	(3, 4, 1)	130	20	No	-
P9	OUT	(5, 6, 0)	(1, 2, 1)	45	4	Yes	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(2, 2, 2)	(2, 2, 1)	65	6	No	T1
P16	IN	(0, 1, 3)	(3, 2, 1)	85	11	No	T1
P2	IN	(1, 2, 0)	(2, 2, 1)	80	8	Yes	T1
P7	IN	(3, 1, 0)	(2, 2, 2)	70	10	No	T1
P9	IN	(2, 0, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(7, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(8, 7, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 4, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(8, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(7, 5, 6)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 3, 6)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(6, 4, 0)	(2, 2, 5)	155	32	No	-
P18	OUT	(2, 5, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 3, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 6, 1)	(3, 2, 2)	115	21	No	-
P3	OUT	(9, 0, 2)	(4, 2, 2)	150	30	No	-
P4	OUT	(6, 7, 0)	(2, 3, 1)	90	12	No	-
P5	OUT	(3, 5, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 4, 0)	(5, 2, 2)	180	36	No	-
P8	OUT	(4, 9, 1)	(3, 4, 1)	130	20	No	-

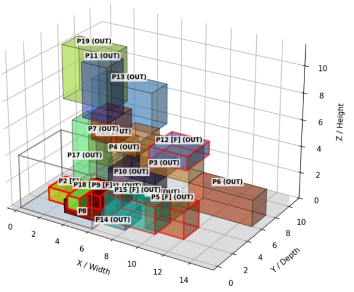
2

Random Restart: 255 → 310 | Iterations: 22 | Duration: 4.0047s | Stop: max_restarts_reached

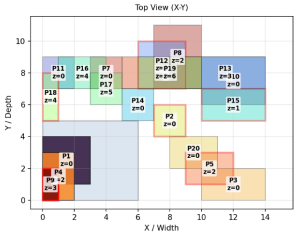
Random Restart — Initial State
Objective Value: 255 | Inside: 3 | Outside: 17 | Weight: 40/40



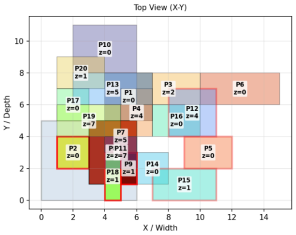
Random Restart — Final State
Objective Value: 310 | Inside: 4 | Outside: 16 | Weight: 37/40



Top View (X-Y)



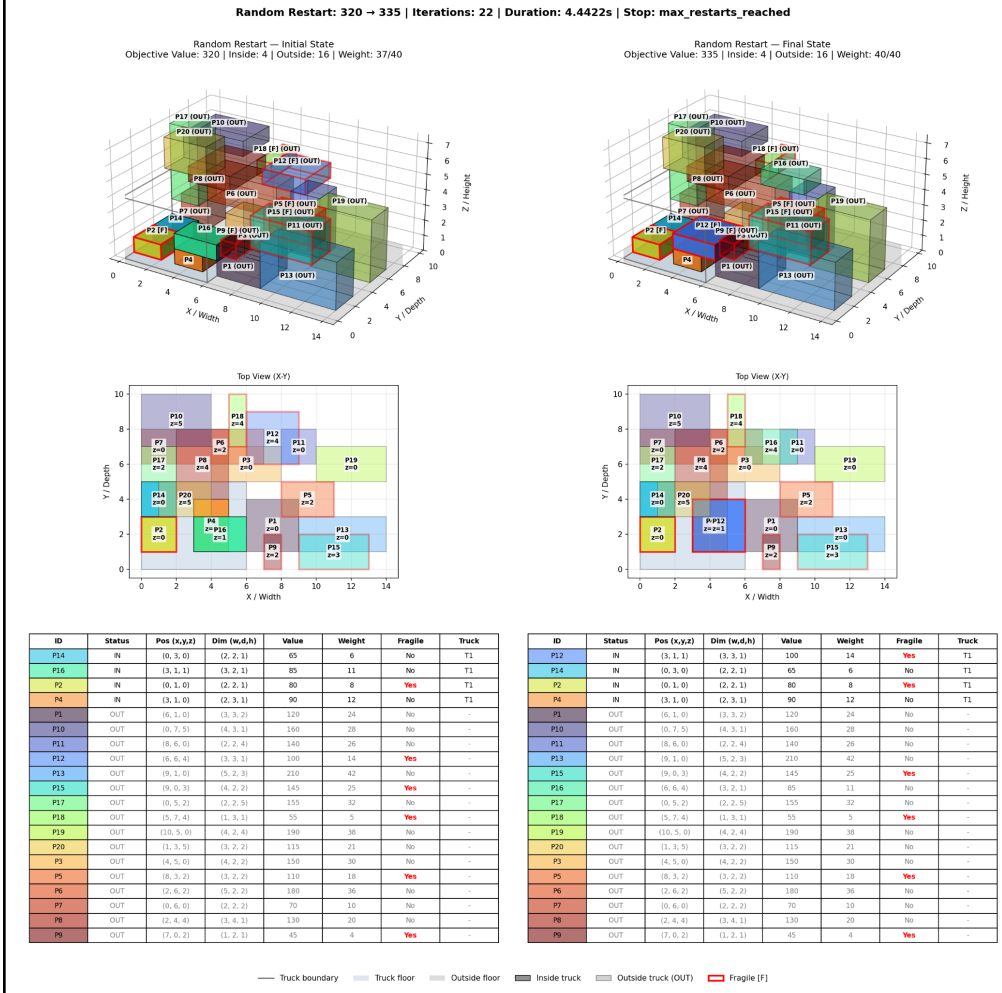
Top View (X-Y)



ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 1, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(0, 0, 3)	(1, 2, 1)	45	4	Yes	T1
P10	OUT	(10, 6, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 7, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 7, 3)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 5, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 7, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(3, 6, 5)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(6, 7, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 4, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(8, 2, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 1, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 7, 2)	(3, 4, 1)	130	20	No	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P18	IN	(4, 0, 1)	(1, 3, 1)	55	5	Yes	T1
P2	IN	(1, 2, 0)	(2, 2, 1)	80	8	Yes	T1
P8	IN	(3, 1, 0)	(3, 4, 1)	130	20	No	T1
P9	IN	(5, 1, 1)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(4, 5, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(2, 8, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(4, 2, 7)	(2, 2, 4)	140	26	No	-
P12	OUT	(8, 4, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(2, 6, 5)	(5, 2, 3)	210	42	No	-
P14	OUT	(6, 1, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(7, 0, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(7, 4, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(1, 5, 0)	(2, 2, 5)	155	32	No	-
P19	OUT	(1, 4, 7)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 7, 1)	(3, 2, 2)	115	21	No	-
P3	OUT	(6, 6, 2)	(4, 2, 2)	150	30	No	-
P4	OUT	(5, 4, 4)	(2, 3, 1)	90	12	No	-
P5	OUT	(9, 2, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(10, 6, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(4, 3, 5)	(2, 2, 2)	70	10	No	-

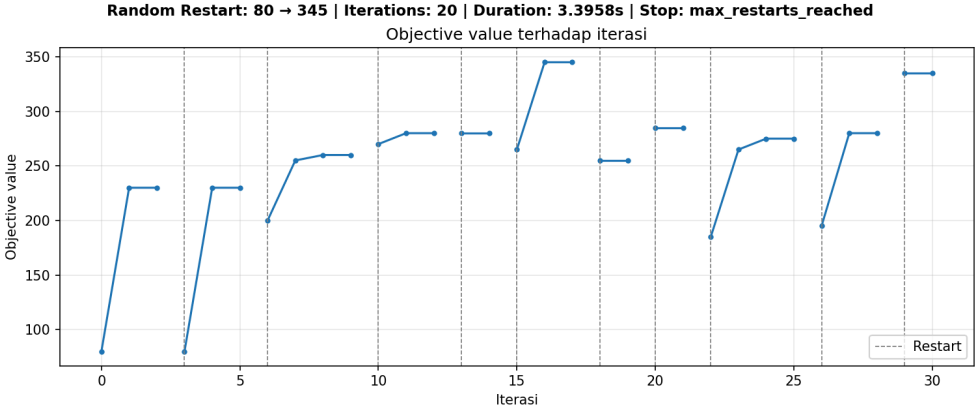
Truck boundary · Truck floor · Outside floor · Inside truck · Outside truck (OUT) · Fragile [F]

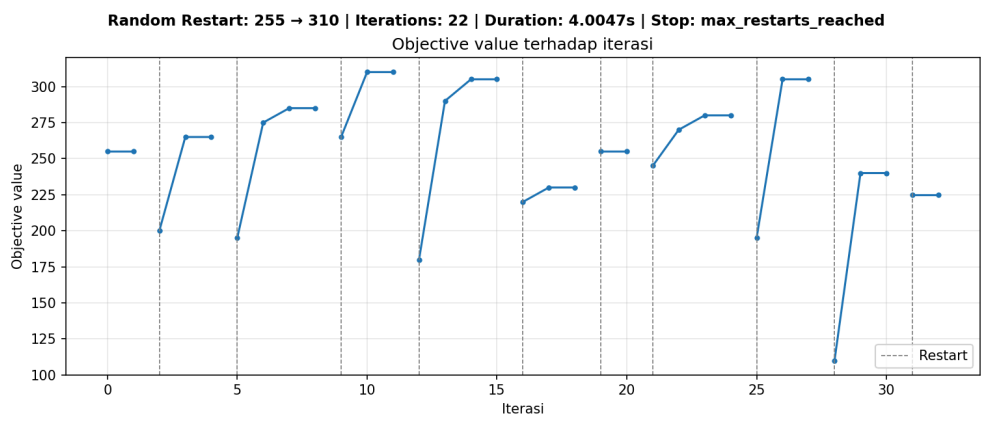
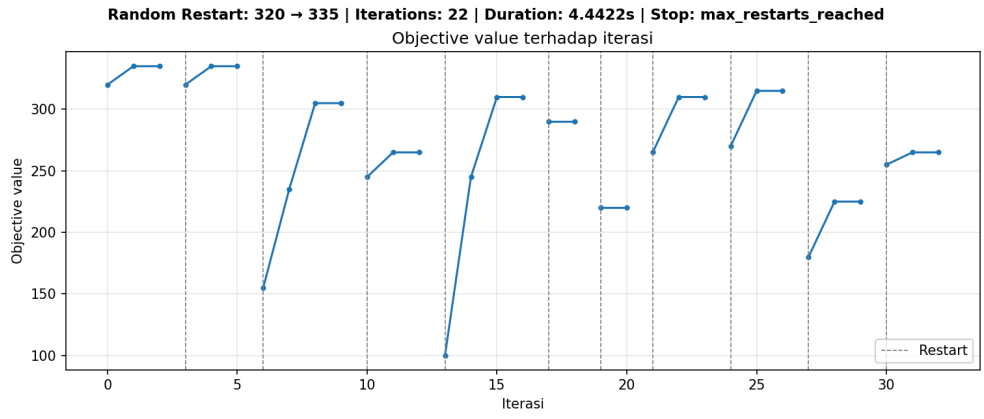
3	

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(3, 1, 1)	(3, 2, 1)	85	11	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 6, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P12	IN	(3, 1, 1)	(3, 3, 1)	100	14	Yes	T1
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 6, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

Berikut ini grafik objective function terhadap iterasi :

Run	Grafik
1	

2	
3	

4.2.5. Ringkasan Hasil Hill Climbing

Tidak ada varian Hill Climbing yang mencapai global maximum 355. Random-Restart memberikan hasil terbaik dan paling konsisten, dengan rata-rata 330 dan selisih terbaik hanya 10 dari optimum.

Random-Restart membutuhkan waktu sekitar 15 kali Steepest-Ascent karena menjalankan 11 kali pencarian. Sideways Move adalah varian paling lambat, karena setiap sideways move tetap membangkitkan dan mengevaluasi seluruh neighbor, sementara hasilnya hampir tidak berbeda dari Steepest-Ascent. Stochastic adalah varian paling efisien: dengan durasi di bawah 1 detik, rata-ratanya lebih baik daripada Steepest-Ascent dan Sideways Move.

Run 2 menunjukkan pengaruh state awal secara jelas. Karena state awalnya sudah penuh (40/40), tidak ada satu operasi swap, move, atau rotate yang dapat menaikkan nilai tanpa melanggar kapasitas. Akibatnya, Steepest-Ascent, Sideways Move, dan Stochastic tetap di 255, dan hanya

Random-Restart yang dapat keluar dengan memulai ulang dari state awal yang berbeda.

4.3. Hasil Simulated Annealing

Eksperimen menggunakan 20 paket dengan peti kemas berukuran (6x5x4) dan kapasitas berat maksimum 40 satuan. Fungsi objektif yang digunakan adalah memaksimalkan total nilai paket yang berhasil dimuat. Setiap run dimulai dari state awal acak yang berbeda, dengan seed 1, 2, dan 3.

Parameter Simulated Annealing yang digunakan adalah cooling rate 0,999, temperatur minimum 0,001, maksimum 5.000 iterasi, maksimum 100 percobaan untuk memperoleh neighbor valid, dan stuck_threshold sebesar 20. Temperatur awal dikalibrasi secara otomatis menggunakan 100 sampel neighbor, dengan target rata-rata probabilitas penerimaan sebesar 0,9.

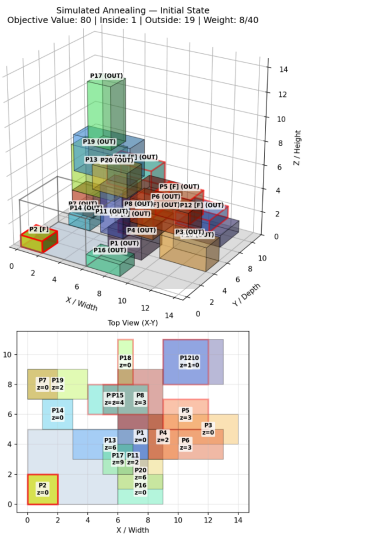
Run	Nilai Objective Function		Durasi (detik)	Frekuensi Stuck
	Awal	Akhir		
1	80	295	2,230090	54
2	225	305	2,269976	48
3	320	230	2,441342	42

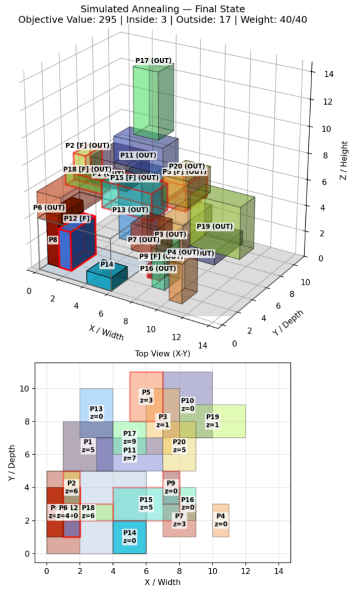
Pada tabel di atas, run 1 mencatat 54 kejadian stuck, Run 2 sebanyak 48, dan Run 3 sebanyak 42. Dalam implementasi ini, satu kejadian stuck dicatat ketika terdapat 20 iterasi berturut-turut tanpa perubahan nilai objektif. Kondisi ini dapat terjadi karena kandidat ditolak atau kandidat dengan nilai objektif sama diterima. Oleh karena itu, frekuensi stuck merupakan indikator stagnasi sementara dan bukan jumlah local optimum yang berhasil diidentifikasi secara pasti.

Dari sisi durasi, ketiga run memiliki durasi yang relatif berdekatan, yaitu sekitar 2,23–2,44 detik. Namun, nilai objektif akhirnya memiliki rentang yang cukup besar, yakni 230–305. Run 1 dan Run 2 berhasil meningkatkan nilai objektif dari state awal, sedangkan Run 3 berakhir dengan nilai yang lebih rendah daripada state awalnya. Hal ini menunjukkan bahwa hasil akhir Simulated Annealing masih dipengaruhi oleh state awal dan proses eksplorasi

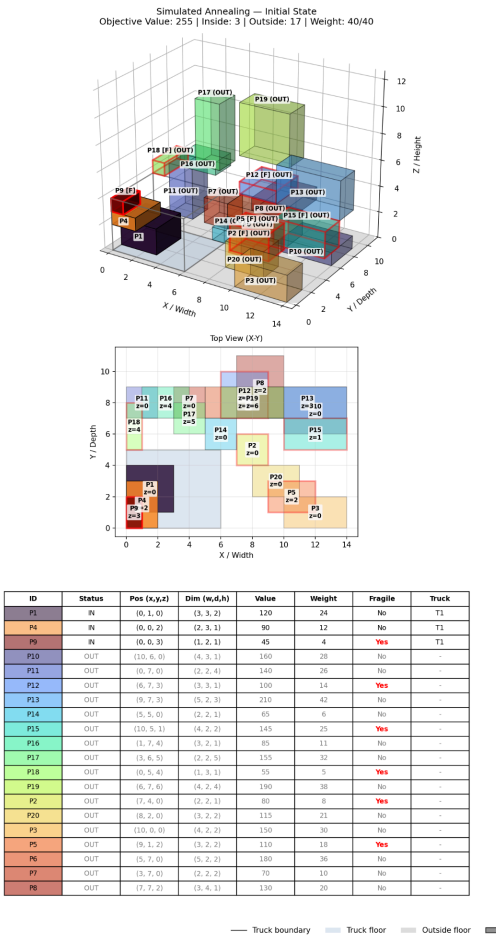
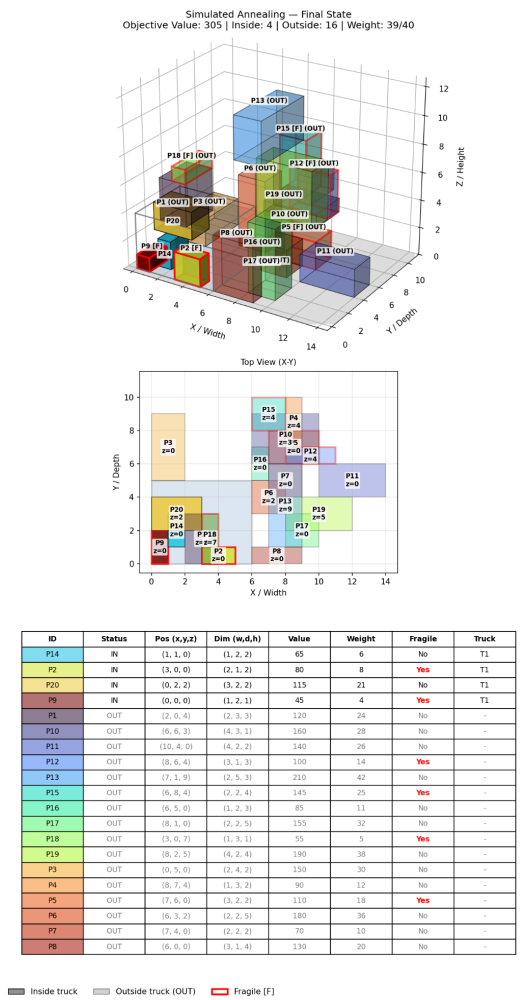
yang bersifat acak. Ketiga run juga berhenti setelah mencapai batas maksimum 5.000 iterasi.

Berikut ini paket yang termuat pada tiap state :

<table>
<tr><th>Run</th><th>State Awal</th><th>State Akhir</th></tr>
<tr><td>1</td><td>P2
Simulated Annealing: 80 → 295 | Iterations: 5000 | D
 

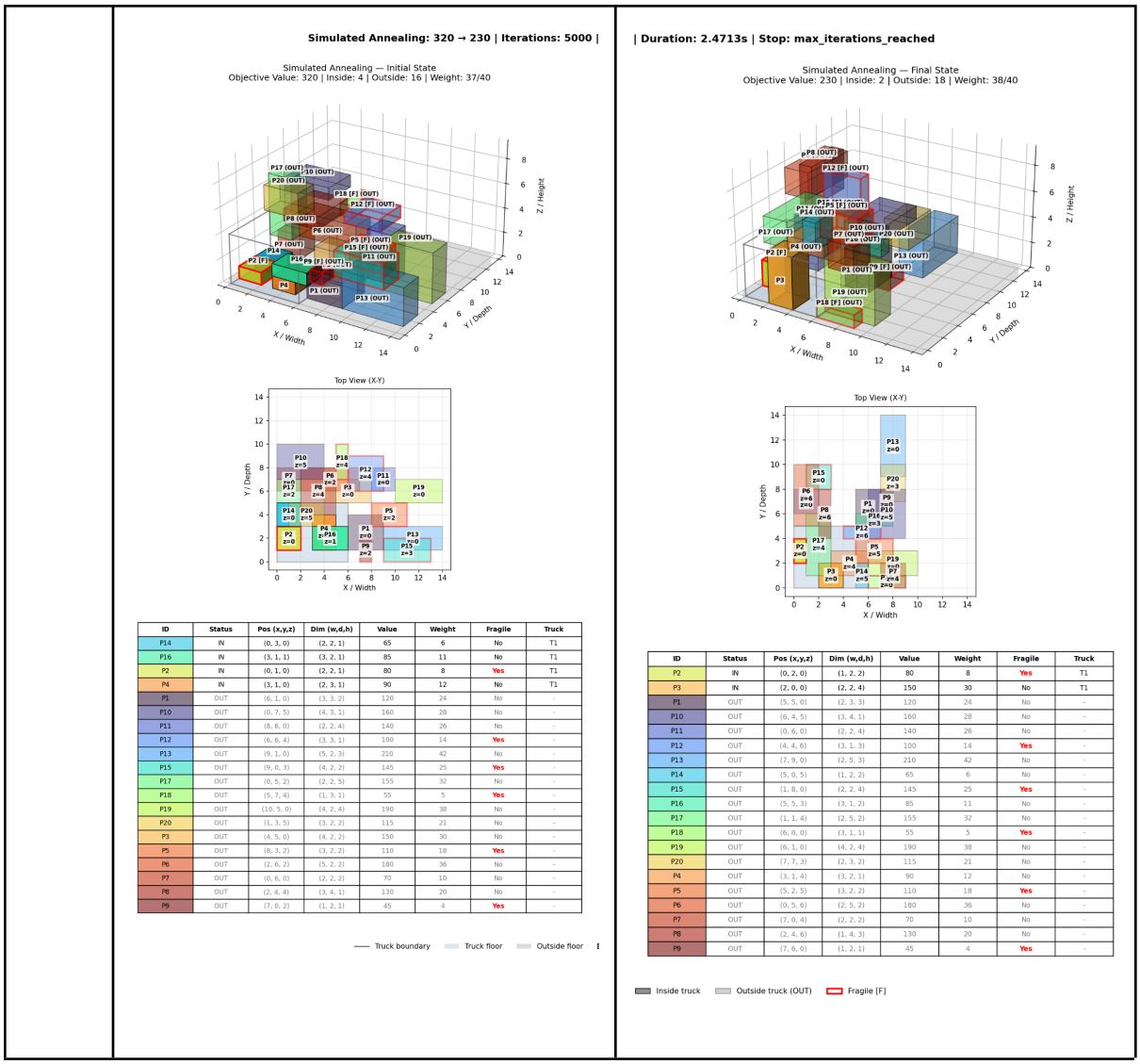
<table>
<tr><th>ID</th><th>Status</th><th>Pos (x,y,z)</th><th>Dim (w,d,h)</th><th>Value</th><th>Weight</th><th>Fragile</th><th>Truck</th></tr>
<tr><td>P2</td><td>IN</td><td>(0, 0, 0)</td><td>(2, 2, 1)</td><td>80</td><td>8</td><td>Yes</td><td>T1</td></tr>
<tr><td>P1</td><td>OUT</td><td>(6, 3, 0)</td><td>(3, 3, 2)</td><td>120</td><td>24</td><td>No</td><td>-</td></tr>
<tr><td>P10</td><td>OUT</td><td>(9, 8, 0)</td><td>(4, 3, 1)</td><td>160</td><td>28</td><td>No</td><td>-</td></tr>
<tr><td>P11</td><td>OUT</td><td>(6, 2, 2)</td><td>(2, 2, 4)</td><td>140</td><td>26</td><td>No</td><td>-</td></tr>
<tr><td>P12</td><td>OUT</td><td>(9, 8, 1)</td><td>(3, 3, 1)</td><td>100</td><td>14</td><td>Yes</td><td>-</td></tr>
<tr><td>P13</td><td>OUT</td><td>(3, 3, 6)</td><td>(5, 2, 3)</td><td>210</td><td>42</td><td>No</td><td>-</td></tr>
<tr><td>P14</td><td>OUT</td><td>(1, 5, 0)</td><td>(2, 2, 1)</td><td>65</td><td>6</td><td>No</td><td>-</td></tr>
<tr><td>P15</td><td>OUT</td><td>(4, 6, 4)</td><td>(4, 2, 2)</td><td>145</td><td>25</td><td>Yes</td><td>-</td></tr>
<tr><td>P16</td><td>OUT</td><td>(6, 0, 0)</td><td>(3, 2, 1)</td><td>85</td><td>11</td><td>No</td><td>-</td></tr>
<tr><td>P17</td><td>OUT</td><td>(5, 2, 9)</td><td>(2, 2, 5)</td><td>155</td><td>32</td><td>No</td><td>-</td></tr>
<tr><td>P18</td><td>OUT</td><td>(6, 8, 0)</td><td>(1, 3, 1)</td><td>55</td><td>5</td><td>Yes</td><td>-</td></tr>
<tr><td>P19</td><td>OUT</td><td>(0, 7, 2)</td><td>(4, 2, 4)</td><td>190</td><td>38</td><td>No</td><td>-</td></tr>
<tr><td>P20</td><td>OUT</td><td>(6, 1, 6)</td><td>(3, 2, 2)</td><td>115</td><td>21</td><td>No</td><td>-</td></tr>
<tr><td>P3</td><td>OUT</td><td>(10, 4, 0)</td><td>(4, 2, 2)</td><td>150</td><td>30</td><td>No</td><td>-</td></tr>
<tr><td>P4</td><td>OUT</td><td>(8, 3, 2)</td><td>(2, 3, 1)</td><td>90</td><td>12</td><td>No</td><td>-</td></tr>
<tr><td>P5</td><td>OUT</td><td>(9, 5, 3)</td><td>(3, 2, 2)</td><td>110</td><td>18</td><td>Yes</td><td>-</td></tr>
<tr><td>P6</td><td>OUT</td><td>(8, 3, 3)</td><td>(5, 2, 2)</td><td>180</td><td>36</td><td>No</td><td>-</td></tr>
<tr><td>P7</td><td>OUT</td><td>(0, 7, 0)</td><td>(2, 2, 2)</td><td>70</td><td>10</td><td>No</td><td>-</td></tr>
<tr><td>P8</td><td>OUT</td><td>(6, 5, 3)</td><td>(3, 4, 1)</td><td>130</td><td>20</td><td>No</td><td>-</td></tr>
<tr><td>P9</td><td>OUT</td><td>(5, 6, 0)</td><td>(1, 2, 1)</td><td>45</td><td>4</td><td>Yes</td><td>-</td></tr>
</table>
Truck boundary Truck floor Outside floor</td><td>P8, P12, P14,
| Duration: 2.3239s | Stop: max_iterations_reached
 

<table>
<tr><th>ID</th><th>Status</th><th>Pos (x,y,z)</th><th>Dim (w,d,h)</th><th>Value</th><th>Weight</th><th>Fragile</th><th>Truck</th></tr>
<tr><td>P12</td><td>IN</td><td>(1, 1, 0)</td><td>(1, 3, 3)</td><td>100</td><td>14</td><td>Yes</td><td>T1</td></tr>
<tr><td>P14</td><td>IN</td><td>(4, 0, 0)</td><td>(2, 2, 1)</td><td>65</td><td>6</td><td>No</td><td>T1</td></tr>
<tr><td>P8</td><td>IN</td><td>(0, 1, 0)</td><td>(1, 3, 4)</td><td>130</td><td>20</td><td>No</td><td>T1</td></tr>
<tr><td>P1</td><td>OUT</td><td>(1, 5, 5)</td><td>(3, 3, 2)</td><td>120</td><td>24</td><td>No</td><td>-</td></tr>
<tr><td>P10</td><td>OUT</td><td>(7, 7, 0)</td><td>(3, 4, 1)</td><td>160</td><td>28</td><td>No</td><td>-</td></tr>
<tr><td>P11</td><td>OUT</td><td>(3, 5, 7)</td><td>(4, 2, 2)</td><td>140</td><td>26</td><td>No</td><td>-</td></tr>
<tr><td>P13</td><td>OUT</td><td>(2, 7, 0)</td><td>(2, 3, 5)</td><td>210</td><td>42</td><td>No</td><td>-</td></tr>
<tr><td>P15</td><td>OUT</td><td>(4, 2, 5)</td><td>(4, 2, 2)</td><td>145</td><td>25</td><td>Yes</td><td>-</td></tr>
<tr><td>P16</td><td>OUT</td><td>(8, 2, 0)</td><td>(1, 2, 3)</td><td>85</td><td>11</td><td>No</td><td>-</td></tr>
<tr><td>P17</td><td>OUT</td><td>(4, 6, 9)</td><td>(2, 2, 5)</td><td>155</td><td>32</td><td>No</td><td>-</td></tr>
<tr><td>P18</td><td>OUT</td><td>(1, 2, 6)</td><td>(3, 1, 1)</td><td>55</td><td>5</td><td>Yes</td><td>-</td></tr>
<tr><td>P19</td><td>OUT</td><td>(8, 7, 1)</td><td>(4, 2, 4)</td><td>190</td><td>38</td><td>No</td><td>-</td></tr>
<tr><td>P2</td><td>OUT</td><td>(1, 3, 6)</td><td>(1, 2, 2)</td><td>80</td><td>8</td><td>Yes</td><td>-</td></tr>
<tr><td>P20</td><td>OUT</td><td>(7, 5, 5)</td><td>(2, 3, 2)</td><td>115</td><td>21</td><td>No</td><td>-</td></tr>
<tr><td>P3</td><td>OUT</td><td>(6, 6, 1)</td><td>(2, 4, 2)</td><td>150</td><td>30</td><td>No</td><td>-</td></tr>
<tr><td>P4</td><td>OUT</td><td>(10, 1, 0)</td><td>(1, 2, 3)</td><td>90</td><td>12</td><td>No</td><td>-</td></tr>
<tr><td>P5</td><td>OUT</td><td>(5, 8, 3)</td><td>(2, 3, 2)</td><td>110</td><td>18</td><td>Yes</td><td>-</td></tr>
<tr><td>P6</td><td>OUT</td><td>(0, 0, 4)</td><td>(2, 5, 2)</td><td>180</td><td>36</td><td>No</td><td>-</td></tr>
<tr><td>P7</td><td>OUT</td><td>(7, 1, 3)</td><td>(2, 2, 2)</td><td>70</td><td>10</td><td>No</td><td>-</td></tr>
<tr><td>P9</td><td>OUT</td><td>(7, 3, 0)</td><td>(1, 2, 1)</td><td>45</td><td>4</td><td>Yes</td><td>-</td></tr>
</table>
Inside truck Outside truck (OUT) Fragile [F]</td></tr>
<tr><td>2</td><td>P1, P4, P9</td><td>P2, P9, P14, P20</td></tr>
</table>

	Simulated Annealing: 255 → 305 | Iterations: 5000 | D 	**| Duration: 2.2547s | Stop: max_iterations_reached** 
3	P2, P4, P14, P16	P2, P3

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P1	IN	(0, 1, 0)	(3, 3, 2)	120	24	No	T1
P4	IN	(0, 0, 2)	(2, 3, 1)	90	12	No	T1
P9	IN	(0, 0, 3)	(1, 2, 1)	45	4	Yes	T1
P10	OUT	(10, 6, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 7, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 7, 3)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(10, 5, 1)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 7, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(3, 6, 5)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(6, 7, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(7, 4, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(8, 2, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 1, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(5, 7, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(3, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 7, 2)	(3, 4, 1)	130	20	No	-

ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(1, 1, 0)	(1, 2, 2)	65	6	No	T1
P2	IN	(3, 0, 0)	(2, 1, 2)	80	8	Yes	T1
P20	IN	(0, 2, 2)	(3, 2, 2)	115	21	No	T1
P9	IN	(0, 0, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(2, 0, 4)	(2, 3, 3)	120	24	No	-
P10	OUT	(6, 6, 3)	(4, 3, 1)	160	28	No	-
P11	OUT	(10, 4, 0)	(4, 2, 2)	140	26	No	-
P12	OUT	(8, 6, 4)	(3, 1, 3)	100	14	Yes	-
P13	OUT	(7, 1, 9)	(2, 5, 3)	210	42	No	-
P15	OUT	(6, 8, 4)	(2, 2, 4)	145	25	Yes	-
P16	OUT	(6, 5, 0)	(1, 2, 3)	85	11	No	-
P17	OUT	(8, 1, 0)	(2, 2, 5)	155	32	No	-
P18	OUT	(3, 0, 7)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(8, 2, 5)	(4, 2, 4)	190	38	No	-
P3	OUT	(0, 5, 0)	(2, 4, 2)	150	30	No	-
P4	OUT	(8, 7, 4)	(1, 3, 2)	90	12	No	-
P5	OUT	(7, 6, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(6, 3, 2)	(2, 2, 5)	180	36	No	-
P7	OUT	(7, 4, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(6, 0, 0)	(3, 1, 4)	130	20	No	-



ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(0, 3, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(3, 1, 1)	(3, 2, 1)	85	11	No	T1
P2	IN	(0, 1, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(6, 1, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 5)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 6, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 6, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 1, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 0, 3)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(0, 5, 2)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 4)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(1, 3, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(8, 3, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(2, 6, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(0, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 4)	(3, 4, 1)	130	20	No	-
P9	OUT	(7, 0, 2)	(1, 2, 1)	45	4	Yes	-

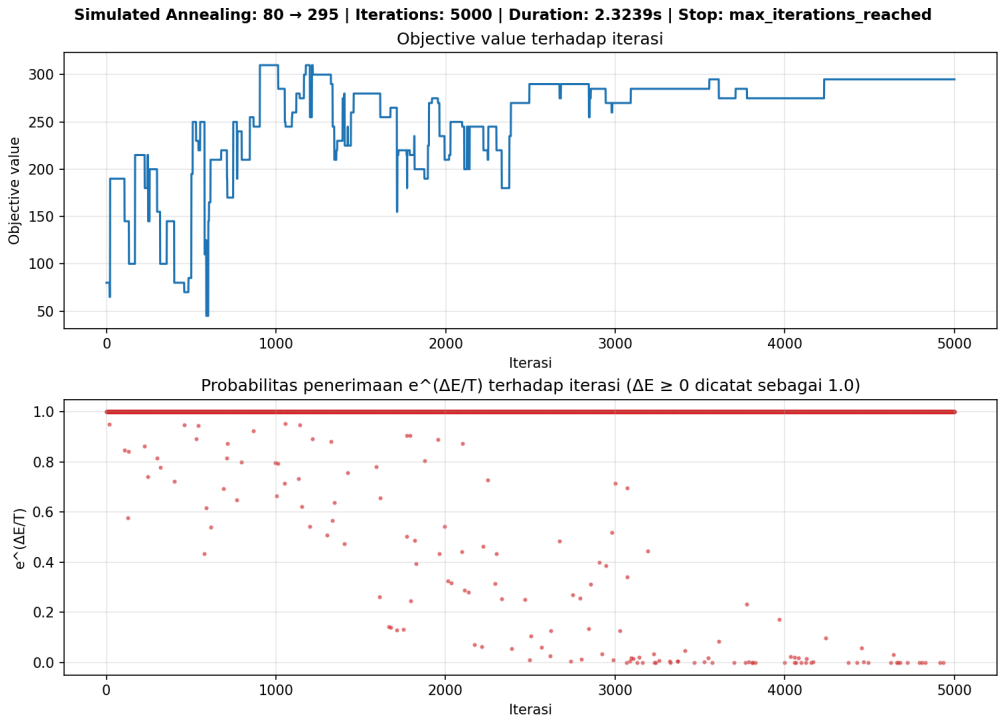
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P2	IN	(0, 2, 0)	(1, 2, 2)	80	8	Yes	T1
P3	IN	(2, 0, 0)	(2, 2, 4)	150	30	No	T1
P1	OUT	(5, 5, 0)	(2, 3, 3)	120	24	No	-
P10	OUT	(6, 4, 5)	(3, 4, 1)	160	28	No	-
P11	OUT	(0, 6, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(4, 4, 6)	(3, 1, 3)	100	14	Yes	-
P13	OUT	(7, 9, 0)	(2, 5, 3)	210	42	No	-
P14	OUT	(5, 0, 5)	(1, 2, 2)	65	6	No	-
P15	OUT	(1, 8, 0)	(2, 2, 4)	145	25	Yes	-
P16	OUT	(5, 5, 3)	(3, 1, 2)	85	11	No	-
P17	OUT	(1, 1, 4)	(2, 5, 2)	155	32	No	-
P18	OUT	(6, 0, 0)	(3, 1, 1)	55	5	Yes	-
P19	OUT	(6, 1, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(7, 7, 3)	(2, 3, 2)	115	21	No	-
P4	OUT	(3, 1, 4)	(3, 2, 1)	90	12	No	-
P5	OUT	(5, 2, 5)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(0, 5, 6)	(2, 5, 2)	180	36	No	-
P7	OUT	(7, 0, 4)	(2, 2, 2)	70	10	No	-
P8	OUT	(2, 4, 6)	(1, 4, 3)	130	20	No	-
P9	OUT	(7, 6, 0)	(1, 2, 1)	45	4	Yes	-

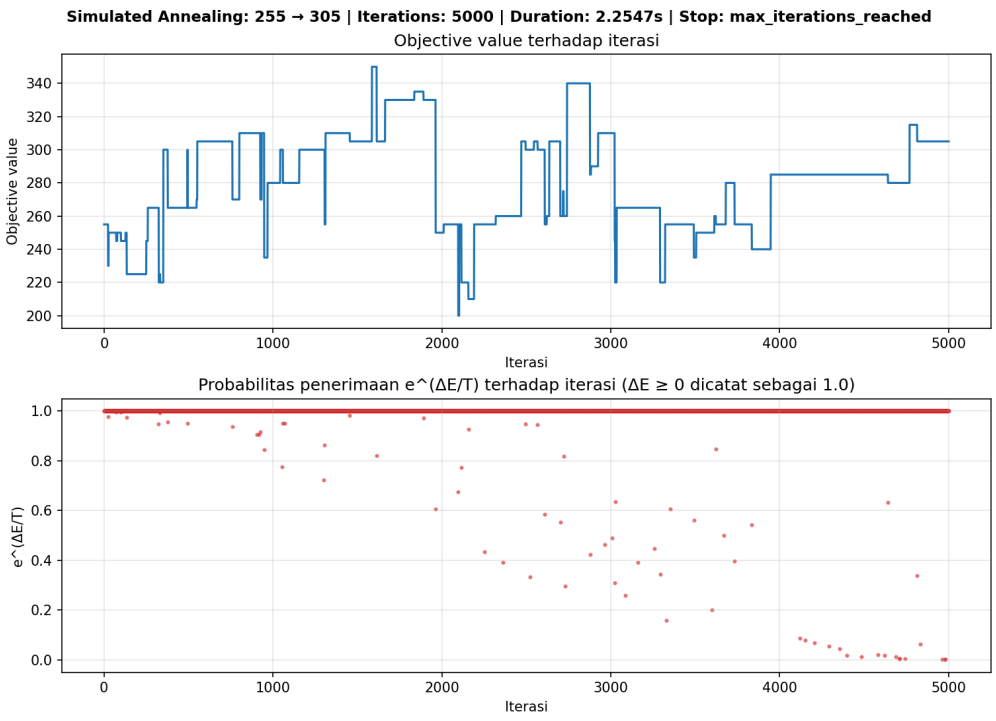
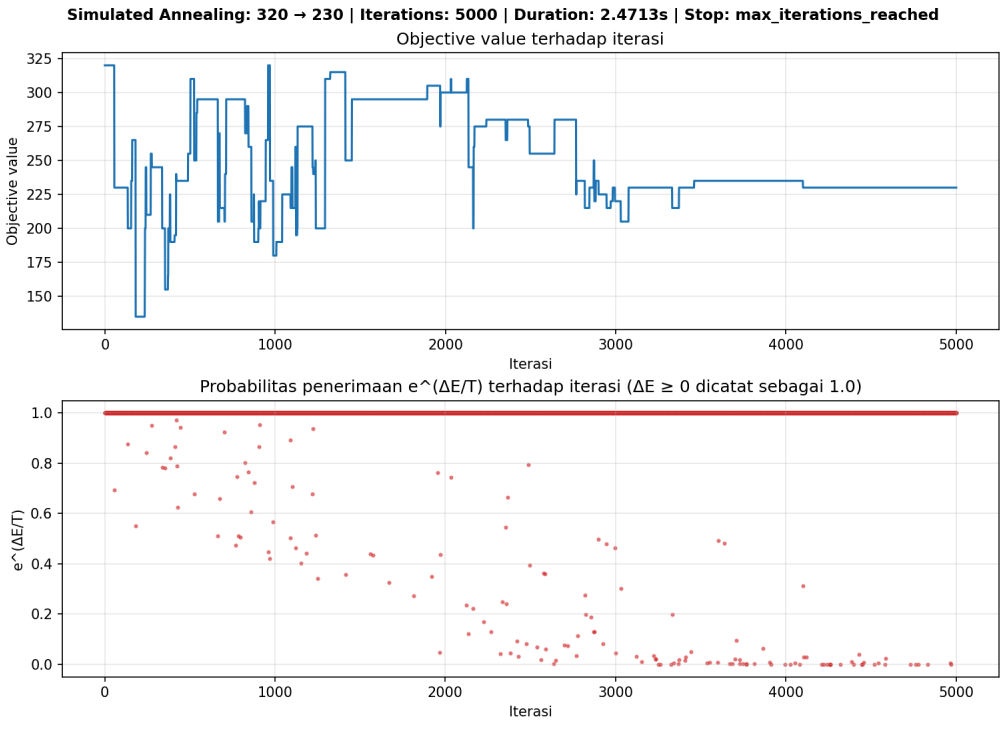
Berdasarkan pengujian tersebut, nilai global optimum untuk persoalan pengisian peti kemas ini adalah 355, yang diperoleh dari kombinasi paket P2, P4, P9, P16, dan P18 dengan total berat 40 satuan. Nilai tersebut digunakan sebagai acuan untuk mengevaluasi kualitas solusi yang dihasilkan oleh Simulated Annealing. Hasil eksperimen menunjukkan bahwa Run 1 menghasilkan nilai objektif akhir 295, Run 2 menghasilkan 305, dan Run 3 menghasilkan 230. Run 2 menghasilkan solusi akhir terbaik dengan nilai objektif 305, atau sekitar 85,92% dari global optimum. Sementara itu, Run 3 memperoleh nilai akhir terendah, yaitu 230, atau sekitar 64,79% dari nilai optimum.

Temuan yang lebih menarik terlihat pada nilai objektif terbaik yang pernah ditemukan selama pencarian (best_value_seen). Run 1 pernah mencapai

nilai 310, Run 2 mencapai 350, dan Run 3 mencapai 320. Dengan demikian, Run 2 sebenarnya pernah menemukan solusi dengan nilai 350, yang setara dengan 98,59% dari global optimum. Namun, nilai objektif state akhirnya hanya 305. Hal tersebut terjadi karena implementasi menyimpan nilai terbaik yang pernah ditemukan secara terpisah dari current state. Ketika algoritma menerima perpindahan ke solusi yang lebih buruk, current state dapat berubah menjadi solusi dengan nilai objektif lebih rendah. Jika pencarian berakhir pada state tersebut, state akhir yang dikembalikan belum tentu merupakan solusi terbaik sepanjang proses pencarian.

Selain itu, berikut ini grafik objective function terhadap iterasi :

Run	Grafik
1	

2	
3	

Grafik objective function memperlihatkan bahwa nilai objektif dapat meningkat, menurun, atau bertahan selama beberapa iterasi. Sementara itu, grafik probabilitas penerimaan menunjukkan bahwa seiring penurunan

temperatur, peluang menerima neighbor yang lebih buruk semakin kecil. Mekanisme ini membantu algoritma mengurangi penerimaan solusi yang kurang baik pada tahap akhir pencarian. Namun, mekanisme tersebut tidak menjamin algoritma berakhir pada solusi optimal.

Sebagai kesimpulan, Simulated Annealing mampu menemukan solusi yang mendekati global optimum, tetapi kualitas state akhirnya belum konsisten. Salah satu perbaikan yang dapat dilakukan adalah menyimpan state terbaik yang pernah ditemukan dan mengembalikannya sebagai solusi akhir. Selain itu, eksperimen dengan variasi jumlah iterasi dan parameter temperatur dapat dilakukan untuk mengetahui pengaruhnya terhadap kualitas solusi serta durasi pencarian.

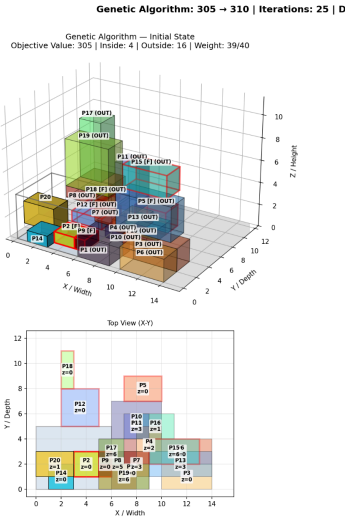
4.4. Hasil Genetic Algorithm

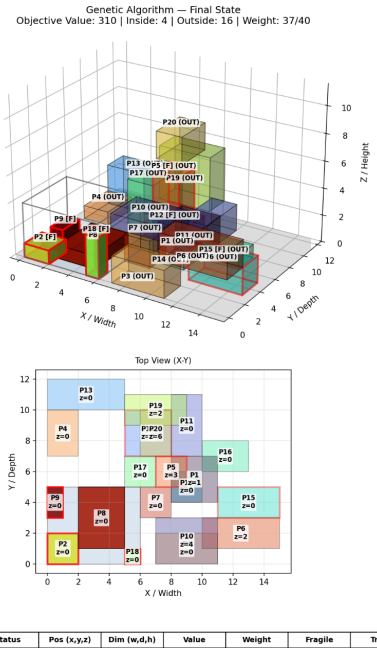
1. Populasi sebagai kontrol (*nPopulation*)

Run	nPopulation	nGeneration	Nilai objective function pada generasi		Durasi (detik)
			awal	akhir	
1	20	25	305	310	1.542594 20
2	20	25	300	305	1.578315
3	20	25	295	300	2.139679 5
1	20	50	270	335	2.996206 4
2	20	50	310	310	2.617391 7
3	20	50	285	315	2.804296 49

1	20	100	295	355	5.084915
2	20	100	300	355	5.170235 1
3	20	100	295	335	4.906789

Setelah dijalankan, berikut kondisi state:

<table>
<tr><th colspan="3">nPopulation = 20; nGeneration = 25</th></tr>
<tr><th>Run</th><th>State Awal</th><th>State Akhir</th></tr>
<tr><td>1</td><td>
Genetic Algorithm: 305 → 310 | Iterations: 25 | D


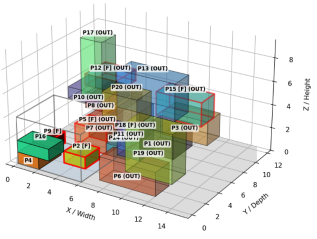
<table>
<tr><th>ID</th><th>Status</th><th>Pos (x,y,z)</th><th>Dim (w,d,h)</th><th>Value</th><th>Weight</th><th>Fragile</th><th>Truck</th></tr>
<tr><td>P14</td><td>IN</td><td>(1, 0, 0)</td><td>(2, 2, 1)</td><td>65</td><td>6</td><td>No</td><td>T1</td></tr>
<tr><td>P2</td><td>IN</td><td>(3, 1, 0)</td><td>(2, 2, 1)</td><td>80</td><td>8</td><td>Yes</td><td>T1</td></tr>
<tr><td>P20</td><td>IN</td><td>(0, 1, 1)</td><td>(3, 2, 2)</td><td>115</td><td>21</td><td>No</td><td>T1</td></tr>
<tr><td>P9</td><td>IN</td><td>(5, 1, 0)</td><td>(1, 2, 1)</td><td>45</td><td>4</td><td>Yes</td><td>T1</td></tr>
<tr><td>P1</td><td>OUT</td><td>(6, 0, 0)</td><td>(3, 3, 2)</td><td>120</td><td>24</td><td>No</td><td></td></tr>
<tr><td>P10</td><td>OUT</td><td>(6, 4, 0)</td><td>(4, 3, 1)</td><td>160</td><td>28</td><td>No</td><td></td></tr>
<tr><td>P11</td><td>OUT</td><td>(7, 4, 3)</td><td>(2, 2, 4)</td><td>140</td><td>26</td><td>No</td><td></td></tr>
<tr><td>P12</td><td>OUT</td><td>(2, 5, 0)</td><td>(3, 3, 1)</td><td>100</td><td>14</td><td>Yes</td><td></td></tr>
<tr><td>P13</td><td>OUT</td><td>(9, 1, 3)</td><td>(5, 2, 3)</td><td>210</td><td>42</td><td>No</td><td></td></tr>
<tr><td>P15</td><td>OUT</td><td>(9, 2, 6)</td><td>(4, 2, 2)</td><td>145</td><td>25</td><td>Yes</td><td></td></tr>
<tr><td>P16</td><td>OUT</td><td>(8, 4, 1)</td><td>(3, 2, 1)</td><td>85</td><td>11</td><td>No</td><td></td></tr>
<tr><td>P17</td><td>OUT</td><td>(5, 2, 6)</td><td>(2, 2, 5)</td><td>155</td><td>32</td><td>No</td><td></td></tr>
<tr><td>P18</td><td>OUT</td><td>(2, 8, 0)</td><td>(1, 3, 1)</td><td>55</td><td>5</td><td>Yes</td><td></td></tr>
<tr><td>P19</td><td>OUT</td><td>(5, 0, 6)</td><td>(4, 2, 4)</td><td>190</td><td>38</td><td>No</td><td></td></tr>
<tr><td>P3</td><td>OUT</td><td>(10, 0, 0)</td><td>(4, 2, 2)</td><td>150</td><td>30</td><td>No</td><td></td></tr>
<tr><td>P4</td><td>OUT</td><td>(8, 2, 2)</td><td>(2, 3, 1)</td><td>90</td><td>12</td><td>No</td><td></td></tr>
<tr><td>P5</td><td>OUT</td><td>(7, 7, 0)</td><td>(3, 2, 2)</td><td>110</td><td>18</td><td>Yes</td><td></td></tr>
<tr><td>P6</td><td>OUT</td><td>(9, 2, 0)</td><td>(5, 2, 2)</td><td>180</td><td>36</td><td>No</td><td></td></tr>
<tr><td>P7</td><td>OUT</td><td>(7, 1, 3)</td><td>(2, 2, 2)</td><td>70</td><td>10</td><td>No</td><td></td></tr>
<tr><td>P8</td><td>OUT</td><td>(5, 0, 5)</td><td>(3, 4, 1)</td><td>130</td><td>20</td><td>No</td><td></td></tr>
</table>
Truck boundary Truck floor Outside floor
</td><td>
ration: 1.6833s | Stop: max_iterations_reached


<table>
<tr><th>ID</th><th>Status</th><th>Pos (x,y,z)</th><th>Dim (w,d,h)</th><th>Value</th><th>Weight</th><th>Fragile</th><th>Truck</th></tr>
<tr><td>P18</td><td>IN</td><td>(5, 0, 0)</td><td>(1, 1, 3)</td><td>55</td><td>5</td><td>Yes</td><td>T1</td></tr>
<tr><td>P2</td><td>IN</td><td>(0, 0, 0)</td><td>(2, 2, 1)</td><td>80</td><td>8</td><td>Yes</td><td>T1</td></tr>
<tr><td>P8</td><td>IN</td><td>(2, 1, 0)</td><td>(3, 4, 1)</td><td>130</td><td>20</td><td>No</td><td>T1</td></tr>
<tr><td>P9</td><td>IN</td><td>(0, 3, 0)</td><td>(1, 2, 1)</td><td>45</td><td>4</td><td>Yes</td><td>T1</td></tr>
<tr><td>P1</td><td>OUT</td><td>(8, 4, 1)</td><td>(3, 3, 2)</td><td>120</td><td>24</td><td>No</td><td></td></tr>
<tr><td>P10</td><td>OUT</td><td>(7, 0, 4)</td><td>(4, 3, 1)</td><td>160</td><td>28</td><td>No</td><td></td></tr>
<tr><td>P11</td><td>OUT</td><td>(8, 7, 0)</td><td>(2, 4, 2)</td><td>140</td><td>26</td><td>No</td><td></td></tr>
<tr><td>P12</td><td>OUT</td><td>(5, 7, 0)</td><td>(3, 3, 1)</td><td>100</td><td>14</td><td>Yes</td><td></td></tr>
<tr><td>P13</td><td>OUT</td><td>(0, 10, 0)</td><td>(5, 2, 3)</td><td>210</td><td>42</td><td>No</td><td></td></tr>
<tr><td>P14</td><td>OUT</td><td>(8, 4, 0)</td><td>(2, 2, 1)</td><td>65</td><td>6</td><td>No</td><td></td></tr>
<tr><td>P15</td><td>OUT</td><td>(11, 3, 0)</td><td>(4, 2, 2)</td><td>145</td><td>25</td><td>Yes</td><td></td></tr>
<tr><td>P16</td><td>OUT</td><td>(10, 6, 0)</td><td>(3, 2, 1)</td><td>85</td><td>11</td><td>No</td><td></td></tr>
<tr><td>P17</td><td>OUT</td><td>(5, 5, 0)</td><td>(2, 2, 5)</td><td>155</td><td>32</td><td>No</td><td></td></tr>
<tr><td>P19</td><td>OUT</td><td>(5, 9, 2)</td><td>(4, 2, 4)</td><td>190</td><td>38</td><td>No</td><td></td></tr>
<tr><td>P20</td><td>OUT</td><td>(6, 7, 6)</td><td>(2, 3, 2)</td><td>115</td><td>21</td><td>No</td><td></td></tr>
<tr><td>P3</td><td>OUT</td><td>(7, 0, 0)</td><td>(4, 2, 2)</td><td>150</td><td>30</td><td>No</td><td></td></tr>
<tr><td>P4</td><td>OUT</td><td>(0, 7, 0)</td><td>(2, 3, 1)</td><td>90</td><td>12</td><td>No</td><td></td></tr>
<tr><td>P5</td><td>OUT</td><td>(7, 5, 3)</td><td>(2, 2, 3)</td><td>110</td><td>18</td><td>Yes</td><td></td></tr>
<tr><td>P6</td><td>OUT</td><td>(10, 1, 2)</td><td>(5, 2, 2)</td><td>180</td><td>36</td><td>No</td><td></td></tr>
<tr><td>P7</td><td>OUT</td><td>(6, 3, 0)</td><td>(2, 2, 2)</td><td>70</td><td>10</td><td>No</td><td></td></tr>
</table>
Inside truck Outside truck (OUT) Fragile [F]
</td></tr>
</table>

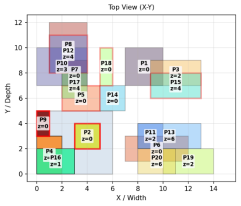
2

Genetic Algorithm: 300 → 305 | Iterations: 25 | Dur

Genetic Algorithm — Initial State
Objective Value: 300 | Inside: 4 | Outside: 16 | Weight: 35/40



Top View (X-Y)

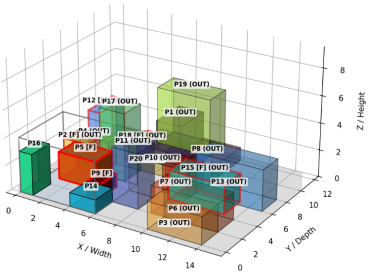


ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P16	IN	(0, 0, 1)	(3, 2, 1)	85	11	No	T1
P2	IN	(3, 2, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(0, 0, 0)	(2, 3, 1)	90	12	No	T1
P5	IN	(0, 3, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(7, 7, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(0, 7, 3)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(1, 8, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(8, 2, 6)	(5, 2, 3)	210	42	No	-
P14	OUT	(5, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(9, 6, 4)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(2, 6, 4)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 7, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 0, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(8, 0, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(9, 7, 2)	(4, 2, 2)	150	30	No	-
P5	OUT	(2, 5, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(7, 1, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(2, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(1, 8, 2)	(3, 4, 1)	130	20	No	-

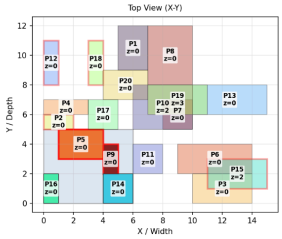
Truck boundary Truck floor Outside floor

Duration: 1.6166s | Stop: max_iterations_reached

Genetic Algorithm — Final State
Objective Value: 305 | Inside: 4 | Outside: 16 | Weight: 39/40



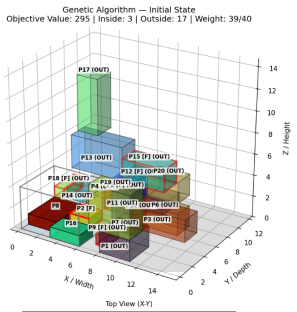
Top View (X-Y)

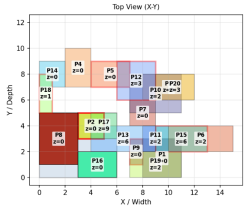


ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(4, 0, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(0, 0, 0)	(1, 2, 3)	85	11	No	T1
P5	IN	(1, 3, 0)	(3, 2, 2)	110	18	Yes	T1
P9	IN	(4, 2, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(5, 9, 0)	(2, 3, 3)	120	24	No	-
P10	OUT	(6, 5, 2)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 2, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(0, 8, 0)	(1, 3, 3)	100	14	Yes	-
P13	OUT	(10, 6, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(11, 1, 2)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(3, 5, 0)	(2, 2, 5)	155	32	No	-
P18	OUT	(3, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(7, 6, 3)	(4, 2, 4)	190	38	No	-
P2	OUT	(0, 5, 0)	(2, 1, 2)	80	8	Yes	-
P20	OUT	(4, 7, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(0, 6, 0)	(3, 1, 2)	90	12	No	-
P6	OUT	(9, 2, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(8, 5, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(7, 8, 0)	(3, 4, 1)	130	20	No	-

Inside truck Outside truck (OUT) Fragile [F]

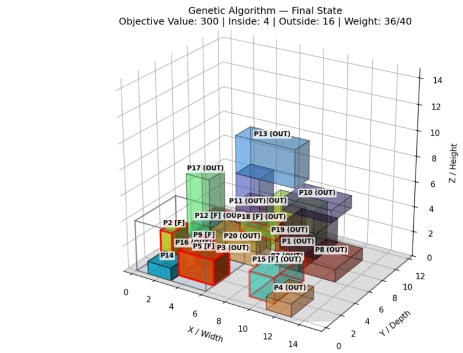
3	**Genetic Algorithm: 295 → 300 | Iterations: 25 | D**	**Duration: 2.2110s | Stop: max_iterations_reached**

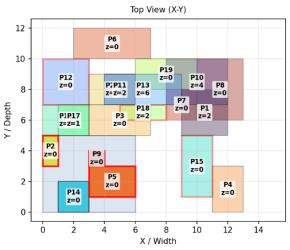




ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P16	IN	(3, 0, 0)	(3, 2, 1)	85	11	No	T1
P2	IN	(3, 3, 0)	(2, 2, 1)	80	8	Yes	T1
P8	IN	(0, 1, 0)	(3, 4, 1)	130	20	No	T1
P1	OUT	(8, 0, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(7, 5, 2)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 2, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 6, 3)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(4, 2, 6)	(5, 2, 3)	210	42	No	-
P14	OUT	(0, 7, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(9, 2, 6)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(4, 3, 9)	(2, 2, 5)	155	32	No	-
P18	OUT	(0, 5, 1)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(7, 0, 2)	(4, 2, 4)	190	38	No	-
P20	OUT	(9, 6, 3)	(3, 2, 2)	115	21	No	-
P3	OUT	(8, 6, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(2, 7, 0)	(2, 3, 1)	90	12	No	-
P5	OUT	(4, 7, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(10, 2, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(7, 4, 0)	(2, 2, 2)	70	10	No	-
P9	OUT	(7, 1, 0)	(1, 2, 1)	45	4	Yes	-

Truck boundary Truck floor Outside floor





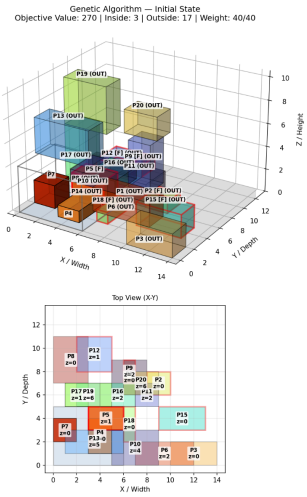
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(1, 0, 0)	(2, 2, 1)	65	6	No	T1
P2	IN	(0, 3, 0)	(1, 2, 2)	80	8	Yes	T1
P5	IN	(3, 1, 0)	(3, 2, 2)	110	18	Yes	T1
P9	IN	(3, 3, 0)	(1, 1, 2)	45	4	Yes	T1
P1	OUT	(9, 5, 2)	(3, 3, 2)	120	24	No	-
P10	OUT	(8, 7, 4)	(4, 3, 1)	160	28	No	-
P11	OUT	(4, 7, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(0, 7, 0)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(4, 7, 6)	(5, 2, 3)	210	42	No	-
P15	OUT	(9, 1, 0)	(2, 4, 2)	145	25	Yes	-
P16	OUT	(0, 5, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(1, 5, 1)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 6, 2)	(3, 1, 1)	55	5	Yes	-
P19	OUT	(6, 8, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(3, 7, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(3, 5, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(11, 0, 0)	(2, 3, 1)	90	12	No	-
P6	OUT	(2, 10, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(8, 6, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(10, 6, 0)	(3, 4, 1)	130	20	No	-

Inside truck Outside truck (OUT) Fragile [F]

nPopulation = 20; nGeneration = 50

1

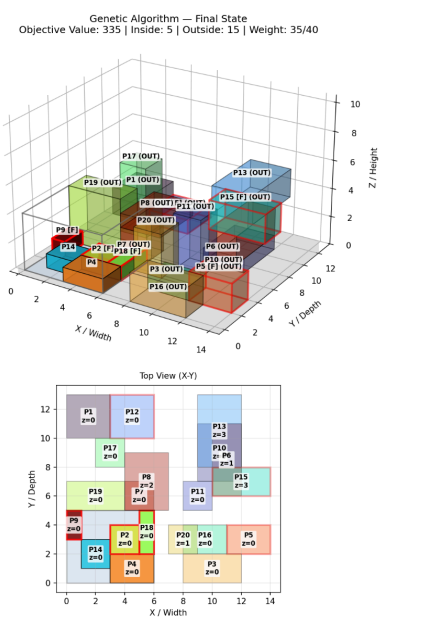
Genetic Algorithm: 270 → 335 | Iterations: 50 | Du



ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P6	IN	(3, 1, 0)	(2, 3, 1)	90	12	No	T1
P5	IN	(3, 3, 1)	(3, 2, 2)	110	18	Yes	T1
P7	IN	(0, 2, 0)	(2, 2, 2)	70	10	No	T1
P1	OUT	(5, 6, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(5, 0, 4)	(4, 3, 1)	160	28	No	-
P11	OUT	(7, 5, 2)	(2, 2, 4)	140	26	No	-
P12	OUT	(2, 8, 1)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(1, 1, 5)	(5, 2, 3)	210	42	No	-
P14	OUT	(2, 5, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(9, 3, 0)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(4, 5, 2)	(3, 2, 1)	85	11	No	-
P17	OUT	(1, 5, 1)	(2, 2, 5)	155	32	No	-
P18	OUT	(6, 2, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(1, 5, 6)	(4, 2, 4)	190	38	No	-
P2	OUT	(8, 6, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(6, 6, 6)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 0, 0)	(4, 2, 2)	150	30	No	-
P6	OUT	(7, 0, 2)	(5, 2, 2)	180	36	No	-
P8	OUT	(0, 7, 0)	(3, 4, 1)	130	20	No	-
P9	OUT	(6, 7, 2)	(1, 2, 1)	45	4	Yes	-

Truck boundary Truck floor Outside floor

ration: 2.8626s | Stop: max_iterations_reached



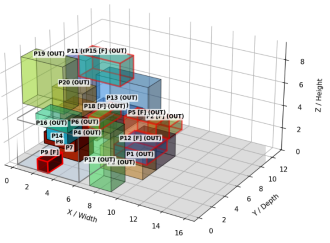
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(1, 1, 0)	(2, 2, 1)	65	6	No	T1
P18	IN	(5, 2, 0)	(1, 3, 1)	55	5	Yes	T1
P2	IN	(3, 2, 0)	(2, 2, 1)	80	8	Yes	T1
P4	IN	(3, 0, 0)	(3, 2, 1)	90	12	No	T1
P9	IN	(0, 3, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(0, 10, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(9, 7, 0)	(3, 4, 1)	160	28	No	-
P11	OUT	(8, 5, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(3, 10, 0)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(9, 8, 3)	(3, 5, 2)	210	42	No	-
P15	OUT	(10, 6, 3)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(8, 2, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(2, 8, 0)	(2, 2, 5)	155	32	No	-
P19	OUT	(0, 5, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(7, 2, 1)	(2, 2, 3)	115	21	No	-
P3	OUT	(8, 0, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(11, 2, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(10, 6, 1)	(2, 5, 2)	180	36	No	-
P7	OUT	(4, 5, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(4, 5, 2)	(3, 4, 1)	130	20	No	-

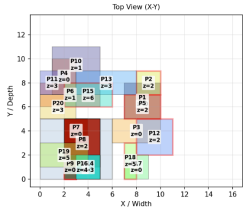
Inside truck Outside truck (OUT) Fragile [F]

2

Genetic Algorithm: 310 → 310 | Iterations: 50 |

Genetic Algorithm — Initial State
Objective Value: 310 | Inside: 4 | Outside: 16 | Weight: 40/40



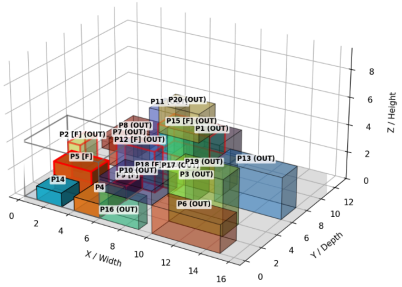


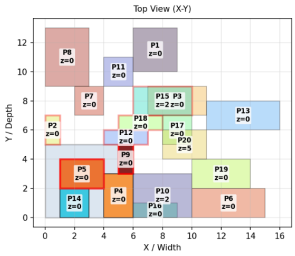
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(3, 0, 3)	(2, 2, 1)	65	6	No	T1
P7	IN	(2, 3, 0)	(2, 2, 2)	70	10	No	T1
P8	IN	(2, 1, 2)	(3, 4, 1)	130	20	No	T1
P9	IN	(2, 0, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(7, 5, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(1, 8, 1)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 7, 3)	(2, 2, 4)	140	26	No	-
P12	OUT	(8, 2, 2)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(3, 7, 3)	(5, 2, 3)	210	42	No	-
P15	OUT	(2, 6, 6)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(2, 0, 4)	(3, 2, 1)	85	11	No	-
P17	OUT	(7, 0, 0)	(2, 2, 5)	155	32	No	-
P18	OUT	(7, 0, 5)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(0, 1, 5)	(4, 2, 4)	190	38	No	-
P2	OUT	(8, 7, 2)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(0, 5, 3)	(3, 2, 2)	115	21	No	-
P3	OUT	(6, 3, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(1, 7, 0)	(2, 3, 1)	90	12	No	-
P5	OUT	(7, 5, 2)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(0, 6, 1)	(5, 2, 2)	180	36	No	-

Truck boundary Truck floor Outside floor

uration: 2.9053s | Stop: max_iterations_reached

Genetic Algorithm — Final State
Objective Value: 310 | Inside: 4 | Outside: 16 | Weight: 40/40





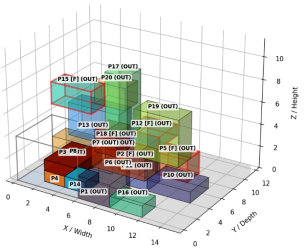
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(1, 0, 0)	(2, 2, 1)	65	6	No	T1
P4	IN	(4, 0, 0)	(2, 3, 1)	90	12	No	T1
P5	IN	(1, 2, 0)	(3, 2, 2)	110	18	Yes	T1
P9	IN	(5, 3, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(6, 10, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(6, 0, 2)	(4, 3, 1)	160	28	No	-
P11	OUT	(4, 9, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(4, 5, 0)	(3, 1, 3)	100	14	Yes	-
P13	OUT	(11, 6, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(6, 7, 2)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(6, 0, 0)	(3, 1, 2)	85	11	No	-
P17	OUT	(8, 5, 0)	(2, 2, 5)	155	32	No	-
P18	OUT	(5, 6, 0)	(3, 1, 1)	55	5	Yes	-
P19	OUT	(10, 2, 0)	(4, 2, 4)	190	38	No	-
P2	OUT	(0, 5, 0)	(1, 2, 2)	80	8	Yes	-
P20	OUT	(8, 4, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(7, 7, 0)	(4, 2, 2)	150	30	No	-
P6	OUT	(10, 0, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(2, 7, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(0, 9, 0)	(3, 4, 1)	130	20	No	-

Inside truck Outside truck (OUT) Fragile [F]

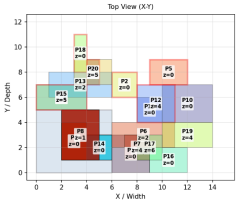
3

Genetic Algorithm: 285 → 315 | Iterations: 50 | Dur

Genetic Algorithm — Initial State
Objective Value: 285 | Inside: 3 | Outside: 17 | Weight: 38/40



Top View (X-Y)

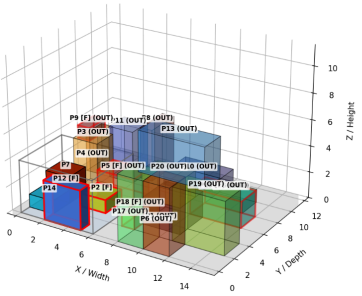


ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(4, 1, 0)	(2, 2, 1)	65	6	No	T1
P4	IN	(2, 1, 0)	(2, 3, 1)	90	12	No	T1
P8	IN	(2, 1, 1)	(3, 4, 1)	130	20	No	T1
P1	OUT	(6, 0, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(10, 4, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(8, 4, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(8, 4, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(1, 6, 2)	(5, 2, 3)	210	42	No	-
P15	OUT	(0, 5, 5)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(9, 0, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(8, 1, 6)	(2, 2, 5)	155	32	No	-
P18	OUT	(3, 8, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 2, 4)	(4, 2, 4)	190	38	No	-
P2	OUT	(6, 6, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(3, 7, 5)	(3, 2, 2)	115	21	No	-
P3	OUT	(0, 5, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(9, 7, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(6, 2, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(7, 1, 4)	(2, 2, 2)	70	10	No	-
P9	OUT	(4, 7, 0)	(1, 2, 1)	45	4	Yes	-

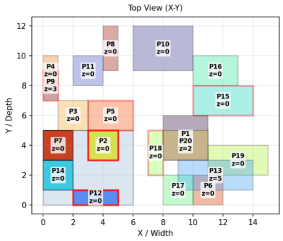
Truck boundary Truck floor Outside floor

uration: 2.8819s | Stop: max_iterations_reached

Genetic Algorithm — Final State
Objective Value: 315 | Inside: 4 | Outside: 16 | Weight: 38/40



Top View (X-Y)



ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P12	IN	(2, 0, 0)	(3, 1, 3)	100	14	Yes	T1
P14	IN	(0, 1, 0)	(2, 2, 1)	65	6	No	T1
P2	IN	(3, 3, 0)	(2, 2, 1)	80	8	Yes	T1
P7	IN	(0, 3, 0)	(2, 2, 2)	70	10	No	T1
P1	OUT	(8, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(6, 9, 0)	(4, 3, 1)	160	28	No	-
P11	OUT	(2, 8, 0)	(2, 2, 4)	140	26	No	-
P13	OUT	(9, 1, 5)	(5, 2, 3)	210	42	No	-
P15	OUT	(10, 6, 0)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(10, 8, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(8, 0, 0)	(2, 2, 5)	155	32	No	-
P18	OUT	(7, 2, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(11, 2, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(8, 3, 2)	(3, 2, 2)	115	21	No	-
P3	OUT	(1, 5, 0)	(2, 2, 4)	150	30	No	-
P4	OUT	(0, 8, 0)	(1, 2, 3)	90	12	No	-
P5	OUT	(3, 5, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(10, 0, 0)	(2, 2, 5)	180	36	No	-
P8	OUT	(4, 9, 0)	(1, 3, 4)	130	20	No	-
P9	OUT	(0, 7, 3)	(1, 2, 1)	45	4	Yes	-

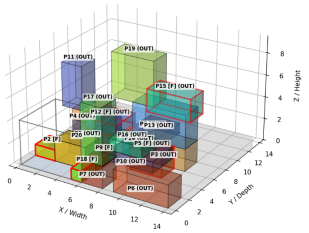
Inside truck Outside truck (OUT) Fragile [F]

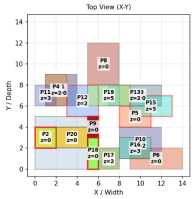
nPopulation = 20; nGeneration = 100

1

Genetic Algorithm: 295 → 355 | Iterations: 100 | Du

Genetic Algorithm — Initial State
Objective Value: 295 | Inside: 4 | Outside: 16 | Weight: 38/40



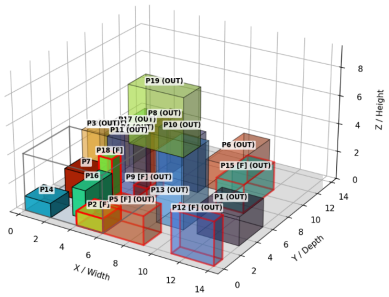


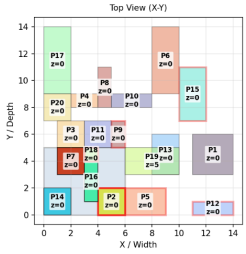
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P18	IN	(5, 0, 0)	(1, 3, 1)	55	5	Yes	T1
P2	IN	(0, 2, 0)	(2, 2, 1)	80	8	Yes	T1
P20	IN	(2, 2, 0)	(3, 2, 2)	115	21	No	T1
P9	IN	(5, 3, 0)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(1, 6, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(8, 1, 2)	(4, 3, 1)	160	28	No	-
P11	OUT	(0, 6, 3)	(2, 2, 4)	140	26	No	-
P12	OUT	(3, 5, 2)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(7, 6, 2)	(5, 2, 3)	210	42	No	-
P14	OUT	(6, 6, 0)	(2, 2, 1)	65	6	No	-
P15	OUT	(9, 5, 5)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(8, 1, 3)	(3, 2, 1)	85	11	No	-
P17	OUT	(6, 0, 2)	(2, 2, 5)	155	32	No	-
P19	OUT	(5, 6, 5)	(4, 2, 4)	190	38	No	-
P3	OUT	(8, 6, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(1, 6, 2)	(2, 3, 1)	90	12	No	-
P5	OUT	(0, 4, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(9, 0, 0)	(5, 2, 2)	180	36	No	-
P7	OUT	(6, 0, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(5, 8, 0)	(3, 4, 1)	130	20	No	-

Truck boundary Truck floor Outside floor

ration: 7.0662s | Stop: max_iterations_reached

Genetic Algorithm — Final State
Objective Value: 355 | Inside: 5 | Outside: 15 | Weight: 40/40





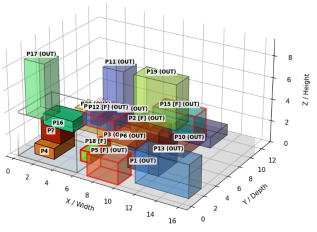
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(0, 0, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(3, 1, 0)	(1, 3, 2)	85	11	No	T1
P18	IN	(3, 4, 0)	(1, 1, 3)	55	5	Yes	T1
P2	IN	(4, 0, 0)	(2, 2, 1)	80	8	Yes	T1
P7	IN	(1, 3, 0)	(2, 2, 2)	70	10	No	T1
P1	OUT	(11, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(5, 8, 0)	(3, 1, 4)	160	28	No	-
P11	OUT	(3, 5, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(11, 0, 0)	(3, 1, 3)	100	14	Yes	-
P13	OUT	(8, 3, 0)	(2, 3, 5)	210	42	No	-
P15	OUT	(10, 7, 0)	(2, 4, 2)	145	25	Yes	-
P17	OUT	(0, 9, 0)	(2, 5, 2)	155	32	No	-
P19	OUT	(6, 3, 5)	(4, 2, 4)	190	38	No	-
P20	OUT	(0, 7, 0)	(2, 2, 3)	115	21	No	-
P3	OUT	(1, 5, 0)	(2, 2, 4)	150	30	No	-
P4	OUT	(2, 8, 0)	(2, 1, 3)	90	12	No	-
P5	OUT	(6, 0, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(8, 9, 0)	(2, 5, 2)	180	36	No	-
P8	OUT	(4, 8, 0)	(1, 3, 4)	130	20	No	-
P9	OUT	(5, 5, 0)	(1, 2, 1)	45	4	Yes	-

Inside truck Outside truck (OUT) Fragile [F]

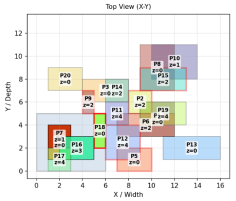
2

Genetic Algorithm: 300 → 355 | Iterations: 100 | Du

Genetic Algorithm — Initial State
Objective Value: 300 | Inside: 4 | Outside: 16 | Weight: 38/40



Top View (X-Y)

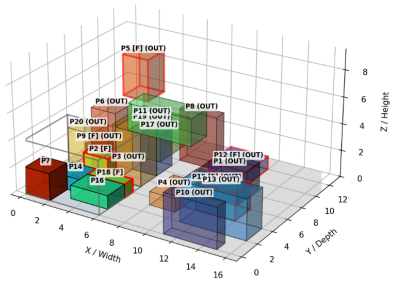


ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P16	IN	(2, 1, 3)	(3, 2, 1)	85	11	No	T1
P18	IN	(5, 2, 0)	(1, 3, 1)	55	5	Yes	T1
P4	IN	(1, 1, 0)	(2, 3, 1)	90	12	No	T1
P7	IN	(1, 2, 1)	(2, 2, 2)	70	10	No	T1
P1	OUT	(9, 3, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(10, 8, 1)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 4, 4)	(2, 2, 4)	140	26	No	-
P12	OUT	(6, 1, 4)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(11, 1, 0)	(5, 2, 3)	210	42	No	-
P14	OUT	(6, 6, 2)	(2, 2, 1)	65	6	No	-
P15	OUT	(9, 7, 2)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(1, 0, 4)	(2, 2, 5)	155	32	No	-
P19	OUT	(9, 4, 4)	(4, 2, 4)	190	38	No	-
P2	OUT	(8, 5, 2)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(1, 7, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 6, 0)	(4, 2, 2)	150	30	No	-
P5	OUT	(7, 0, 0)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(7, 3, 2)	(5, 2, 2)	180	36	No	-
P8	OUT	(9, 7, 0)	(3, 4, 1)	130	20	No	-
P9	OUT	(4, 5, 2)	(1, 2, 1)	45	4	Yes	-

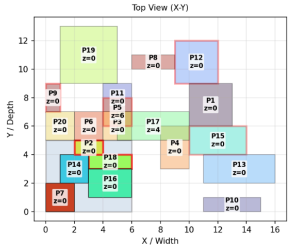
Truck boundary Truck floor Outside floor

uration: 9.8605s | Stop: max_iterations_reached

Genetic Algorithm — Final State
Objective Value: 355 | Inside: 5 | Outside: 15 | Weight: 40/40

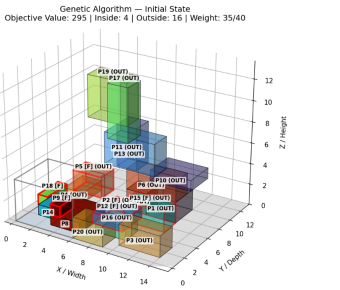
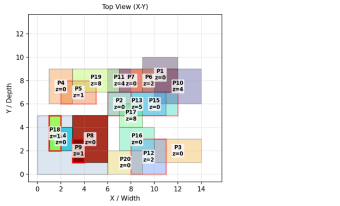
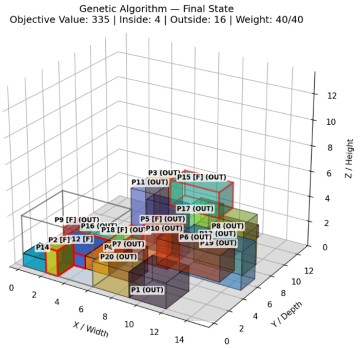
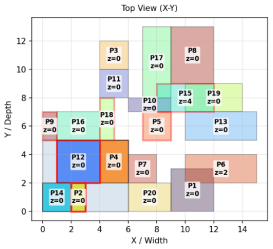


Top View (X-Y)



ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(1, 2, 0)	(2, 2, 1)	65	6	No	T1
P16	IN	(3, 1, 0)	(3, 2, 1)	85	11	No	T1
P18	IN	(3, 3, 0)	(3, 1, 1)	55	5	Yes	T1
P2	IN	(2, 4, 0)	(2, 1, 2)	80	8	Yes	T1
P7	IN	(0, 0, 0)	(2, 2, 2)	70	10	No	T1
P1	OUT	(10, 6, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(11, 0, 0)	(4, 1, 3)	160	28	No	-
P11	OUT	(4, 7, 0)	(2, 2, 4)	140	26	No	-
P12	OUT	(9, 9, 0)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(11, 2, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(10, 4, 0)	(4, 2, 2)	145	25	Yes	-
P17	OUT	(5, 5, 4)	(5, 2, 2)	155	32	No	-
P19	OUT	(1, 9, 0)	(4, 4, 2)	190	38	No	-
P20	OUT	(0, 5, 0)	(2, 2, 3)	115	21	No	-
P3	OUT	(4, 5, 0)	(2, 2, 4)	150	30	No	-
P4	OUT	(8, 3, 0)	(2, 3, 1)	90	12	No	-
P5	OUT	(4, 6, 6)	(2, 2, 3)	110	18	Yes	-
P6	OUT	(2, 5, 0)	(2, 2, 5)	180	36	No	-
P8	OUT	(6, 10, 0)	(3, 1, 4)	130	20	No	-
P9	OUT	(0, 7, 0)	(1, 2, 1)	45	4	Yes	-

Inside truck Outside truck (OUT) Fragile [F]

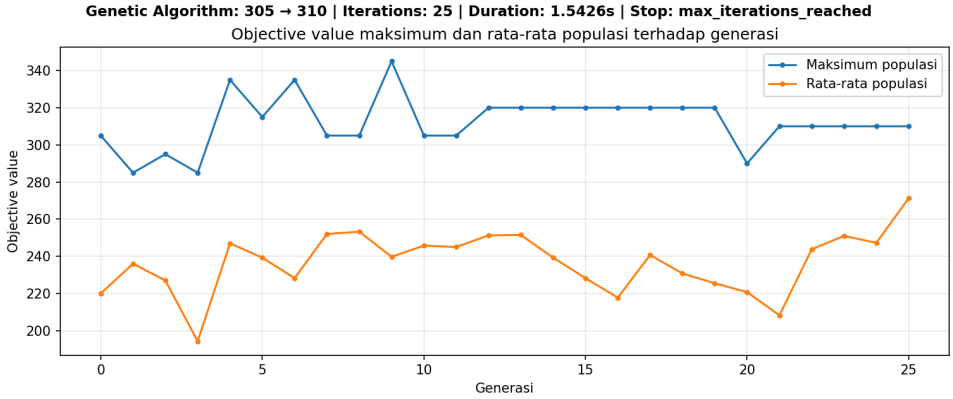
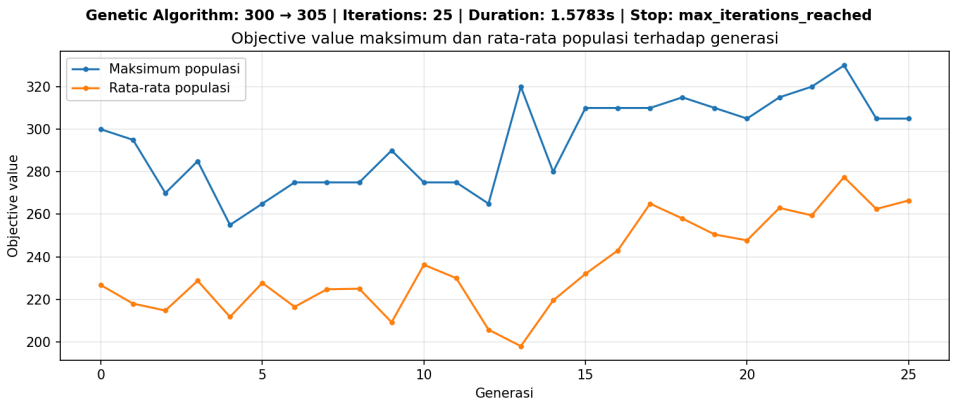
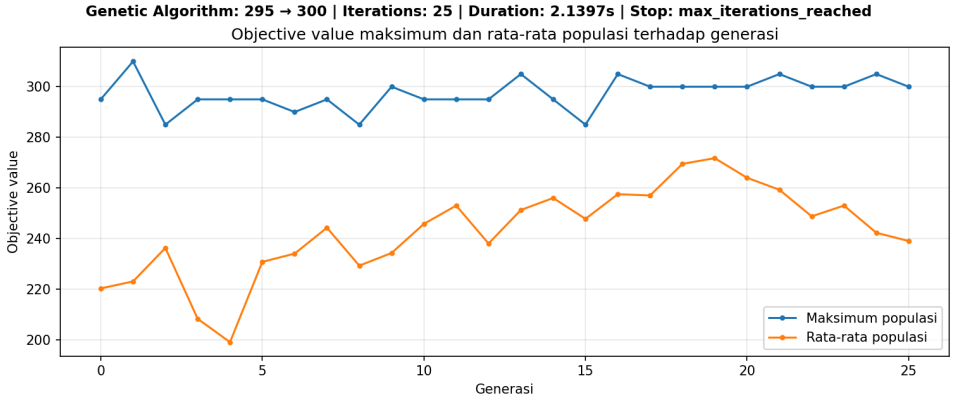
3	**Genetic Algorithm: 295 → 335 | Iterations: 100 |**   Truck boundary Truck floor Outside floor	**Duration: 7.4144s | Stop: max_iterations_reached**   Inside truck Outside truck (OUT) Fragile [F]

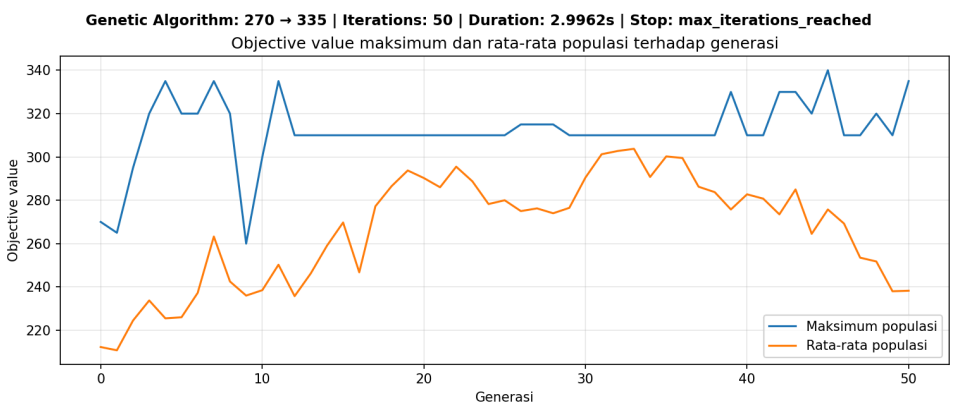
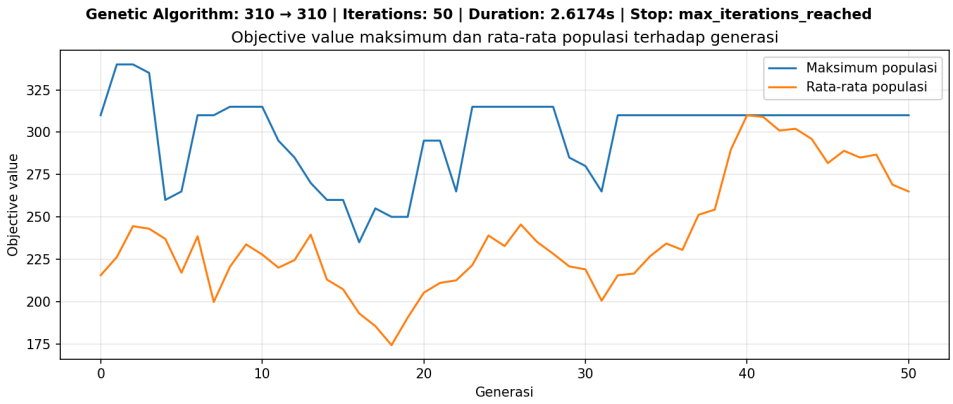
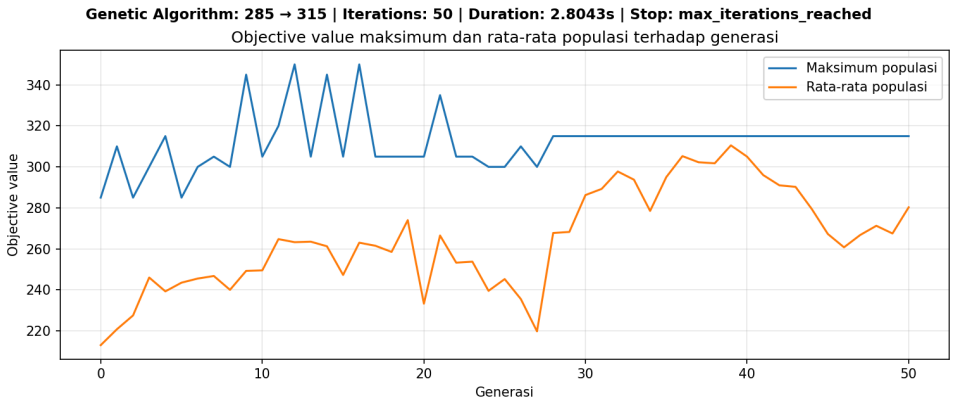
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P14	IN	(1, 2, 0)	(2, 2, 1)	65	6	No	T1
P18	IN	(1, 2, 1)	(1, 3, 1)	55	5	Yes	T1
P6	IN	(3, 1, 0)	(3, 4, 1)	130	20	No	T1
P9	IN	(3, 1, 1)	(1, 2, 1)	45	4	Yes	T1
P1	OUT	(9, 7, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(10, 6, 4)	(4, 3, 1)	160	28	No	-
P11	OUT	(6, 7, 4)	(2, 2, 4)	140	26	No	-
P12	OUT	(8, 0, 2)	(3, 3, 1)	100	14	Yes	-
P13	OUT	(6, 5, 5)	(5, 2, 3)	210	42	No	-
P15	OUT	(8, 5, 0)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(7, 2, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(7, 4, 8)	(2, 2, 5)	155	32	No	-
P19	OUT	(3, 7, 8)	(4, 2, 4)	190	38	No	-
P2	OUT	(6, 5, 0)	(2, 2, 1)	80	8	Yes	-
P20	OUT	(6, 0, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(10, 1, 0)	(4, 2, 2)	150	30	No	-
P4	OUT	(1, 6, 0)	(2, 3, 1)	90	12	No	-
P5	OUT	(2, 6, 1)	(3, 2, 2)	110	18	Yes	-
P6	OUT	(7, 7, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(7, 7, 0)	(2, 2, 2)	70	10	No	-

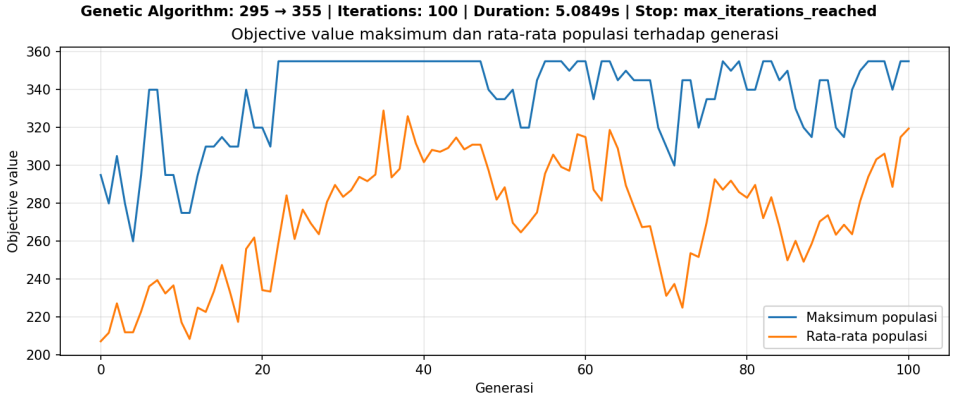
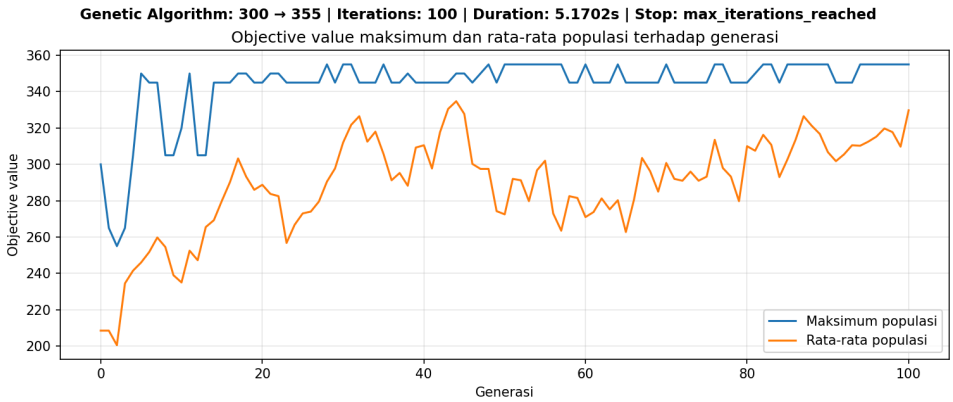
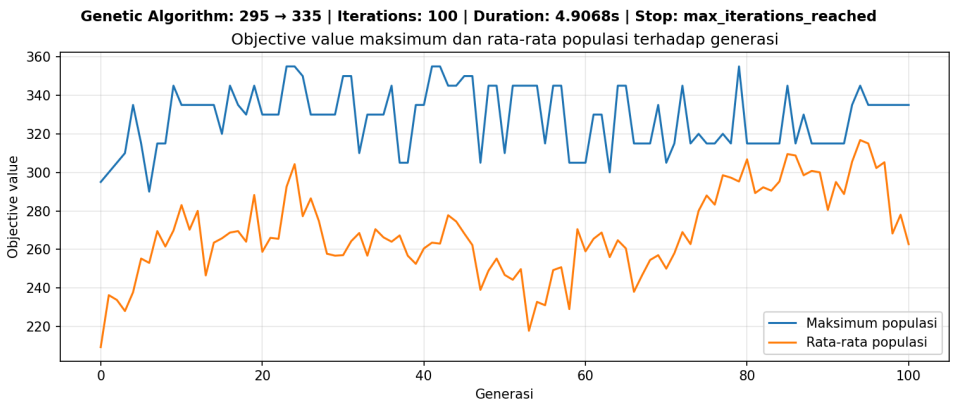
ID	Status	Pos (x,y,z)	Dim (w,d,h)	Value	Weight	Fragile	Truck
P12	IN	(1, 2, 0)	(3, 3, 1)	100	14	Yes	T1
P14	IN	(0, 0, 0)	(2, 2, 1)	65	6	No	T1
P2	IN	(2, 0, 0)	(1, 2, 2)	80	8	Yes	T1
P4	IN	(4, 2, 0)	(2, 3, 1)	90	12	No	T1
P1	OUT	(9, 0, 0)	(3, 3, 2)	120	24	No	-
P10	OUT	(6, 7, 0)	(3, 1, 4)	160	28	No	-
P11	OUT	(4, 8, 0)	(2, 2, 4)	140	26	No	-
P13	OUT	(10, 5, 0)	(5, 2, 3)	210	42	No	-
P15	OUT	(8, 7, 4)	(4, 2, 2)	145	25	Yes	-
P16	OUT	(1, 5, 0)	(3, 2, 1)	85	11	No	-
P17	OUT	(7, 8, 0)	(2, 5, 2)	155	32	No	-
P18	OUT	(4, 5, 0)	(1, 3, 1)	55	5	Yes	-
P19	OUT	(10, 7, 0)	(4, 2, 4)	190	38	No	-
P20	OUT	(6, 0, 0)	(3, 2, 2)	115	21	No	-
P3	OUT	(4, 10, 0)	(2, 2, 4)	150	30	No	-
P5	OUT	(7, 5, 0)	(2, 2, 3)	110	18	Yes	-
P6	OUT	(10, 2, 2)	(5, 2, 2)	180	36	No	-
P7	OUT	(6, 2, 0)	(2, 2, 2)	70	10	No	-
P8	OUT	(9, 9, 0)	(3, 4, 1)	130	20	No	-
P9	OUT	(0, 5, 0)	(1, 2, 1)	45	4	Yes	-

Plot nilai objective function:

Run	Plot
nPopulation=20; nGeneration=25	

1	
2	
3	
nPopulation=20; nGeneration=50	

1	
2	
3	
nPopulation=20; nGeneration=100	

1	
2	
3	

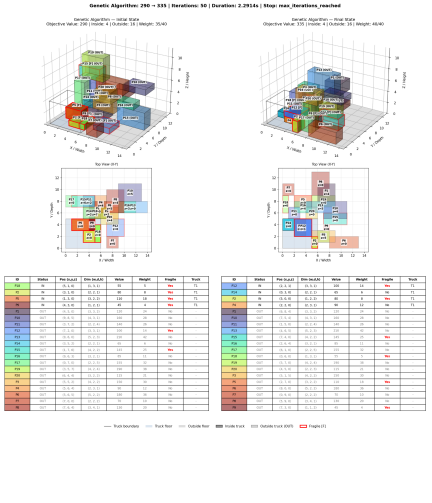
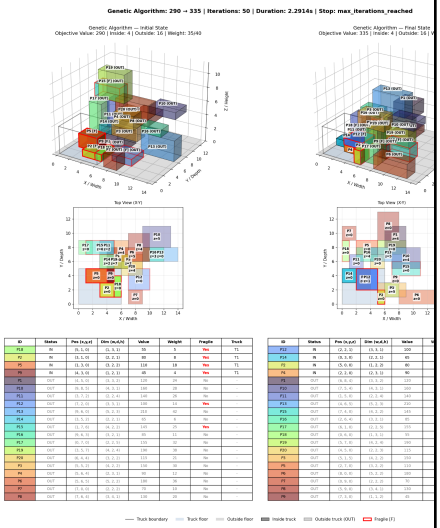
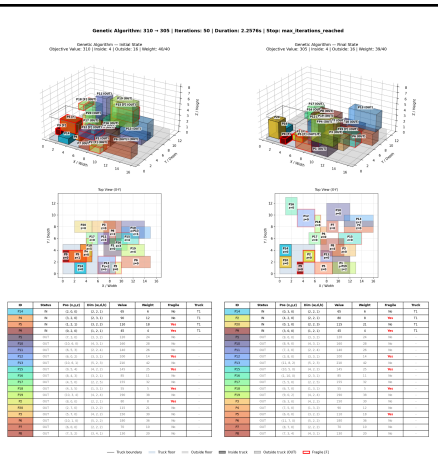
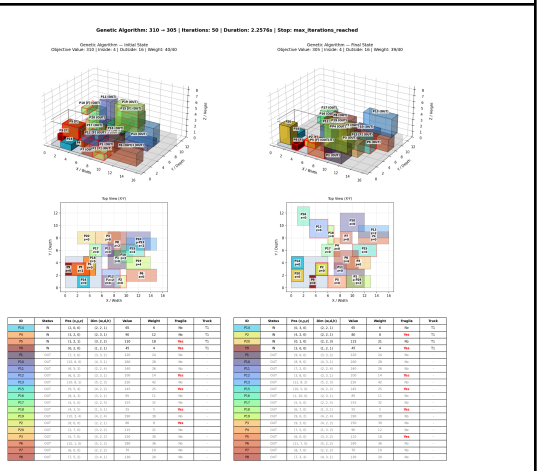
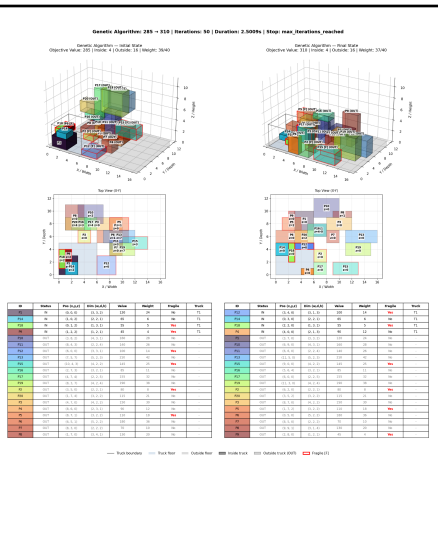
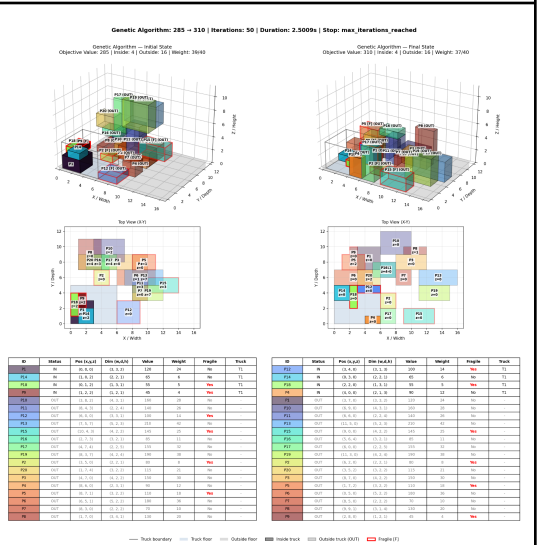
2. Jumlah iterasi (*nGeneration*) sebagai kontrol

Run	nPopulation	nGeneration	Nilai objective function	Durasi (detik)

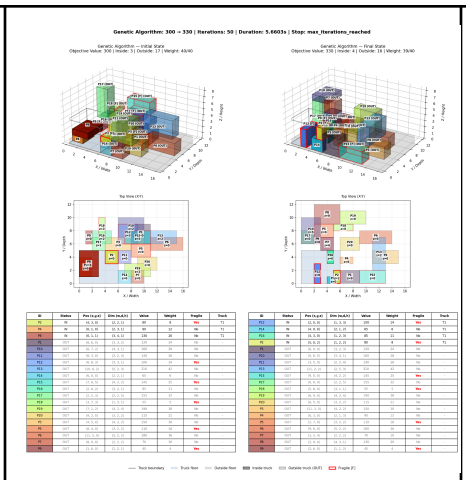
			pada generasi		
			awal	**akhir**	
1	10	50	290	335	1.18562540
2	10	50	310	305	1.34759829
3	10	50	285	310	1.4211380
1	20	50	300	330	2.90764409
2	20	50	315	320	2.89604970
3	20	50	310	275	2.678071400
1	40	50	295	310	0.582158 (crossover limit)
2	40	50	310	315	7.6435889
3	40	50	310	350	7.270832799

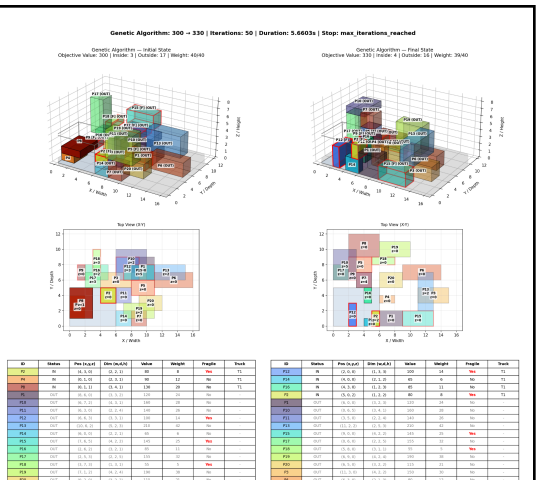
Setelah dijalankan, berikut kondisi state:

nPopulation = 10; nGeneration = 50		
Run	State Awal	State Akhir

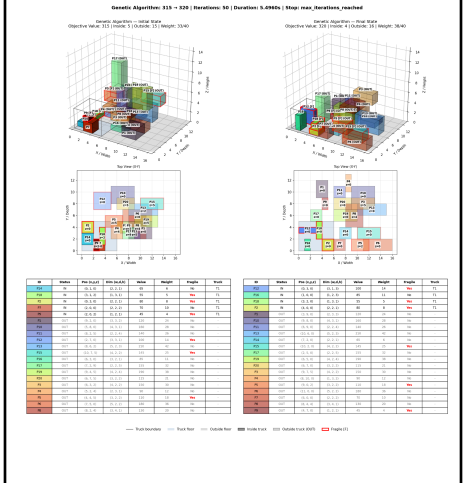
1		
2		
3		
nPopulation = 20; nGeneration = 50		

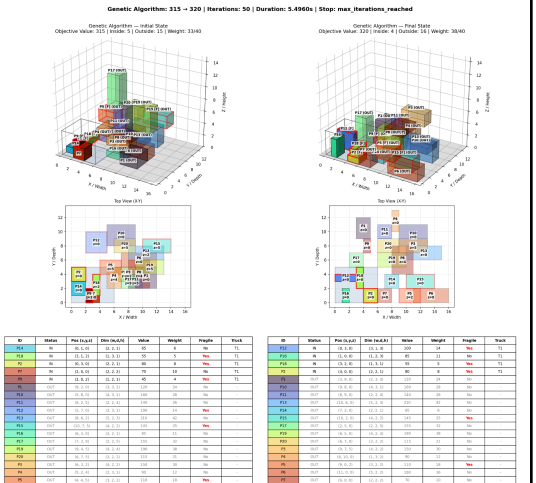
1


Genetic Algorithm: 300 → 330 | Iterations: 50 | Duration: 5.6603s | Stop: max_iterations_reached
Genetic Algorithm — Initial State
Objective Value: 300 | Inside: 3 | Outside: 17 | Weight: 40/40
Genetic Algorithm — Final State
Objective Value: 330 | Inside: 4 | Outside: 16 | Weight: 39/40
Top View (XY)
X / Width
Y / Depth
Truck boundary
Truck floor
Outside floor
Inside truck
Outside truck (OUT)
Fragile (F)

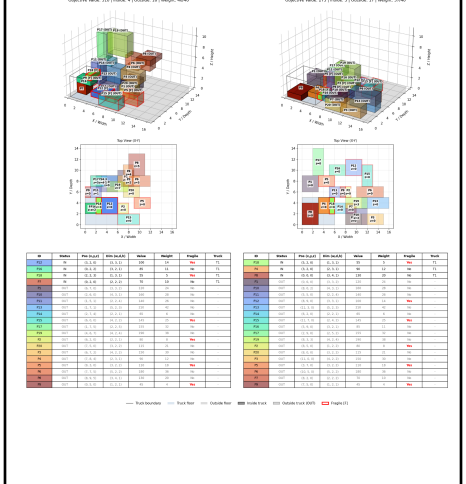

Genetic Algorithm: 300 → 330 | Iterations: 50 | Duration: 5.6603s | Stop: max_iterations_reached
Genetic Algorithm — Initial State
Objective Value: 300 | Inside: 3 | Outside: 17 | Weight: 40/40
Genetic Algorithm — Final State
Objective Value: 330 | Inside: 4 | Outside: 16 | Weight: 39/40
Top View (XY)
X / Width
Y / Depth
Truck boundary
Truck floor
Outside floor
Inside truck
Outside truck (OUT)
Fragile (F)

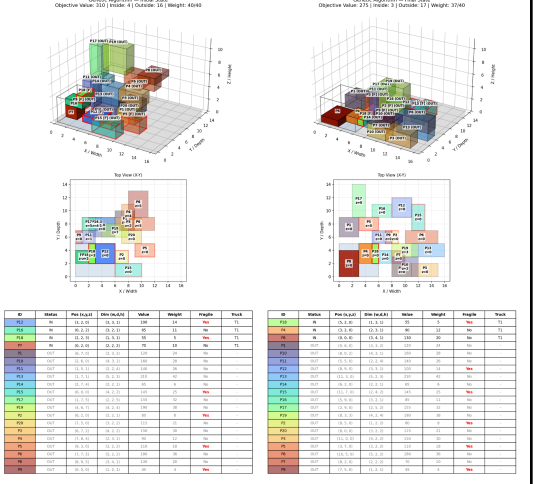
2

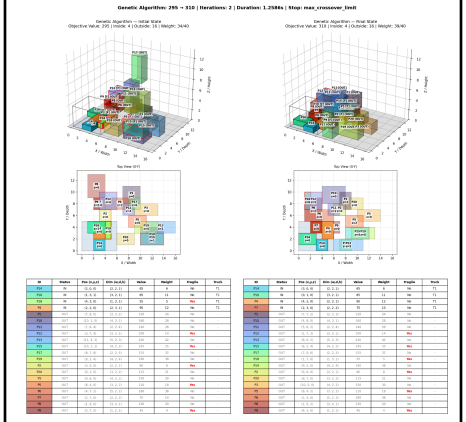
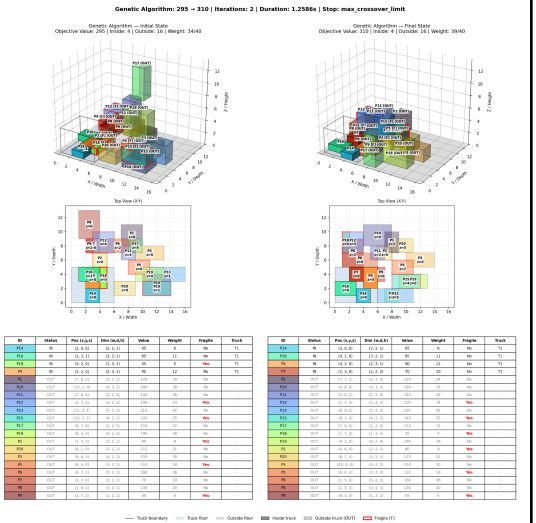
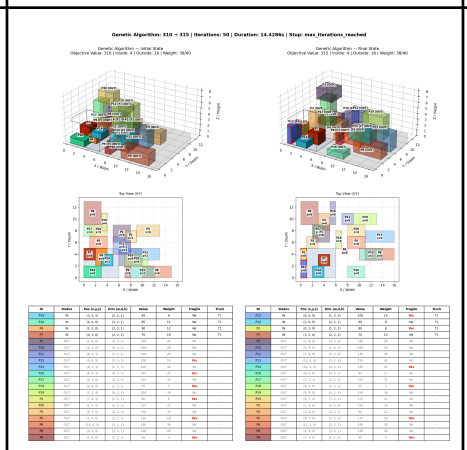
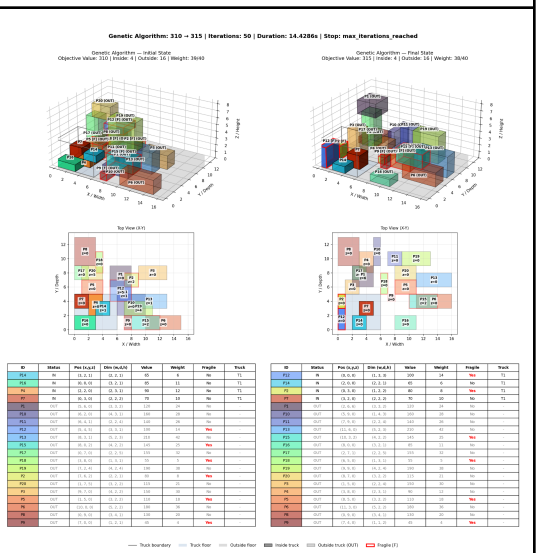

Genetic Algorithm: 315 → 320 | Iterations: 50 | Duration: 5.4960s | Stop: max_iterations_reached
Genetic Algorithm — Initial State
Objective Value: 315 | Inside: 5 | Outside: 15 | Weight: 33/40
Genetic Algorithm — Final State
Objective Value: 320 | Inside: 4 | Outside: 16 | Weight: 38/40
Top View (XY)
X / Width
Y / Depth
Truck boundary
Truck floor
Outside floor
Inside truck
Outside truck (OUT)
Fragile (F)

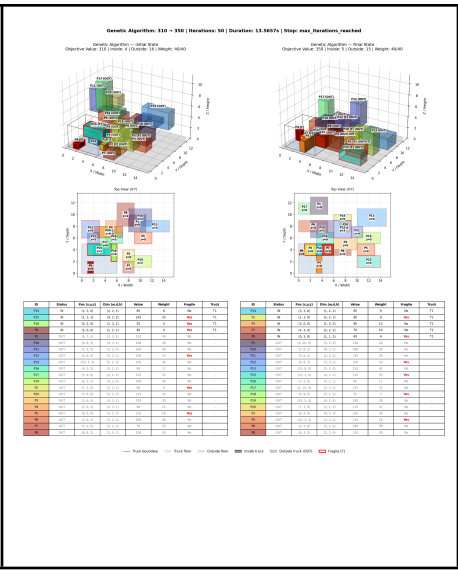
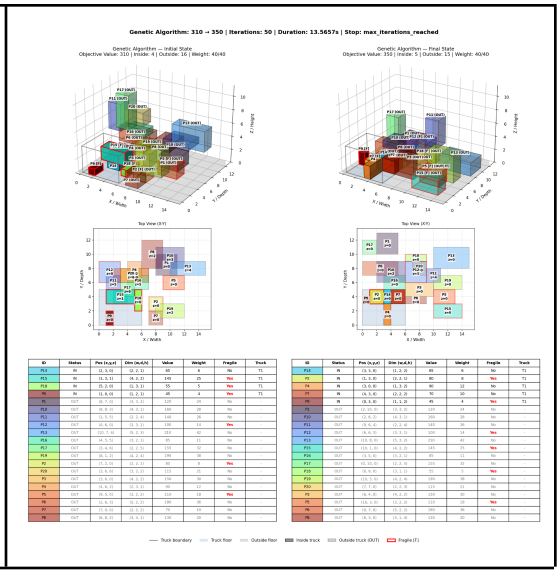

Genetic Algorithm: 315 → 320 | Iterations: 50 | Duration: 5.4960s | Stop: max_iterations_reached
Genetic Algorithm — Initial State
Objective Value: 315 | Inside: 5 | Outside: 15 | Weight: 33/40
Genetic Algorithm — Final State
Objective Value: 320 | Inside: 4 | Outside: 16 | Weight: 38/40
Top View (XY)
X / Width
Y / Depth
Truck boundary
Truck floor
Outside floor
Inside truck
Outside truck (OUT)
Fragile (F)

3

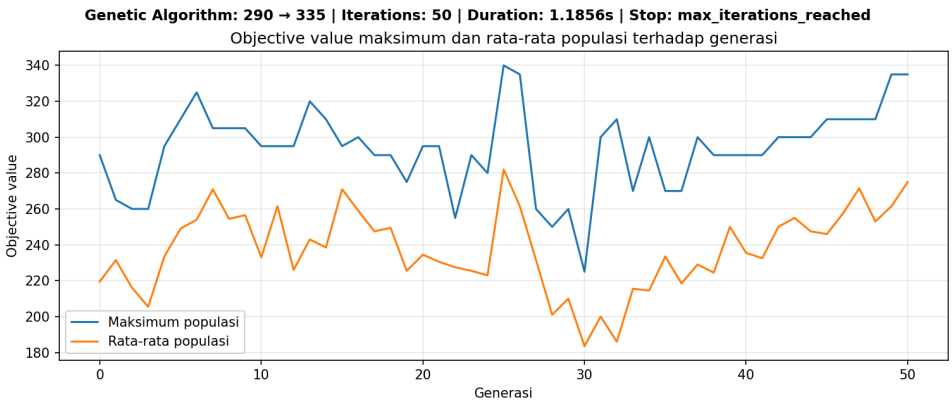

Genetic Algorithm: 310 → 275 | Iterations: 50 | Duration: 5.4117s | Stop: max_iterations_reached
Genetic Algorithm — Initial State
Objective Value: 310 | Inside: 4 | Outside: 16 | Weight: 40/40
Genetic Algorithm — Final State
Objective Value: 275 | Inside: 3 | Outside: 17 | Weight: 37/40
Top View (XY)
X / Width
Y / Depth
Truck boundary
Truck floor
Outside floor
Inside truck
Outside truck (OUT)
Fragile (F)

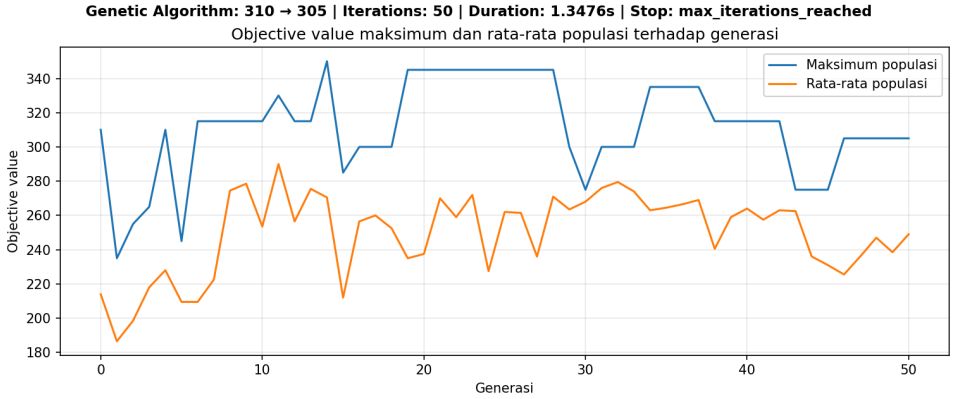
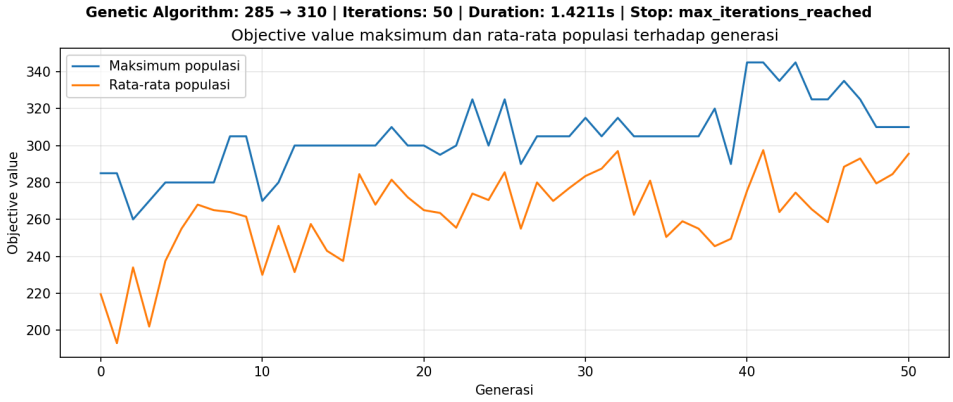
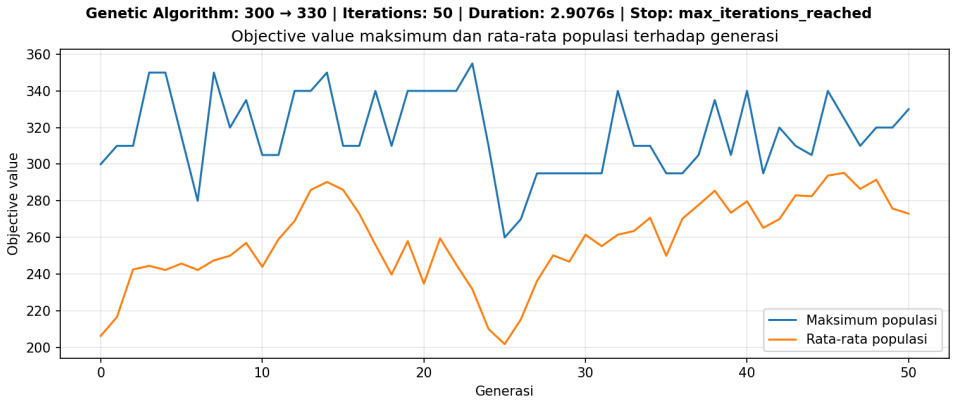

Genetic Algorithm: 310 → 275 | Iterations: 50 | Duration: 5.4117s | Stop: max_iterations_reached
Genetic Algorithm — Initial State
Objective Value: 310 | Inside: 4 | Outside: 16 | Weight: 40/40
Genetic Algorithm — Final State
Objective Value: 275 | Inside: 3 | Outside: 17 | Weight: 37/40
Top View (XY)
X / Width
Y / Depth
Truck boundary
Truck floor
Outside floor
Inside truck
Outside truck (OUT)
Fragile (F)

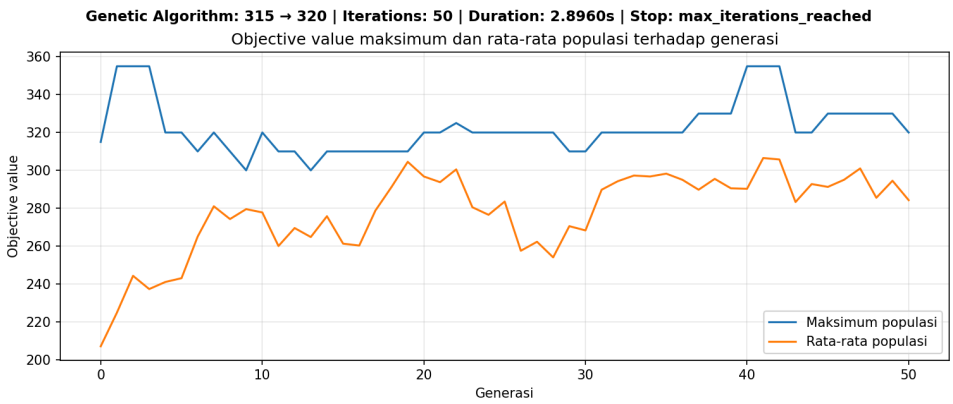
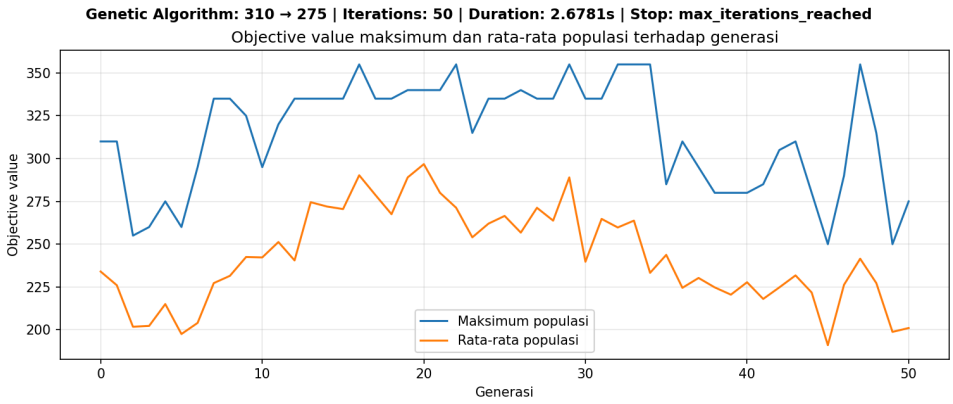
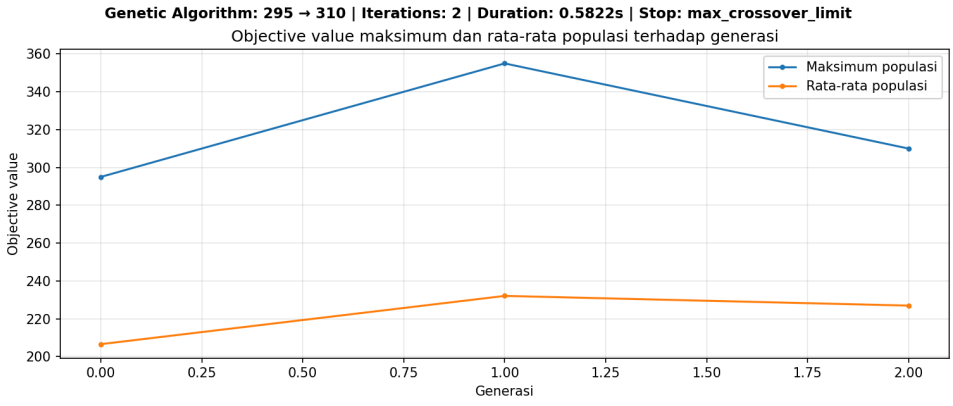
nPopulation = 40; nGeneration = 50		
1		
2		

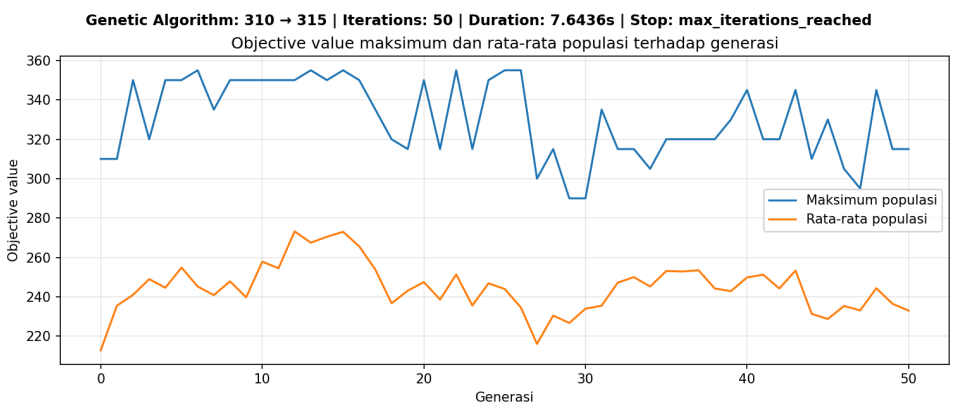
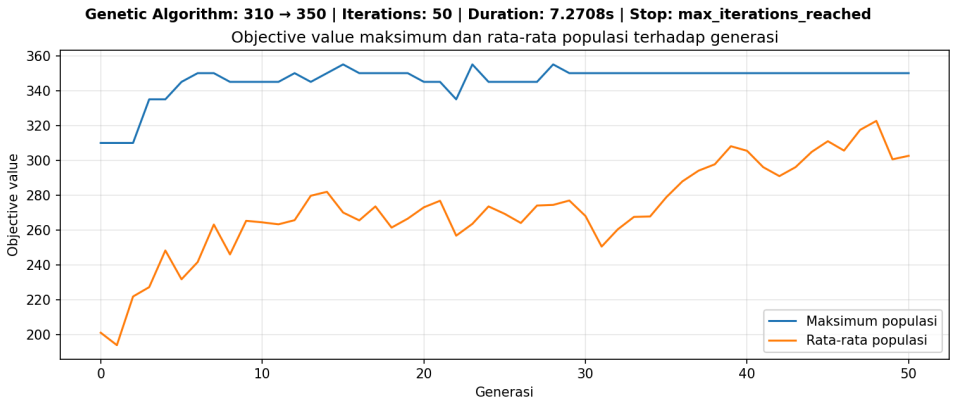
3		

Plot nilai objective function:

Run	Plot
nPopulation=10; nGeneration=50	
1	

2	
3	
nPopulation=20; nGeneration=50	
1	

2	
3	
nPopulation=40; nGeneration=50	
1	

2	
3	

4.5. Hasil Multi-Truck

Eksperimen membandingkan empat varian Hill Climbing, Simulated Annealing, dan Genetic Algorithm pada `input_multi.txt`. Data berisi 14 paket dan tiga truk: T1 berukuran 10 × 10 × 6 dengan kapasitas 40, T2 berukuran 8 × 8 × 5 dengan kapasitas 30, serta T3 berukuran 6 × 12 × 6 dengan kapasitas 35. Total kapasitas berat armada adalah 105, sedangkan berat seluruh paket 146 dan jumlah nilai seluruh paket 1.205. Karena muatan tersebar pada tiga ruang dengan kapasitas dan ukuran berbeda, kelayakan bergantung pada berat, geometri, orientasi, tabrakan, dan aturan fragile.

Pengujian dilakukan dengan cara empat varian Hill Climbing dan Simulated Annealing masing-masing dijalankan tiga kali dengan seed 101, 102, dan 103. State awal untuk setiap seed digunakan bersama oleh varian Hill Climbing dan Simulated Annealing pada seed yang sama. Parameter Hill Climbing adalah 1.000 iterasi untuk Stochastic, maksimal 10 sideways move berturut-turut untuk Sideways Move, dan tiga restart tambahan untuk Random

Restart (maksimal empat pencarian steepest ascent). Simulated Annealing memakai suhu awal terkalibrasi per seed, target penerimaan 0,9 dari 100 sampel, laju pendinginan 0,99, suhu minimum 0,001, serta batas 5.000 iterasi. Genetic Algorithm diuji dalam dua rangkaian dengan tiga run per konfigurasi:

- Populasi tetap 20, dengan 25, 50, dan 100 generasi.
- Generasi tetap 50, dengan populasi 10, 20, dan 40.

Seed GA dibangkitkan berurutan dari generator seed dasar 0. Durasi adalah waktu yang dicatat oleh hasil algoritma; waktu pembentukan state awal dan kalibrasi suhu SA tidak termasuk durasi pencarian SA.

Tabel berikut merangkum nilai objective state akhir. Simpangan baku adalah simpangan baku populasi dari tiga run, sehingga menjadi gambaran variasi pada sampel kecil ini.

Algoritma / konfigurasi	Rata-rata objective akhir	Rentang	Simpangan baku	Rata-rata durasi (detik)
Steepest Ascent	846,67	810–895	35,67	3,732
Sideways Move (maks. 10)	853,33	810–895	34,72	11,116
Stochastic (1.000 iterasi)	736,67	680–840	73,18	0,291
Random Restart (3 restart tambahan)	863,33	840–895	23,21	15,150
Simulated Annealing	810,00	800–830	14,14	0,252
GA, populasi 20, 25 generasi	710,00	685–730	18,71	0,694
GA, populasi 20, 50 generasi	653,33	530–775	100,03	1,290
GA, populasi 20, 100 generasi	673,33	645–730	40,07	2,457
GA, populasi 10, 50 generasi	636,67	525–695	78,99	0,523

GA, populasi 20, 50 generasi	661,67	585–755	70,40	1,256
GA, populasi 40, 50 generasi	723,33	645–790	59,77	3,289

Dalam eksperimen multitruck ini, solusi valid terbaik bernilai 895 dari batas atas berbasis kapasitas berat sebesar 910. Random Restart memberikan rata-rata tertinggi di antara metode lokal, sedangkan Steepest Ascent mendapatkan solusi tunggal terbaik dengan durasi yang jauh lebih rendah. Simulated Annealing paling konsisten dan cepat pada konfigurasi yang diuji, tetapi objective akhirnya lebih rendah. Untuk GA, populasi lebih besar cenderung membantu pada 50 generasi, sementara penambahan generasi dengan populasi tetap 20 tidak memperbaiki rata-rata solusi akhir secara konsisten. Semua kesimpulan berlaku pada input, parameter, dan tiga seed yang diuji; belum ada bukti bahwa nilai 895 adalah optimum global.

4.6. Analisis dan Perbandingan

Berdasarkan eksperimen, **Genetic Algorithm (GA) memperoleh nilai objektif terbaik sebesar 355**, diikuti HC Random Restart sebesar 345, HC Stochastic sebesar 335, dan Simulated Annealing sebesar 305. Namun, karena nilai global optimum yang sebenarnya belum diketahui, nilai 355 belum dapat dipastikan sebagai solusi optimal global. Hasil ini menunjukkan bahwa GA berhasil menemukan solusi terbaik di antara seluruh percobaan yang dilakukan. Mekanisme seleksi, crossover, dan mutasi yang memungkinkan GA mengeksplorasi berbagai kombinasi penempatan paket. Sementara itu, HC Random Restart mencoba kembali pencarian dari titik awal yang berbeda, HC Stochastic memilih langkah secara acak, dan Simulated Annealing memungkinkan penerimaan solusi yang lebih buruk berdasarkan temperatur untuk membantu keluar dari optimum lokal.

Dari rata-rata nilai objektif akhir, HC Random Restart memperoleh hasil tertinggi sebesar 330, diikuti GA sebesar 320,56, HC Stochastic sebesar 293,33, dan Simulated Annealing sebesar 276,67. Dengan demikian, GA tidak memiliki rata-rata hasil tertinggi, tetapi berhasil menemukan nilai terbaik keseluruhan. Perbedaan ini menunjukkan bahwa kemampuan menemukan solusi terbaik tidak selalu berarti menghasilkan solusi terbaik secara konsisten. Perlu diperhatikan bahwa GA diuji sebanyak 18 kali

dengan beberapa konfigurasi, sedangkan setiap algoritma local search diuji tiga kali, sehingga perbandingan rata-rata tersebut belum sepenuhnya setara.

HC Stochastic merupakan algoritma tercepat dengan rata-rata durasi sekitar 0,50 detik, diikuti Simulated Annealing sebesar 2,35 detik, HC Random Restart sebesar 4,70 detik, dan GA sebesar 5,08 detik. Durasi GA dipengaruhi oleh jumlah individu dalam populasi serta jumlah generasi yang diproses. Pada setiap generasi, GA melakukan seleksi, crossover untuk membentuk individu baru, dan mutasi. Proses tersebut membutuhkan komputasi lebih banyak dibandingkan pencarian dengan satu solusi aktif.

HC Random Restart memiliki rentang nilai akhir terkecil, yaitu 35, dari nilai 310 hingga 345, sehingga hasilnya relatif paling konsisten di antara ketiga algoritma local search yang diuji. Sebaliknya, HC Stochastic memiliki rentang 80, sedangkan Simulated Annealing memiliki rentang 75. Pada GA, konsistensi bergantung pada konfigurasi yang digunakan: populasi 20 dengan 25 generasi menghasilkan rentang nilai akhir hanya 10, sedangkan populasi 20 dengan 50 generasi pada eksperimen variasi populasi menghasilkan rentang 55. Variasi ini menunjukkan bahwa hasil algoritma dipengaruhi oleh proses acak dan konfigurasi eksperimen. Selain itu, kode GA belum menerapkan elitism, yaitu mempertahankan individu terbaik ke generasi berikutnya. Akibatnya, solusi yang pernah ditemukan dapat hilang dari populasi akhir.

Ketika ukuran populasi dipertahankan sebesar 20, peningkatan jumlah generasi dari 25 menjadi 50 dan 100 meningkatkan rata-rata nilai akhir dari 305 menjadi 320 dan 348,33. Namun, rata-rata durasi juga meningkat dari 1,84 menjadi 2,88 dan 8,11 detik. Ini menunjukkan bahwa penambahan generasi memberikan kesempatan lebih banyak untuk memperbaiki solusi, tetapi membutuhkan waktu lebih lama. Sementara itu, ketika jumlah generasi dipertahankan sebesar 50, peningkatan populasi dari 10 menjadi 20 dan 40 menghasilkan rata-rata nilai akhir masing-masing 316,67, 308,33, dan 325. Hasilnya tidak meningkat secara monoton, meskipun durasi rata-rata cenderung bertambah. **Jadi, penambahan generasi cenderung meningkatkan kualitas solusi pada eksperimen ini, sedangkan penambahan populasi tidak selalu menjamin hasil yang lebih baik.**

BAB 5
KESIMPULAN DAN SARAN

5.1. Kesimpulan

Berdasarkan hasil eksperimen, GA mampu menghasilkan solusi dengan nilai terbaik 355 dan rata-rata nilai akhir 320,56. Hasil ini lebih baik dibandingkan HC Stochastic dan Simulated Annealing, tetapi masih di bawah HC Random Restart yang memiliki rata-rata 330. GA membutuhkan waktu eksekusi yang relatif lebih lama, tetapi peningkatan jumlah generasi cenderung menghasilkan solusi yang lebih baik.

5.2. Saran

Pengembangan selanjutnya dapat dilakukan dengan menerapkan elitism agar solusi terbaik dari generasi sebelumnya tidak hilang. Selain itu, jumlah percobaan sebaiknya disamakan agar perbandingan antar algoritma lebih adil dan hasilnya lebih representatif.

DAFTAR PUSTAKA

Matplotlib Development Team. (2025). Matplotlib 3.10 Documentation. https://matplotlib.org/stable/

Russell, S. J., & Norvig, P. (2022). Artificial Intelligence: A Modern Approach (4th ed.). Pearson.

Tim Pengajar IF3170. (2026). Beyond Classical Search [Salindia kuliah]. IF3170 Inteligensi Artifisial, Program Studi Teknik Informatika, Institut Teknologi Bandung.

Khodra, M. L. (2021). Beyond Classical Search: Classical vs Local Search [Video]. YouTube. https://www.youtube.com/watch?v=oGTa9kGE3P8

Khodra, M. L. (2021). Beyond Classical Search: Genetic Algorithm [Video]. YouTube. https://www.youtube.com/watch?v=aI41TGo9J9c

Khodra, M. L. (2021). Beyond Classical Search: Hill-climbing Search [Video]. YouTube. https://www.youtube.com/watch?v=L0b0CN3Fds8

Khodra, M. L. (2021). Beyond Classical Search: Simulated Annealing [Video]. YouTube. https://www.youtube.com/watch?v=AdhG7HnhYrk

Khodra, M. L. (2021). Beyond Classical Search: State (Value, Neighbor) [Video]. YouTube. https://www.youtube.com/watch?v=_6rla37QorM

LAMPIRAN

- Pembagian Tugas Anggota

Nama	**NIM**	**Kontribusi**
Aziza Dharma Putri	13524017	- Class Neighbor - Implementasi seluruh variasi algoritma HC - Implementasi Visualisasi
Suryani Mulia Utami	13524042	- Class entity (Package, Truck) - Class State - Class Parser - Implementasi Genetic Algorithm
Alya Nur Rahmah	13524081	- Implementasi algoritma Simulated Annealing - Implementasi Multi-Truck

- Form Penggunaan Generative AI

  Surat Pernyataan Penggunaan AI